OWASP
Smart Contract Security

scs.owasp.org ↗

## Security Handbook :

# EVM Forensics and DeFi Recovery

04

Security Handbook Series

GRANT SUPPORT • 2025-2026

ecosystem support program

Supported by Ethereum Foundation's Ecosystem Support Program (ESP)

# Contents

# Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

# EVM Forensics and DeFi Recovery

## About This Handbook

**EVM Forensics and DeFi Recovery** is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

# Copyright and License



# Author and Creative Credits

PROJECT LEAD

## Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

## Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

## Siddharth Neekher

Lead UI Designer @ CredShields

**OWASP Smart Contract Security Project**

## Purpose and Scope

This handbook is a field guide to post-incident analysis of Ethereum Virtual Machine (EVM) exploits, from the moment a responder learns an incident occurred through evidence collection, root-cause analysis, attribution, and the recovery decisions that follow. It organizes root cause around a four-tier framework built from systematic review of public post-mortems: Tier 1 protocol logic design flaws, Tier 2 lifecycle and governance issues, Tier 3 external dependencies such as oracles and bridges, and Tier 4 classic implementation bugs like reentrancy and broken access control. Real incidents rarely fit one tier. Euler Finance's March 2023 loss combined a Tier 4 regression in its donation-attack health check with governance timing pressure on the fix, and the handbook treats exploit chains that span tiers as the norm rather than the exception. It also covers the analysis specialties that general incident response guidance skips: precision and rounding forensics in the style of Balancer's stable-pool exploits, oracle and flash-loan attack reconstruction, and bridge and cross-chain forensics for a category that MITRE AADAPT and multiple industry loss reports rank as Web3's largest single source of stolen value, from Ronin in March 2022 to Wormhole the same month to Nomad in August 2022.

The scope extends past analysis into what a protocol team can actually do once root cause is known. Non-atomic attacks, ones spread across multiple transactions or blocks rather than a single flash-loan sandwich, open a rescue window that skilled responders have used to intercept funds before a hostile discovery does. The Poly Network incident of August 2021, where roughly six hundred million dollars in assets were returned after direct negotiation with the attacker, sits alongside the handbook's DeFi Recovery Patterns Database: pause and upgrade, governance and timelock parameter fixes, compensation design, fork and migration, negotiation, and pre-authorized whitehat rescue under a published safe harbor agreement. The handbook supports incident response teams, protocol engineers, and forensic researchers, and it complements the Incident Response Handbook (06), which owns the broader response process this handbook's forensic and recovery work feeds into.

## Target Audience

Security specialists, incident responders, blockchain forensics analysts, protocol engineering teams, and auditors conducting root-cause or post-mortem analysis on EVM-based incidents.

## How to Use This Handbook

Start with Part I to fix the four-tier root-cause framework, the legal and ethical ground rules, and the evidence-first mindset that governs everything that follows: collect before you change state. Parts II and III are the working core, covering on-chain and off-chain evidence preservation and then the analysis techniques (transaction tracing, contract and vulnerability analysis, attribution, and multi-chain and bridge forensics) that turn raw evidence into a root-cause finding. Part IV is the recovery reference: read it once end to end before an incident, then return to the specific pattern (pause and upgrade, governance fix, compensation, fork, negotiation, or whitehat safe harbor) that fits the situation at hand. Part V catalogs tools and automation for repeatable evidence collection. A responder mid-incident should go straight to the evidence checklist in Appendix B and the tool references in Appendix D, then work back into Parts II and III as the picture clarifies.

## Relationship to SCSVS, SCSTG, and SCWE

This handbook is the forensic and recovery counterpart to OWASP's smart contract standards, applied after a control has already failed. The Smart Contract Security Verification Standard (SCSVS) defines the controls an exploited contract should have had; Part III maps each exploited weakness back to the SCSVS control that would have caught it. The Smart Contract Security Testing Guide (SCSTG) describes how to test for a vulnerability before deployment; this handbook describes how to reconstruct that same vulnerability class after it has already been exploited on-chain, closing the loop from missed test case to confirmed root cause. The Smart Contract Weakness Enumeration (SCWE) catalogs the weakness taxonomy this handbook's four-tier framework and Part III mapping rely on directly. The companion Incident Response Handbook (06) owns the organizational response process (triage, communications, stakeholder coordination) that this handbook's evidence collection and recovery patterns feed into, and the Web3 Attack Vectors Mapping Handbook (10) places the exploited weaknesses this handbook analyzes within the broader WA01 to WA15 attack taxonomy.

# Part I: Foundations

This part fixes the vocabulary a responder needs before touching a single transaction: what blockchain forensics is for, the research-derived four-tier framework used to classify root cause throughout the handbook, and the legal, ethical, and procedural rules that decide whether evidence collected today survives contact with a courtroom, a regulator, or an insurer later. Everything downstream, evidence collection in Part 2, analysis technique in Part 3, and recovery decisions in Part 4, depends on getting these three things right first.

## 1. Introduction to EVM Forensics

This chapter sets the vocabulary the rest of the handbook depends on: what blockchain forensics is for, how it fits alongside incident response and the OWASP smart contract standards, and the four-tier root-cause framework used to classify every incident discussed later.

### 1.1 Goals of Blockchain Forensics

Blockchain forensics reconstructs what happened during a security incident on a distributed ledger: which transactions moved which assets, in what order, through which entry point, and why the contract logic allowed it. On the Ethereum Virtual Machine (EVM), the execution model shared by Ethereum and dozens of compatible chains including Polygon, Arbitrum, Optimism, BNB Chain, and Avalanche's C-Chain, the goals mirror traditional digital forensics as codified in NIST Special Publication 800-86, Guide to Integrating Forensic Techniques into Incident Response: identify, preserve, analyze, and present evidence so it supports a technical root cause and, where relevant, a legal or insurance process. Root cause analysis (RCA) answers what broke and why; attribution (Chapter 9) answers who did it and how confidently; recovery support (Part 4) answers what can still be done about it.

Two properties separate EVM forensics from a typical host intrusion investigation. First, the primary evidence source, the chain itself, is public and append-only, available to attacker and investigator alike. You are not hunting for logs an intruder might have wiped; you are querying a dataset the adversary cannot alter after the fact. Second, that same transparency enables determ-

inistic replay in a way host forensics rarely allows: given the exact state at the block before an exploit transaction, a forked EVM node re-executes it and reproduces every storage write, byte for byte (Section 3.2). What blockchain forensics does not get for free is identity. Addresses are pseudonymous, so attribution draws on an entirely different evidentiary chain than technical root cause, built from behavioral patterns and off-chain intelligence rather than storage diffs.

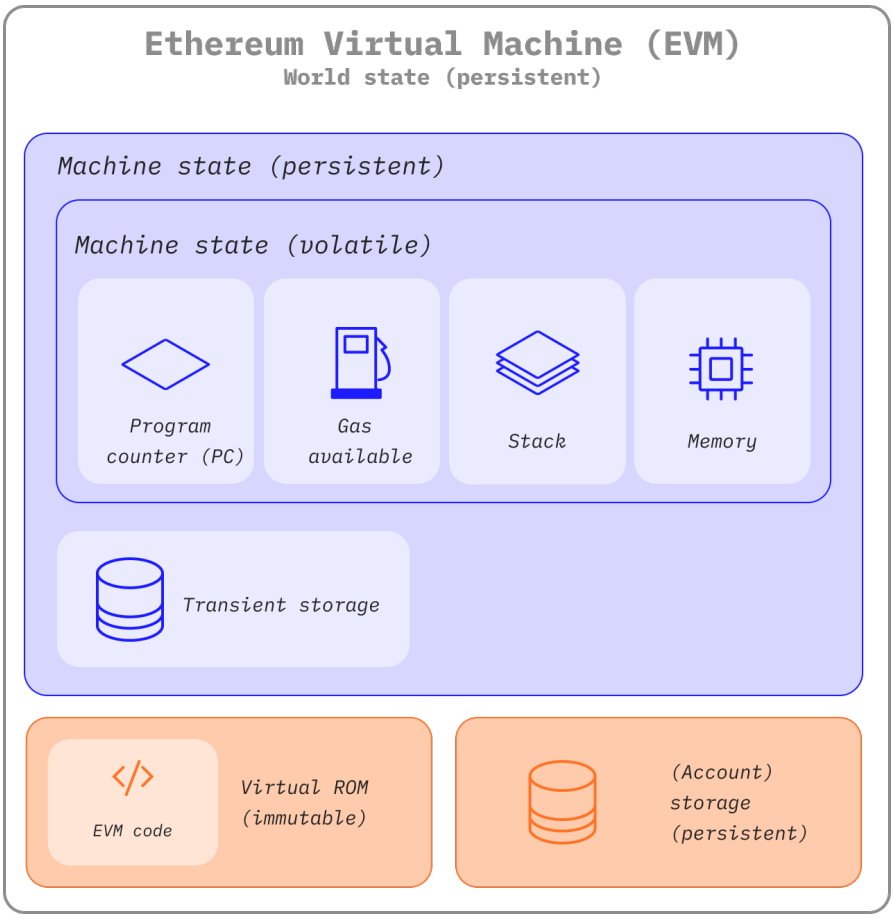


*Figure. The EVM state surfaces an investigator must distinguish during replay. Stack, memory, program counter, and gas exist only inside an execution frame; transient storage survives internal calls only for the transaction; account storage and world state persist and are committed into the post-state root. Source: ethereum.org, Ethereum Virtual Machine, CC BY 4.0.*

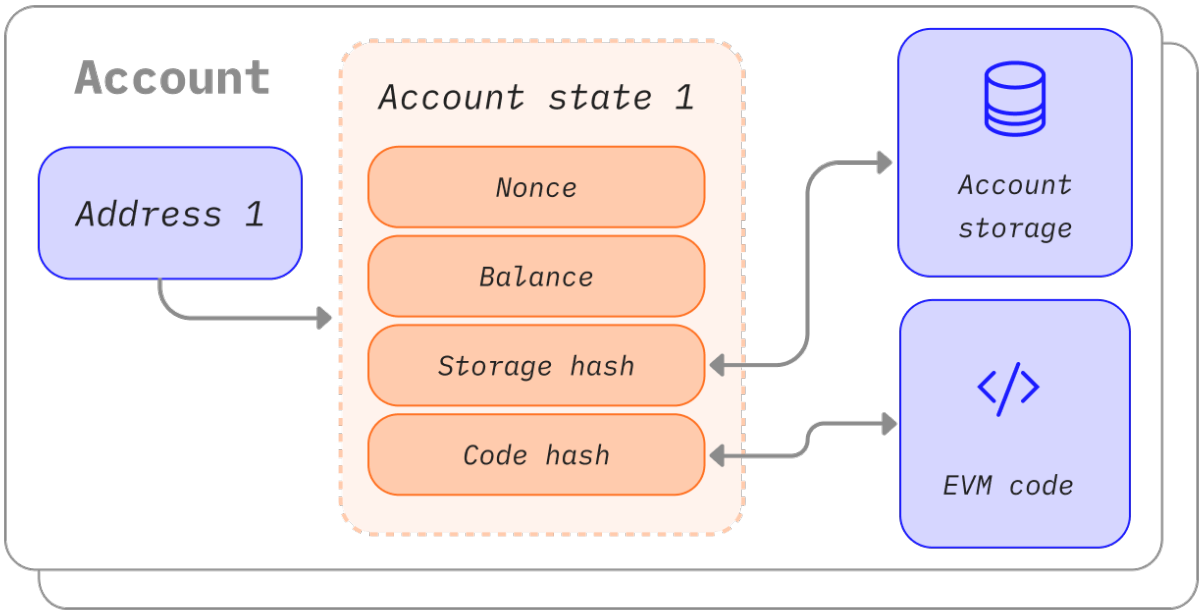


*Figure. Ethereum has two account types with different evidentiary meaning. An externally owned account's action is authorized by a signature, while a contract account acts only because prior execution reached its code. Labeling both simply as "wallets" erases the distinction between an initiating actor and an automated hop in the trace. Source: ethereum.org, Ethereum Accounts, CC BY 4.0.*

## 1.2 Relationship to Incident Response

Forensics is a technical discipline; incident response (IR) is the organizational process that surrounds it, comms, stakeholder coordination, and decision authority. The NIST SP 800-61 Revision 3 framework, finalized in April 2025 as a Cybersecurity Framework (CSF) 2.0-aligned community profile, organizes response around four functions: Govern, Identify and prepare, Detect and analyze, and Respond and recover. This handbook's forensic work sits almost entirely inside Detect and analyze, tracing the technical mechanism of the attack, and feeds directly into Respond and recover, where the root cause determines which recovery pattern (Part 4) is even feasible.

The companion Incident Response Handbook (06) owns the broader process: who declares an incident, who talks to users, when to engage a war room, and how the organization learns afterward. Treat that handbook as the outer loop and this one as the inner loop it calls into whenever an EVM-based incident needs technical reconstruction. A responder working from Handbook 06's runbook should expect to hand off to this handbook's Part 2 evidence checklist the moment an on-chain event is confirmed, and to hand a finished root-cause finding back before recovery decisions in Part 4 are made.

## 1.3 Relationship to SCSVS, SCSTG, and SCWE

This handbook is the forensic and recovery counterpart to OWASP's smart contract standards, applied after a control has already failed rather than before one is deployed. The Smart Contract Security Verification Standard (SCSVS) defines the verifiable controls a contract should have im-

plemented; Section 8.2 of this handbook maps each exploited weakness back to the specific SCSVS control whose absence or failure permitted it. The Smart Contract Security Testing Guide (SCSTG) describes how to test for a vulnerability class before it ships; this handbook describes how to reconstruct that same vulnerability class after it has already been exploited on-chain, closing the loop from missed test case to confirmed root cause. The Smart Contract Weakness Enumeration (SCWE) catalogs the taxonomy of weaknesses; the four-tier framework in Section 1.5 is built to cross-reference SCWE entries at both the tier and sub-category level, so a forensic finding is never a one-off label but a pointer into a shared, auditable taxonomy.

| Standard | What it defines | This handbook's relationship |
|---|---|---|
| SCSVS | Verifiable security controls a contract should implement | Section 8.2 maps each exploited weakness to the control that would have caught it |
| SCSTG | Test procedures to catch vulnerabilities pre-deployment | This handbook reconstructs the same vulnerability classes post-exploit |
| SCWE | Taxonomy of smart contract weaknesses | The four-tier framework (Section 1.5) cross-references SCWE entries per tier |

The OWASP Web3 Attack Vectors Top 15 supplies a third axis: it frames the same weaknesses as attacker-facing vectors (bridge compromise as WA02, oracle manipulation, and so on), and the Web3 Attack Vectors Mapping Handbook (10) is where that full WA01 through WA15 taxonomy is placed against this handbook's findings.

## 1.4 How to Use This Handbook

Read this Part once, end to end, to internalize the four-tier framework and the evidence-first discipline in Chapter 3; after that, treat the handbook as a reference you jump into mid-incident. Parts 2 and 3 are the working core: on-chain and off-chain evidence collection, then the analysis techniques, transaction tracing, contract and vulnerability analysis, attribution, and cross-chain forensics, that turn raw evidence into a defensible root-cause finding. Part 4 is the recovery reference, meant to be read once before an incident and then consulted for the specific pattern that fits the situation at hand. Part 5 catalogs tools and scripts for repeatable evidence collection.

If you are mid-incident right now, do not start at Chapter 1. Go straight to the evidence checklist in Appendix B and the tool references in Appendix D, execute Chapter 3's collect-before-changing-state rule, then work backward into Parts 2 and 3 as the picture clarifies. Return to this Part only when you need the legal grounding in Chapter 2 or the tier classification in Section 1.5 to write up the finding.

## 1.5 Four-Tier Root Cause Framework

A one-off narrative of what went wrong is not repeatable across incidents or comparable across a team's caseload. This handbook uses a four-tier framework so that "root cause" always means the same thing regardless of who wrote the finding: a placement in one or more of four categories, each with its own detection approach, its own SCWE cross-references, and its own set of realistic recovery options once the mechanism is understood.

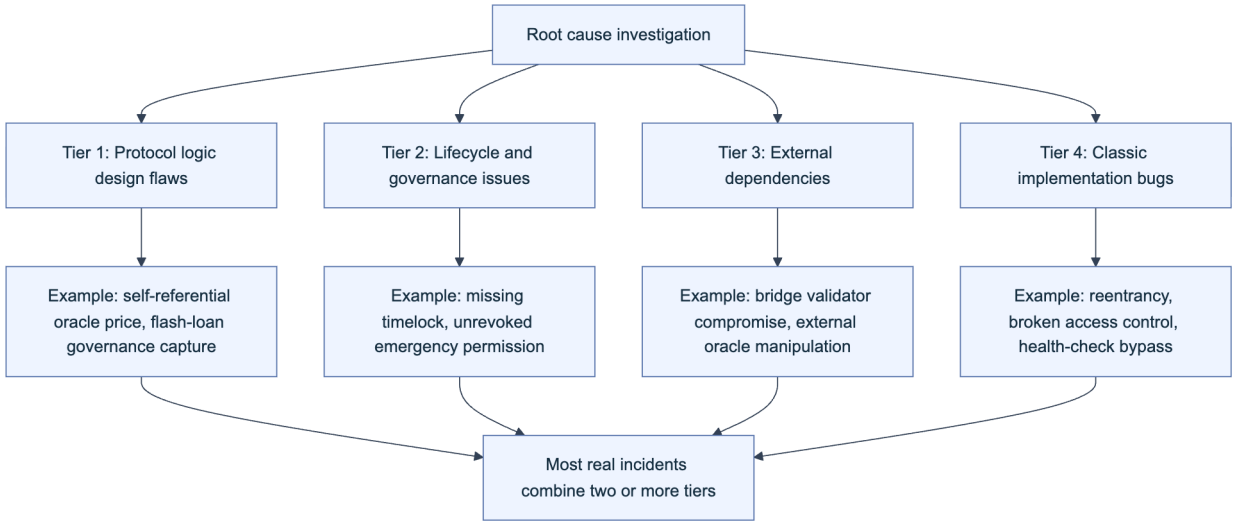


*Figure 1. The four-tier root cause framework. Every finding in Part 3 is classified against one or more of these tiers; Section 1.5.6 covers why most real incidents span more than one.*

| Tier | Focus | Representative incident |
|---|---|---|
| Tier 1 | Protocol logic design flaws | Mango Markets oracle self-manipulation, Oct 2022 |
| Tier 2 | Lifecycle and governance issues | Beanstalk flash-loan governance capture, Apr 2022 |
| Tier 3 | External dependencies (oracles, bridges, off-chain) | Ronin Bridge validator compromise, Mar 2022 |
| Tier 4 | Classic implementation bugs | The DAO reentrancy, Jun 2016 |

### 1.5.1 Provenance: Research-Derived Taxonomy (Systematic Literature and Incident Analysis)

The framework is not an ad hoc list; it is built the way MITRE's ATT&CK matrix was built, from systematic observation rather than theory. Academic security research has independently converged on a similar structure. "SoK: Decentralized Finance (DeFi) Attacks" (Zhou et al., IEEE Symposium on Security and Privacy 2023, arXiv:2208.13035) reviewed 77 academic works alongside roughly 30 real-world incidents totaling over $3 billion in reported losses, and derived

a common reference frame that separates conceptual and design-level flaws from implementation-level bugs, the same separation this framework's Tier 1 and Tier 4 encode. MITRE's AADAPT project (Adversarial Actions in Digital Asset Payment Technologies) extends the ATT&CK methodology specifically to digital-asset and DeFi adversary behavior, cataloging observed techniques rather than hypothesized ones.

This handbook's four tiers were cross-checked against both bodies of work and against a broader corpus of public post-mortems, incident post-mortem threads published by the exploited protocols themselves, third-party forensic write-ups, and industry loss aggregators such as Chainalysis's annual crypto crime reporting. The practical implication for a responder: when a finding does not cleanly fit one tier, that is a signal to keep investigating rather than a framework failure, because the literature this taxonomy is drawn from shows that clean single-tier incidents are the minority (Section 1.5.6).

### 1.5.2 Tier 1: Protocol Logic Design Flaws

Tier 1 covers flaws in the intended mechanism itself, not a coding mistake but a design that behaves exactly as written and is still exploitable. The clearest examples come from price and incentive design. In October 2022, Avraham Eisenberg opened large offsetting positions in Mango Markets' own MNGO perpetual futures market, used the resulting inflated mark price as collateral, and borrowed out roughly $114 million from the protocol's treasury, a strategy he later publicly described as legal trading rather than hacking; he was convicted of fraud and market manipulation by a US federal jury in April 2024. The design flaw was that the protocol trusted a price it could itself be used to move.

In April 2022, an attacker used a single flash-loaned position to acquire a supermajority of Beanstalk's governance token supply for one transaction, pass a malicious proposal instantly because no timelock delayed emergency governance actions, and drain roughly $182 million. Both incidents share a Tier 1 signature: no line of code was "buggy" in isolation, the mechanism (oracle price, governance weight) did exactly what it was designed to do, and the design failed to account for an attacker temporarily controlling that mechanism's input. Detecting Tier 1 issues requires modeling the protocol's economic assumptions, not just its code paths, which is why Section 8.5's oracle and flash-loan forensics and the Economic Design guidance referenced there sit downstream of this tier.

### 1.5.3 Tier 2: Lifecycle and Governance Issues

Tier 2 covers the space between deployment and steady-state operation: upgrade sequencing, permission scope that outlives its purpose, and governance mechanics that are correct in isolation but dangerous under time pressure. A recurring pattern is the emergency permission granted for a specific operational need and never revoked once that need passed, exactly the kind of standing access this handbook's evidence procedures (Chapter 4) are built to reconstruct after the fact by diffing role-grant events against the timeline of actual use.

Governance timing is the other recurring failure mode. A timelock long enough to let token holders react to a malicious proposal is also long enough to let an active exploit continue draining funds while a legitimate emergency fix waits in queue, a tension Part 4's governance recovery pattern (Chapter 13) treats directly. Tier 2 findings are frequently the hardest to communicate to a non-technical stakeholder because there is no single malicious line of code to point at; the finding is closer to "the protocol's own change-management process created the window," and the evidence trail lives as much in governance forum posts, multisig transaction history, and deployment logs as in the exploited contract itself.

### 1.5.4 Tier 3: External Dependencies (Oracles, Bridges, Off-Chain)

Tier 3 covers failures in systems a protocol depends on but does not fully control: price oracles, cross-chain bridges and message-passing layers, and off-chain keepers or relayers. This tier accounts for a disproportionate share of total value lost industry-wide. Chainalysis's 2022 reporting found that cross-chain bridge exploits alone accounted for roughly two-thirds of that year's total stolen crypto value (Chainalysis, "2022 Biggest Year Ever for Crypto Hacking"), a concentration Chapter 10 examines in full.

Three incidents anchor this tier for the rest of the handbook. In March 2022, attackers compromised five of nine validator signatures securing the Ronin Bridge, partly through a spear-phishing campaign against a Sky Mavis employee and partly through an emergency RPC permission granted to the Axie DAO in 2021 and never revoked, and drained roughly $625 million; the FBI publicly attributed the intrusion to the Lazarus Group in an April 2022 statement. In February 2022, a flaw in Wormhole's signature verification on the Solana side let an attacker mint roughly 120,000 wrapped ETH without depositing matching collateral, a loss of roughly $325 million that Jump Crypto backstopped directly. In August 2022, Nomad Bridge shipped an upgrade that initialized its trusted message root to a value treated as already-proven, letting anyone replay a copied message with a different recipient; the result was a chaotic, crowd-sourced drain of roughly $190 million. All three are Tier 3 because the exploited logic sat in the validation of external input, not in the protocol's own internal accounting.

### 1.5.5 Tier 4: Classic Smart Contract Implementation Bugs

Tier 4 covers the vulnerability classes every Solidity engineer is trained to check for: reentrancy, integer overflow and underflow (largely mitigated by default since Solidity 0.8's built-in checked arithmetic), missing or misordered access control, and unchecked external call return values. This is the oldest tier, dating to The DAO's reentrancy exploit in June 2016, roughly $60 million in ether drained through a recursive external call that withdrew funds before the contract's internal balance was updated, an incident consequential enough that it led to the Ethereum and Ethereum Classic chain split.

Tier 4 bugs remain common not because the classes are unknown but because a correct-looking regression can reintroduce them. Euler Finance's March 2023 loss of roughly $197 million originated in a `donateToReserves` function that let an attacker donate collateral in a way that bypassed the protocol's health check, a Tier 4 implementation regression in check ordering rather than a novel technique. Detecting Tier 4 issues is what static analysis and the SCSTG's test-driven guidance are built for, which is exactly why a confirmed Tier 4 root cause is the strongest evidence that a specific SCSVS control was either absent or not enforced (Section 8.2).

### 1.5.6 Exploit Chains: Multiple Tiers in Real Incidents

Treating a finding as finished the moment one bug is confirmed is the most common forensic mistake this handbook warns against. The literature surveyed in Section 1.5.1 and this handbook's own incident review both show that high-value exploits typically chain two or more tiers together, and a root-cause writeup that stops at the first tier understates both the severity and the fix required.

Euler Finance is the clearest teaching example. The initial loss was a pure Tier 4 bug, the `donateToReserves` health-check bypass. What turned a serious bug into a near-total loss and a difficult recovery was Tier 2: Euler's governance required a DAO vote and timelock delay to ship a fix, a process with a multi-day floor that could not outrun an attacker already holding the drained funds.

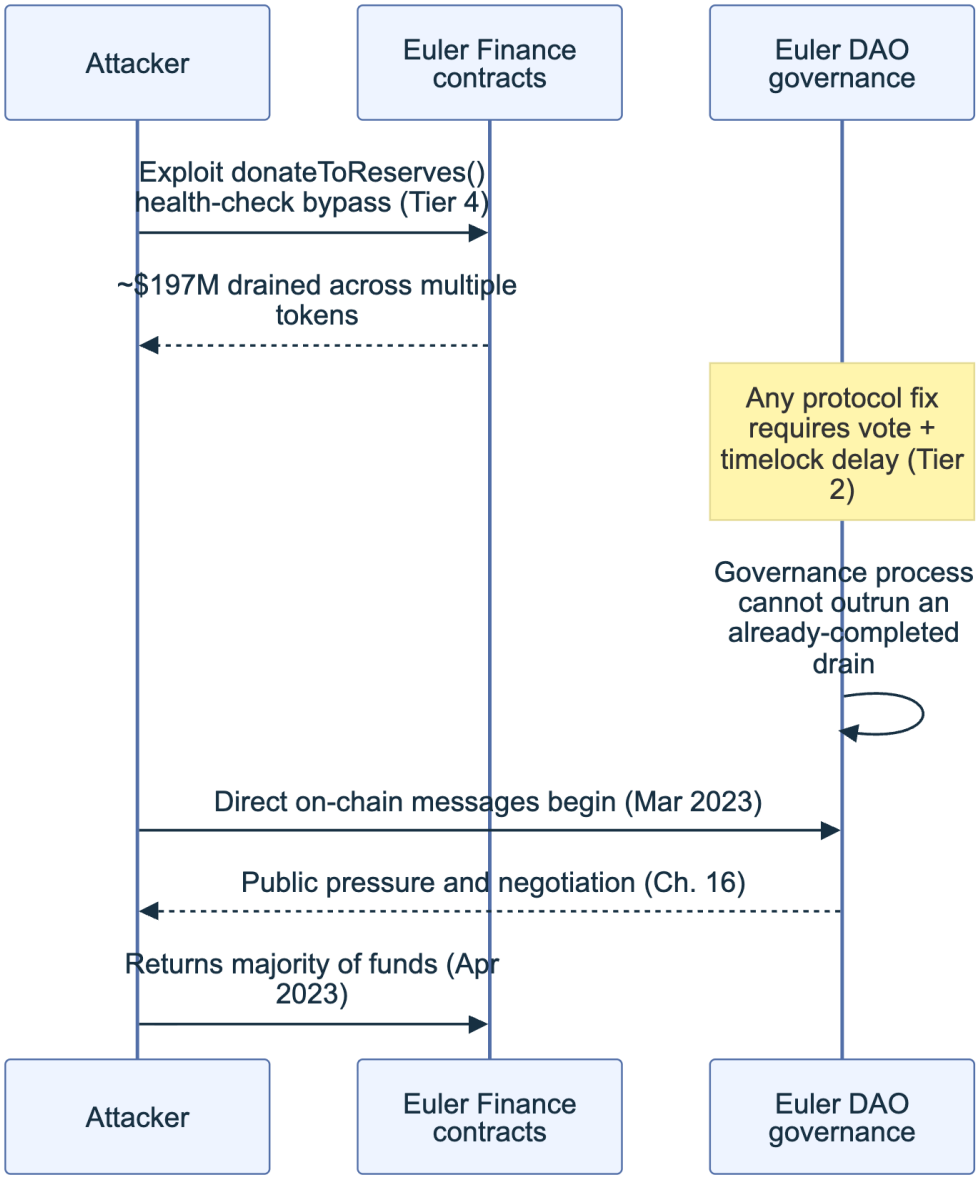


*Figure 2. Euler Finance as a two-tier exploit chain. The Tier 4 bug caused the loss; the Tier 2 governance constraint shaped what could be done about it in the following weeks, which is why the eventual recovery ran through negotiation (Chapter 16) rather than a fast on-chain fix.*

The forensic implication is procedural: after confirming a first-tier root cause, explicitly ask whether a second tier shaped either the attack's scale or the response's constraints, and document both. Chapter 10's bridge analysis (frequently Tier 3 plus Tier 4, an external validation failure paired with an internal accounting bug) and Part 4's recovery pattern selection both assume this multi-tier view as the default, not the exception.

# 2. Legal and Ethical Considerations

Before touching a single transaction, a responder needs to know what makes evidence usable later, in a courtroom, a regulator's inbox, or an insurer's claims process, and where the ethical lines sit around privacy, disclosure, and law enforcement contact.

## 2.1 Evidence Admissibility and Chain of Custody

Evidence collected without a defensible chain of custody is evidence a court, insurer, or counterparty can dispute regardless of its technical accuracy. RFC 3227, Guidelines for Evidence Collection and Archiving, remains the baseline reference for the underlying discipline: collect in order of volatility, document who collected what and when, minimize interaction with the evidence itself, and preserve the original alongside any copy used for analysis. In a US legal context, Federal Rule of Evidence 901 requires that evidence be authenticated, that its proponent produce evidence sufficient to show the item is what it is claimed to be, which for on-chain data means a hash-verified export tied to a documented collection process, not a screenshot with no provenance.

Maintain a chain-of-custody log as a first-class artifact from the first minute of an incident, not a document assembled afterward from memory.

| Field | Purpose |
|---|---|
| Artifact ID | Unique reference for the collected item (tx hash, log export, screenshot) |
| Collected by | Named individual, not a shared team account |
| Collection timestamp (UTC) | When the artifact was captured, independent of the event's own timestamp |
| Source | RPC endpoint, indexer, or system the artifact came from |
| Hash (SHA-256) | Cryptographic fingerprint computed at the moment of collection |
| Storage location | Where the artifact is archived, write-once where possible (Section 5.3) |
| Access log | Who has viewed or copied the artifact since collection |

## 2.2 Privacy and Data Handling

On-chain transaction data is public by design, but a forensic investigation quickly touches data that is not: know-your-customer (KYC) records held by exchanges, RPC and application logs that may carry IP addresses, and personally identifiable information (PII) collected during victim outreach or a later compensation process (Chapter 14). That data falls under ordinary privacy regulation regardless of the blockchain context, including the EU's General Data Protection Regulation (GDPR) and comparable regimes elsewhere.

Treat privacy as a collection-time discipline, not an afterthought: collect only the PII a specific step of the investigation requires, document the legal basis and retention period for anything you do collect, and apply the same access controls and retention limits described in Section 5.4 to off-chain personal data as to any other evidence artifact. A compensation process that requires collecting user wallet-to-identity mappings (Section 14.1) is the point in this handbook's workflow where privacy exposure is highest, and it is the point where legal counsel (Section 2.3) should be involved before collection begins, not after.

## 2.3 Coordination with Law Enforcement and Legal Counsel

Deciding whether and when to involve law enforcement is a legal decision, not a technical one, and it should never be made unilaterally by the responder who found the bug. Route it through counsel first. In the United States, the FBI's Internet Crime Complaint Center (IC3) is the standard reporting channel for cybercrime affecting US persons or entities; equivalent national channels exist across other jurisdictions (Europol's cybercrime reporting for the EU, and national CERTs elsewhere). The Ronin Bridge case (Section 1.5.4) shows the coordination working as intended: the FBI's public attribution to the Lazarus Group and the US Treasury's subsequent sanctions action against the associated wallet address gave exchanges and counterparties a documented basis to freeze related funds, something no individual protocol team could have accomplished alone.

Sanctions screening deserves its own line item because it gates recovery, not just disclosure. Before facilitating any movement of funds, whether returning assets to users or negotiating with an attacker (Chapter 16), screen every counterparty address against the US Office of Foreign Assets Control (OFAC) Specially Designated Nationals (SDN) list; moving funds to or from a sanctioned address exposes the protocol and its team to liability independent of the original incident.

**Checklist: engaging law enforcement and counsel** - Confirm losses and evidence meet the threshold legal counsel sets for external reporting. - Engage outside counsel before any public statement or first law enforcement contact. - File through the appropriate national channel (IC3 in the US, Europol or national CERT elsewhere). - Screen every recipient and source address against the OFAC SDN list before any fund movement. - Share raw evidence only as counsel directs; preserve the full set independently. - Log every law enforcement contact: agency, agent or case officer, case number, date.

## 2.4 Responsible Disclosure and Attribution

Premature or overconfident public attribution creates two distinct risks: it can tip off an active attacker before funds are frozen, and it can wrongly implicate an address or entity based on incomplete evidence, exposure this handbook's attribution confidence levels (Section 9.4) exist specifically to prevent. Coordinate disclosure timing with the parties who can act on it, exchanges that can freeze an attacker's deposit, bridges that can pause a compromised route, before any

public statement, following the same responsible-disclosure instinct that governs vulnerability reporting generally but applied to an incident already in progress rather than a pre-disclosure bug.

Attribution claims made in a public post-mortem should be sourced and confidence-graded the same way a threat intelligence report would be: distinguish a claim backed by on-chain clustering and independently corroborated off-chain intelligence from a claim based on a single behavioral similarity. Chapter 9 develops this distinction in full; the ethical rule to carry forward from this chapter is that attribution is a statement about evidence strength, not a statement of certainty, and should never be dressed up as more confident than the underlying evidence supports.

---

## 3. Evidence-First Mindset

The single most consequential decision in the first hour of an incident is procedural, not technical: whether to collect evidence before or after changing anything on-chain or off. This chapter makes the case for collect first, always.

### 3.1 Collect Before Changing State

Every state-changing action available to a responder, pausing a contract, upgrading a proxy, revoking a compromised role, moving remaining funds to safety, also destroys or alters the exact evidence a root-cause finding depends on. A pause transaction changes contract state going forward; it does nothing to preserve the state as it existed the moment before the exploit, which is what a forensic reconstruction actually needs (Section 3.2). Acting first and documenting later inverts the priority this handbook is built around.

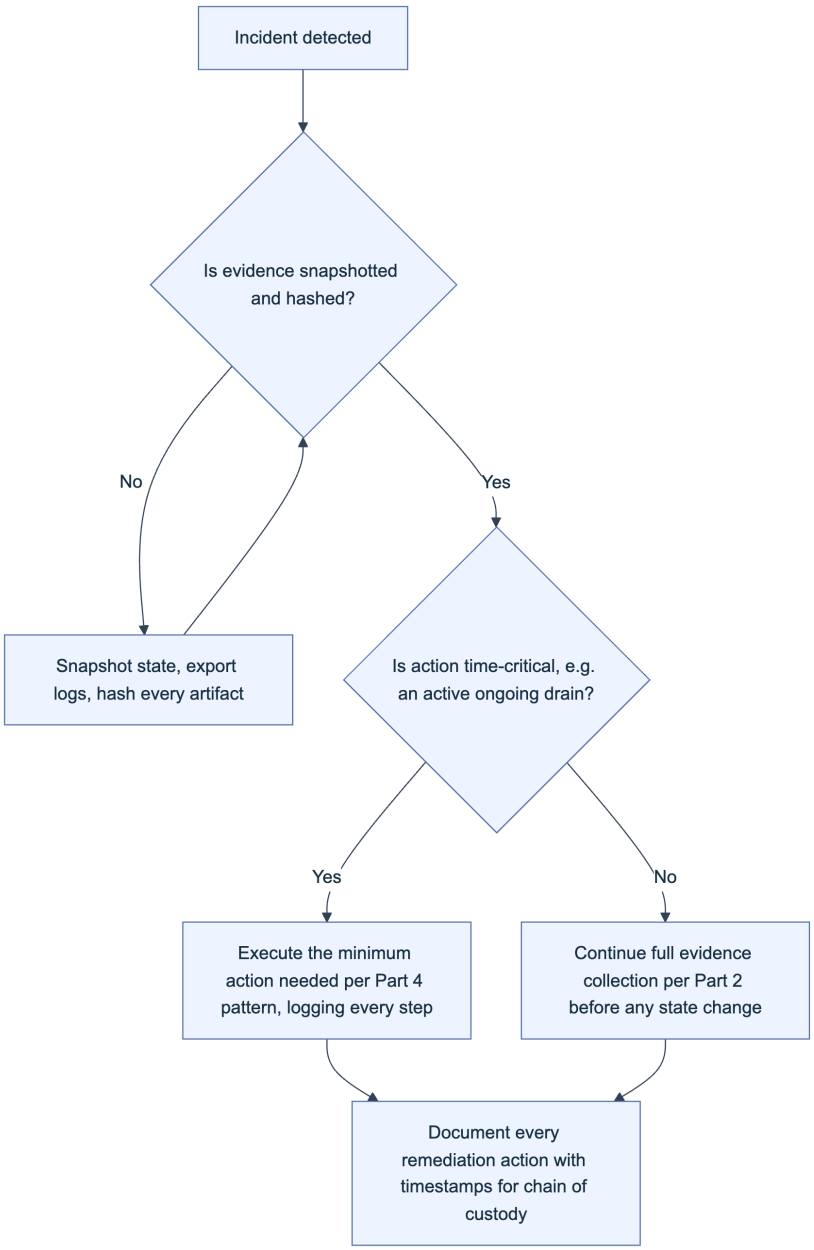


*Figure 3. The evidence-first decision flow. Even a time-critical response passes through evidence capture first; the only branch that skips ahead is an active, ongoing drain, and even that branch logs every action taken.*

**Checklist: before you touch anything** - Do not pause or upgrade the contract before state is snapshotted and hashed. - Do not revoke attacker-facing permissions that are still producing traceable on-chain evidence. - Do not issue a public statement before legal and communications alignment (Section 2.4). - Do not move remaining funds without OFAC screening and counsel sign-off (Section 2.3). - Do capture the full mempool and surrounding block context immediately, before triage, not after.

## 3.2 Immutability and Reproducibility of On-Chain Data

The append-only property of a blockchain is the forensic investigator's single biggest structural advantage over traditional host forensics: the record an attacker created cannot be retroactively edited, and every honest node holds the same history. That immutability translates directly into reproducibility. Given the exact block immediately preceding an exploit, a forked EVM node, using tooling such as Foundry or Hardhat (Section 7.4), re-executes the exploit transaction deterministically and reproduces every storage write, log, and internal call, which is what makes the reproduction techniques in Section 8.6 possible at all.

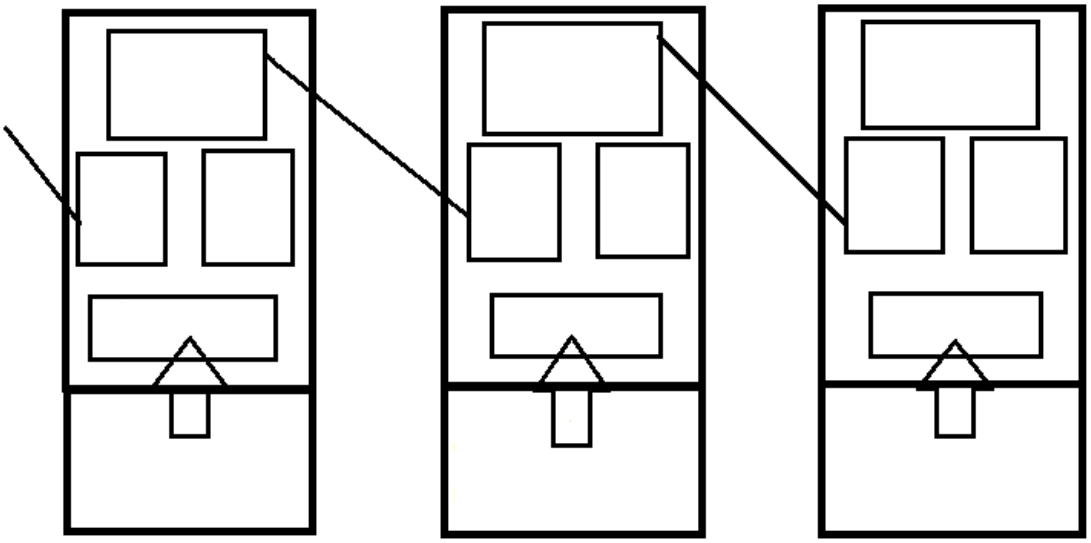

*Figure. The structural mechanism behind append-only history: each block's header commits to the block before it, so altering any past block would break every header chained after it, which is why a forked node can deterministically replay history rather than merely trust it. Source: Wikimedia Commons, CC BY-SA 4.0.*

That guarantee only holds if the data source is actually complete. Not every node retains full historical state.

| Node type | State availability | Forensic suitability |
|---|---|---|
| Archive node | Full historical state at every block | Required for exact pre-incident storage reconstruction |
| Full node (pruned) | Recent state only; older state discarded | Adequate for recent incidents, insufficient for older ones |
| Light client | Headers only, no state | Not suitable as a primary evidence source |
| Third-party RPC or indexer | Varies by provider; may rate-limit or lag | Useful for corroboration, never the sole source of truth |

Pull the state you need to reconstruct an incident from an archive node or a hash-verified export (Section 5.2), and treat a third-party indexer's output as corroborating evidence to be cross-checked, not as the canonical record.

## 3.3 Off-Chain Evidence (Logs, Screenshots, Comms)

On-chain data is durable by construction; off-chain evidence is not, and it decays fast. RPC provider logs roll off retention windows measured in days or weeks. Front-end and indexer logs disappear when a team takes a compromised service offline for remediation, often the very action an incident triggers. Community channels where an attacker or a confused user first posted, Discord, Telegram, X, get edited or deleted, sometimes by the platform's own moderation before the team even sees them.

Treat off-chain artifacts with the same urgency as active on-chain evidence: capture screenshots and full page exports the moment they are found, hash them immediately per the chain-of-custody fields in Section 2.1, and record the capture timestamp independent of any timestamp embedded in the content itself, since a screenshot's own metadata is not proof of when it was taken. Chapter 6 details the specific off-chain sources (RPC logs, monitoring alerts, communications, deployment artifacts) and how to preserve each; the discipline to carry forward from this section is that off-chain evidence is the most perishable asset in the entire investigation and should be the first thing collected, not the last.

---

**Key controls for Part 1**

- Classify every incident's root cause against the four-tier framework (Section 1.5) before writing a recovery recommendation, and explicitly check for a second tier before closing the finding.
- Snapshot state, hash it, and log chain of custody before pausing, upgrading, or messaging an attacker.
- Screen every address you might send funds to or receive from against the OFAC SDN list before any recovery action.
- Route public disclosure and law enforcement contact through legal counsel, never through the responder who found the bug.
- Prefer archive-node or hash-verified exports over live RPC or indexer output as your evidentiary source of truth.
- Capture off-chain evidence, logs, screenshots, communications, first: it decays faster than anything on-chain ever will.

# Part II: Evidence Collection and Preservation

An exploit's evidence trail exists in two very different places at once: an append-only ledger that will outlive the incident by design, and a scattering of off-chain systems (Remote Procedure Call (RPC) logs, indexers, dashboards, Slack threads) that will not. This part is the responder's field manual for both halves: what to pull from the chain, how to preserve it so a later re-derivation matches byte for byte, and how to capture the off-chain context that on-chain data alone cannot explain.

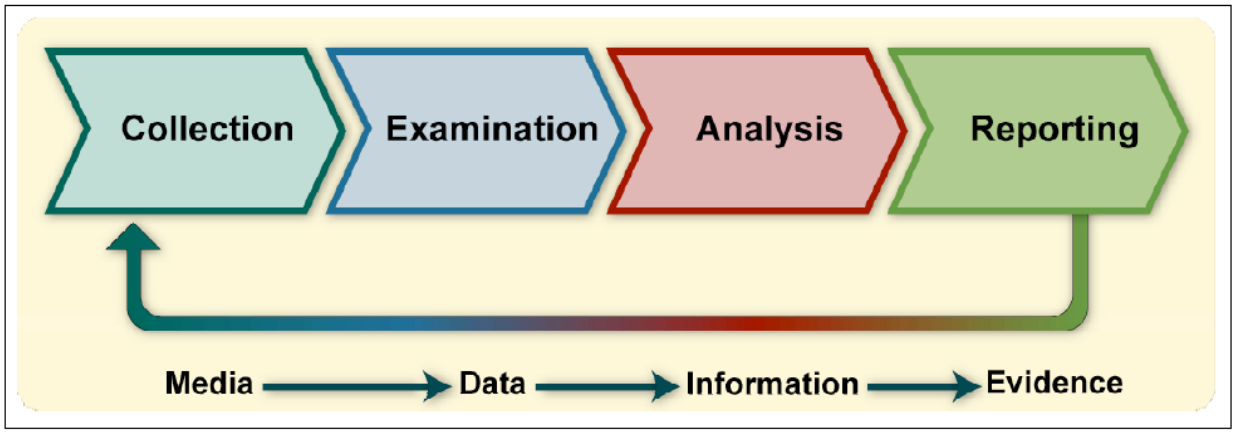


**Figure 3-1. Forensic Process**

*Figure. NIST's forensic process separates collection, examination, analysis, and reporting. On-chain transparency does not collapse these stages: an RPC response is collected data, a decoded trace is examined information, and only a documented analysis tied to its preserved source becomes defensible evidence. Source: NIST SP 800-86, Figure 3-1, public domain (U.S. government work).*

# 4. On-Chain Evidence

On-chain evidence is unusual among forensic artifacts because the source of truth is globally replicated and, past a finality boundary, effectively immutable. That property does not make collection trivial. A responder still has to choose the right RPC surface, decide how deep to trace, and capture data before a pruning node, a rate-limited API, or your own assumptions cause you to lose a detail that mattered.

## 4.1 Transaction Data (Tx Hash, Block, Input, Value)

The transaction is the atomic unit of every on-chain investigation, and the first artifact to pin down is its hash, because everything else keys off it. From a full or archive node, `eth_getTransactionByHash` returns the transaction envelope: `from`, `to`, `value`, `gas`, `gasPrice` (or the EIP-1559 `maxFeePerGas` / `maxPriorityFeePerGas` pair), `nonce`, and the raw `input` calldata. `eth_getTransactionReceipt` returns the execution outcome: `status` (success or revert), `gasUsed`, `cumulativeGasUsed`, `contractAddress` (if the transaction deployed a contract), and the `logs` array covered in 4.3. Record both, not just one; the envelope tells you what was requested, the receipt tells you what actually happened.

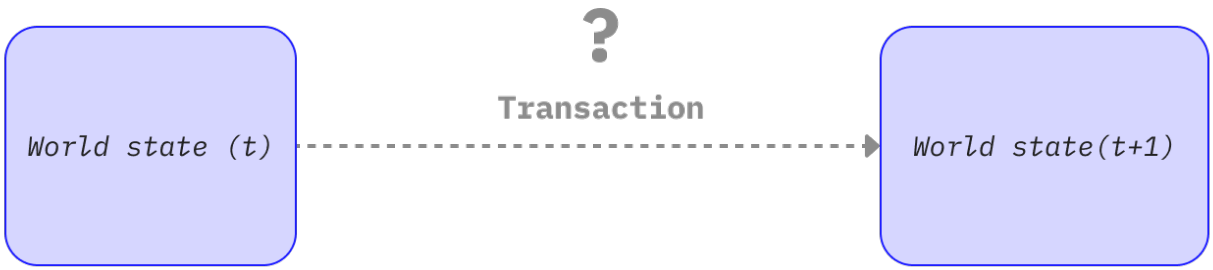


*Figure. A transaction is best understood as a requested state transition. The signed envelope proves what an account requested; execution against the parent state determines whether the transition succeeds, and the receipt and post-state commitments prove what the chain accepted. Source: ethereum.org, Transactions, CC BY 4.0.*

Decode the `input` field against the target contract's Application Binary Interface (ABI) to recover the function selector (the first four bytes) and the arguments as typed values, not raw hex. If the contract is unverified, look up the selector in a public database such as 4byte.directory or OpenChain's signature database before falling back to bytecode-level disassembly. Record the block number and block hash together, not the block number alone: a block number is only unambiguous once the chain has passed finality, and citing the hash alongside the number lets a later reader detect if they are looking at a different fork than you were.

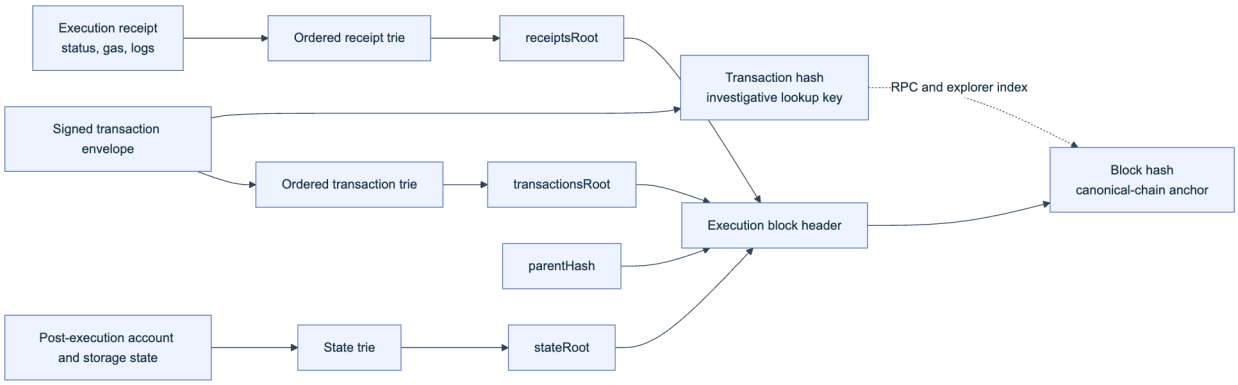


*Figure. Ethereum evidence anchors. The transaction hash is the stable lookup key used by RPC endpoints and explorers, while the execution block header commits separately to the ordered transaction trie, receipt trie, and post-execution state trie. Preserve the transaction hash, receipt, block number, and block hash together; the transaction hash by itself is not a consensus proof that the transaction was included in a particular canonical block.*

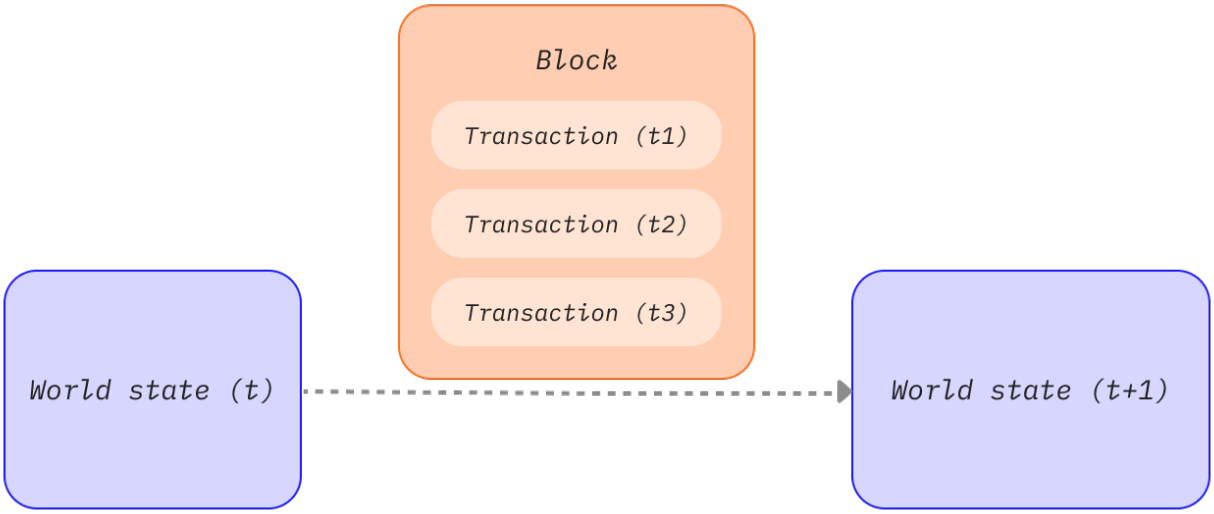


*Figure. Transactions inside a block are ordered state transitions, not an unordered event set. Replaying an exploit transaction against the wrong parent block or after applying neighboring transactions in a different order can produce a different storage state, gas result, or oracle price. Source: ethereum.org, Blocks, CC BY 4.0.*

```
# Etherscan V2 unified API: same endpoint, chainid parameter selects the network
ETHERSCAN_API="https://api.etherscan.io/v2/api"
curl -sG "$ETHERSCAN_API" \
  --data-urlencode "chainid=1" \
  --data-urlencode "module=proxy" \
  --data-urlencode "action=eth_getTransactionByHash" \
  --data-urlencode "txhash=0x<TXHASH>" \
  --data-urlencode "apikey=$ETHERSCAN_API_KEY" | jq .

curl -sG "$ETHERSCAN_API" \
  --data-urlencode "chainid=1" \
  --data-urlencode "module=proxy" \
  --data-urlencode "action=eth_getTransactionReceipt" \
  --data-urlencode "txhash=0x<TXHASH>" \
  --data-urlencode "apikey=$ETHERSCAN_API_KEY" | jq .
```

Etherscan's V2 API consolidated what used to be separate per-chain endpoints and keys into one API surface addressed by `chainid`, covering more than 60 EVM-compatible networks under a single account (Etherscan API V2 docs), which matters for a responder who needs the same query shape across an incident that touches several chains.

| Field | Source call | Why it matters for the case |
|---|---|---|
| Tx hash | Reported by victim, explorer, or alert | Primary key for every downstream query |
| Block number + hash | `eth_getTransactionByHash` | Anchors the evidence to a specific, citable chain state |
| `from` (EOA or contract) | `eth_getTransactionByHash` | Identifies the initiating actor, feeds Chapter 9 attribution |
| `input` calldata, decoded | ABI decode against verified source or Sourcify | Reveals the exact function and arguments invoked |
| `status` | `eth_getTransactionReceipt` | Confirms the call succeeded; a reverted attempt is still evidence of intent |
| `gasUsed` vs `gas` limit | `eth_getTransactionReceipt` | An out-of-gas revert looks different from a `require` revert; both matter |

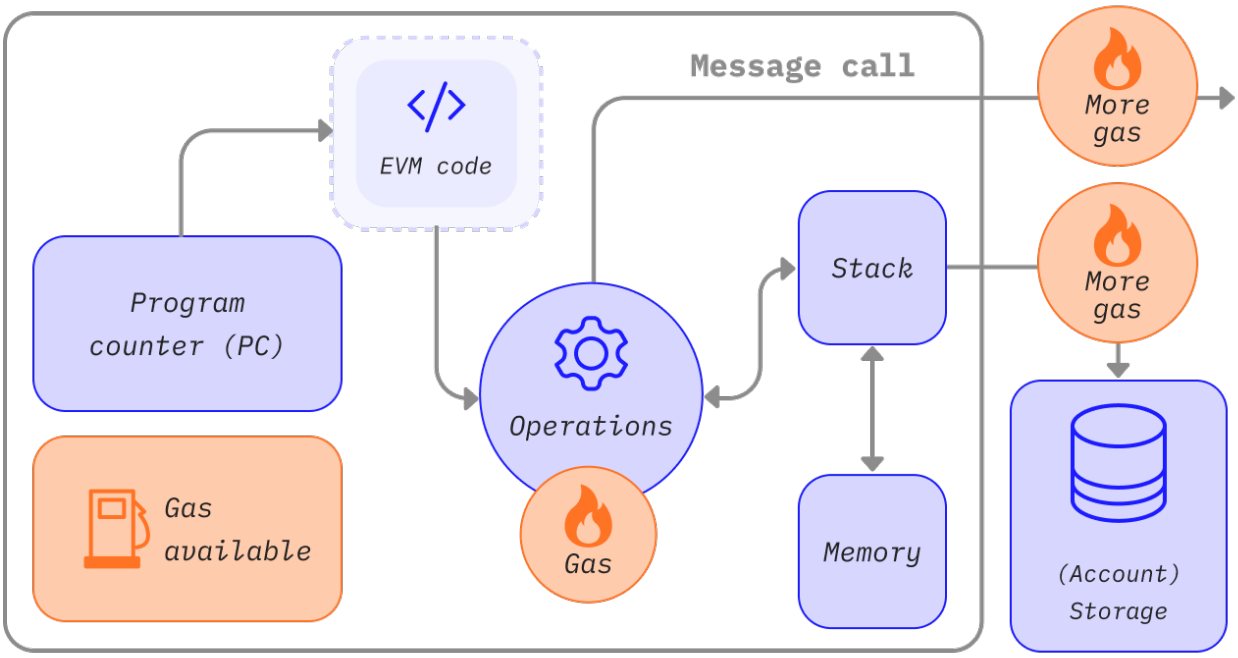


*Figure. Gas is metering evidence, not just a fee. Different EVM operations consume different gas, so a trace's gas deltas help distinguish an out-of-gas failure, an unexpectedly expensive storage write, and a branch that never executed. Preserve both the transaction limit and per-frame consumption. Source: ethereum.org, Gas and Fees, CC BY 4.0.*

### 4.2 Contract State and Storage

Transaction data shows what was called; storage shows what changed. Raw EVM storage is a flat $2^{256}$ slot key space per contract address, and Solidity's storage layout packs state variables into slots according to declaration order and type size, so reading a slot in isolation is meaningless without the layout. Pull the layout from the verified source (via Etherscan, Sourcify, or `solc --storage-layout`) before interpreting raw values, and note that structs, mappings, and dynamic arrays use derived slot positions (`keccak256(key . slot)` for mappings) rather than sequential ones.

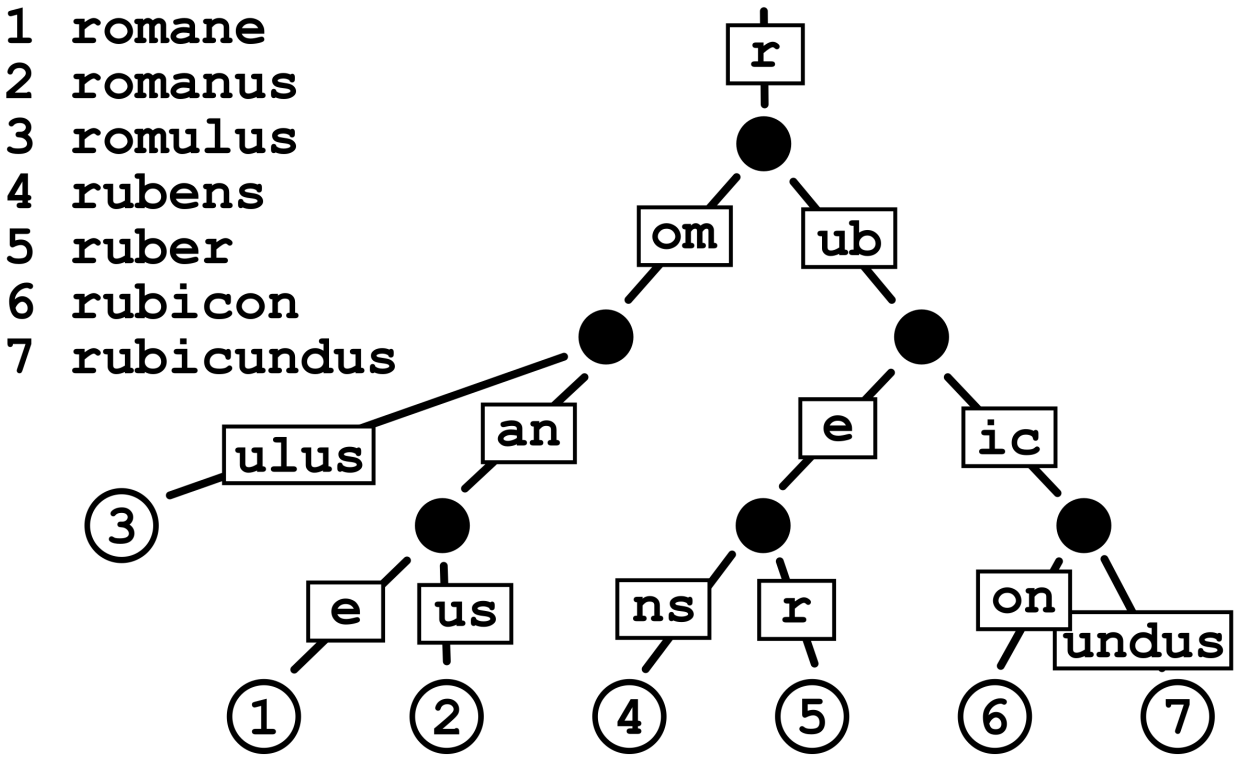


*Figure. The general shape of the key-value structure underlying Ethereum's state and storage tries: a derived, compressed path from a key (here a word, on-chain a keccak256 hash) down to its stored value, which is why a raw slot number means nothing without the derivation rule that produced it. Source: Claudio Rocchini via Wikimedia Commons, CC BY-SA 3.0.*

```
# Read a specific storage slot at a specific historical block (requires an archive
        node)
cast storage 0xCONTRACT_ADDRESS 0 --block 18500000 --rpc-url $ARCHIVE_RPC_URL

# Read the ERC-1967 implementation slot for a proxy, at the block just before the in-
        cident
cast storage 0xPROXY_ADDRESS \
  0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bb \
  --block 18499999 --rpc-url $ARCHIVE_RPC_URL
```

Two details separate a usable storage snapshot from a useless one. First, `eth_getStorageAt` against a full node only serves current state by default; historical reads at an arbitrary past block require an archive node that retains all intermediate state, since a pruning node discards it after a

retention window. If your own infrastructure runs pruned nodes, plan your archive-node access before you need it, not during the incident. Second, always pair a storage read with the block number and block hash it was taken at, and re-verify the same read against a second, independently operated node or provider. Two matching independent reads at the same block hash are far stronger evidence than a single read you cannot cross-check.

For proxy contracts, capture both the proxy's storage and the implementation contract's storage and code hash, and record which implementation was active at the moment of the incident using the ERC-1967 implementation slot shown above. A post-incident upgrade that changes the implementation address will silently break naive re-derivation if you only recorded the proxy address without pinning the implementation at the relevant block.

## 4.3 Event Logs and Internal Transactions

Events are the structured, indexed trail a contract deliberately emits, and they are usually the fastest way to reconstruct what moved without re-executing every opcode. Each log entry carries the emitting contract's address, up to four `topics` (topic0 is the keccak256 hash of the event signature; topics 1 through 3 hold indexed parameters), and an ABI-encoded `data` field for non-indexed parameters. `eth_getLogs` retrieves logs by address, topic filter, and block range; retrieve the full range for a transaction with `eth_getTransactionReceipt`'s `logs` array so you get every log the transaction emitted, in emission order, not just the ones matching a filter you guessed in advance.

Internal transactions are calls, delegatecalls, and value transfers that happen inside a transaction's execution but never appear as their own top-level transaction. They are not part of consensus state and are not retrievable via standard JSON-RPC; you need a full execution trace. Two APIs cover this: geth's `debug_traceTransaction` (with the `callTracer` for a call-tree view) and the OpenEthereum-lineage `trace_transaction` / `trace_replayTransaction`, both of which most archive-node providers and Etherscan's trace endpoints expose.

```
# Full call tree for internal transactions, including value transfers and reverted sub-
        calls
cast rpc debug_traceTransaction 0x<TXHASH> '{"tracer":"callTracer"}' --rpc-url
        $ARCHIVE_RPC_URL | jq .
```

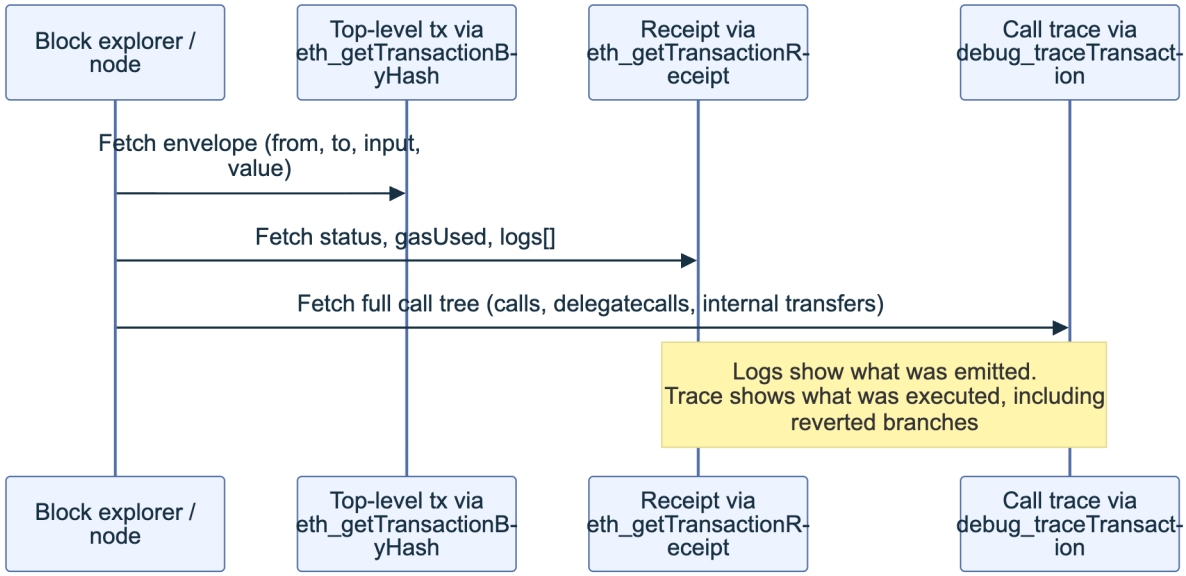


*Figure 1. Three complementary views of one transaction. A responder who only reads the receipt misses internal transfers; one who only reads the trace misses which events downstream consumers (indexers, front ends) actually saw.*

A reentrancy or a multi-hop token drain is frequently invisible in the top-level transaction's `to` and `value` fields and only visible in the trace, because the attacker contract routes value through several internal calls before it ever surfaces as an indexed event. Preserve the full trace output alongside the logs; a root-cause finding built only from `Transfer` events will misattribute value movement whenever a contract moves funds without emitting a matching event, which Chapter 8's precision and invariant forensics returns to directly.

## 4.4 Relevant Addresses (EOAs and Contracts)

Every address touched by the incident is a node in the case graph, and each needs its type, role, and provenance recorded before you move to the next transaction. Distinguish an externally owned account (EOA, controlled by a private key) from a contract account (controlled by deployed code) using `eth_getCode`: an EOA returns empty bytecode (`0x`), a contract returns its runtime bytecode. This single check resolves a common early-investigation error, treating a smart-contract wallet or a proxy as if it were a human-controlled key.

For each address, capture:

- **Type**: EOA or contract, and if a contract, whether it is a proxy, a factory-deployed clone, or a standard deployment.
- **Deployment provenance**: the deploying transaction hash, the deployer address, and (for `CREATE2`) the salt and init code hash, since `CREATE2` lets an address be predicted and re-used across chains, which matters for both attribution and cross-chain correlation (10.1).

- **Verified source status**: whether the contract is verified on Etherscan, Sourcify, or Blockscout, and a locally archived copy of the verified source and ABI regardless, since a verification record can be removed or a service can go offline.
- **Role in the incident**: attacker EOA, attacker contract, victim protocol contract, intermediary (mixer, bridge, DEX router), or destination exchange or wallet.
- **First and last seen**: the block range over which the address is relevant to this specific incident, not its entire history.

```
# Confirm EOA vs contract
cast code 0xADDRESS --rpc-url $RPC_URL   # "0x" => EOA; non-empty => contract

# Pull deployer and deployment tx (Etherscan V2, module=contract)
curl -s "https://api.etherscan.io/v2/api?chain-
        id=1&module=contract&action=getcontractcreation&contractaddresses=0xADDRESS&ap
        ikey=$ETHERSCAN_API_KEY" | jq .
```

## 4.5 Token Transfers and Approval Flows

Most DeFi incidents are, at the value-movement layer, a sequence of token transfers and approvals, and these are captured through standard event signatures rather than bespoke parsing. The ERC-20 `Transfer(address indexed from, address indexed to, uint256 value)` event has topic0 `0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3`, and `Approval(address indexed owner, address indexed spender, uint256 value)` has topic0 `0x8c5be1e5ebec7d5bd14f71427d1e84f3dd0314c0f7b2291e5b200ac8c7c3b925`. ERC-721 reuses the `Transfer` signature with the third parameter as an indexed `tokenId` instead of a `value`, so distinguish the two by checking whether the emitting contract implements `ERC165` `supportsInterface` for the ERC-721 interface ID before assuming decimal semantics. ERC-1155 uses distinct `TransferSingle` and `TransferBatch` events.

```
# All ERC-20 Transfer events out of an attacker-controlled address, over the incident's
        block range
cast logs --address 0xTOKEN_CONTRACT \
  --from-block 18500000 --to-block 18500050 \
  'Transfer(address indexed,address indexed,uint256)' \
  --rpc-url $ARCHIVE_RPC_URL
```

Approval flows deserve equal attention, because an `Approval` event without a following `Transfer` is still evidence: it shows a victim (or an attacker's own decoy contract) granted spending rights, which is frequently the enabling step in a phishing-driven drain that a purely transfer-focused trace would miss entirely. Reconstruct the full sequence per token: approvals granted, the specific `transferFrom` calls that consumed them, and any `Approval` events that reset an allowance to zero (a common pattern immediately before or after an exploit, as an attacker either clears a com-

promised approval to cover tracks or a protocol's emergency response revokes one). Cross-reference the approval spender address against the address roles captured in 4.4; an approval to an address with no prior interaction history is a strong behavioral signal on its own.

### 4.6 Cross-Chain and Bridge Data Where Applicable

An incident that spans chains multiplies the evidence-collection burden, because no single chain's state proves what happened on another; the responder has to independently collect and then correlate two (or more) separate evidence sets. Capture, per chain, the same categories from 4.1 through 4.5, plus the bridge-specific artifacts: the source-chain lock, burn, or deposit event; the message or proof structure the bridge protocol uses to attest to that event (a Merkle proof, a validator multisignature, an optimistic fraud-proof window, or a light-client update, depending on the bridge's design); and the destination-chain mint, unlock, or release transaction that consumed that message.

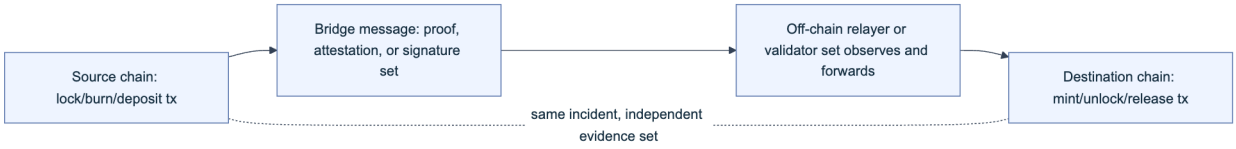


*Figure 2. A bridge transfer produces at least three separate evidence artifacts across two chains. Collect all three; the destination-chain mint alone does not prove the source-chain lock was legitimate, which is exactly the gap several major bridge exploits (covered in Chapter 10) exploited.*

Record the message identifier or nonce the bridge protocol uses to correlate the two legs (most bridges assign a sequence number or a hash of the message payload), since block timestamps alone are not reliable correlation keys across chains with different block times and clock skew. Where the bridge is validated by an external validator set or relayer network rather than by an on-chain light client, also capture the specific signatures or attestations submitted, and from which validator addresses or operator keys, because a validator-trust compromise is a distinct root cause from a verification-logic bug in the bridge contract itself, and the evidence needed to tell them apart is exactly this attestation data. Cross-chain evidence consistency is revisited in 10.3.

## 5. Data Preservation

Collecting evidence once is not the same as preserving it. On-chain data is technically re-derivable by anyone with node access, which invites a dangerous assumption: that you never need to archive it because it is "always there." Reorg risk, node pruning, rate-limited or discontinued APIs, and the possibility that opposing counsel or a skeptical stakeholder wants proof your copy has not been altered all argue for treating on-chain data with the same preservation discipline as any other digital evidence.

## 5.1 Exporting and Archiving Blockchain Data

Export every artifact from Chapter 4 in a durable, self-describing format, not as a live link to a service that might change or disappear. For each transaction of interest, save: the raw JSON responses from `eth_getTransactionByHash` and `eth_getTransactionReceipt`, the full trace output, the decoded logs, the verified source and ABI (or a note that none was available, plus the raw bytecode), and, for contract-state questions, the raw storage reads with their block hash. Prefer raw RPC JSON over an explorer's rendered HTML page: the JSON is what the node actually returned and is machine-verifiable against a hash; a screenshot of a web page is evidence of what a web page displayed, one derivative step removed from the chain data itself.

```
mkdir -p case-0042/{tx,receipts,traces,logs,storage,source}
cast tx 0x<TXHASH> --json --rpc-url $ARCHIVE_RPC_URL > case-0042/tx/0x<TXHASH>.json
cast receipt 0x<TXHASH> --json --rpc-url $ARCHIVE_RPC_URL > case-0042/receipts/0x<TX-
        HASH>.json
cast rpc debug_traceTransaction 0x<TXHASH> '{"tracer":"callTracer"}' \
  --rpc-url $ARCHIVE_RPC_URL > case-0042/traces/0x<TXHASH>.json
```

For a broader export beyond a handful of transactions, use a purpose-built indexer or a full-node export rather than paginating an explorer's rate-limited free-tier API. Projects such as The Graph (subgraph indexing), Dune Analytics (SQL over decoded on-chain data), or a self-hosted archive node paired with `geth`'s or `reth`'s built-in export tooling scale to the volume a serious incident produces. Independently archive verified source code and metadata through Sourcify, whose stated goal is preventing exactly the failure mode this section warns about: proprietary explorer services that could restrict access or shut down and take verification history with them (Sourcify project introduction).

## 5.2 Timestamping and Integrity (Hashing)

Once evidence is exported to a file, prove two things about that file going forward: that it has not changed since export, and that it existed at or before a specific point in time. Hashing solves the first problem. Compute a SHA-256 digest of every exported artifact at collection time, record the digest in a manifest alongside the file path and collection timestamp, and recompute the digest before every subsequent use of the file to confirm it still matches.

```
find case-0042 -type f -exec sha256sum {} \; > case-0042/MANIFEST.sha256
# Verify later:
sha256sum -c case-0042/MANIFEST.sha256
```

Hashing alone does not prove *when* a file was created, only that it has not changed since whenever the hash was computed; a self-generated hash and timestamp are trivially forgeable by the party that generated them. RFC 3161 solves this with an independent Time-Stamp Authority (TSA): you submit a hash of your data (never the data itself) to the TSA, and it returns a signed token binding that hash to a trusted time, which any verifier can later check against the TSA's

certificate without the TSA ever having seen your underlying evidence. OpenTimestamps offers a decentralized variant of the same idea built on the Bitcoin blockchain: it aggregates many hash commitments into a Merkle tree and anchors the tree root in a Bitcoin transaction, so the proof of existence-at-a-time rests on Bitcoin's own immutability rather than on trusting a single TSA operator.

```
# RFC 3161 timestamp of the case manifest via a public TSA
openssl ts -query -data case-0042/MANIFEST.sha256 -sha256 -no_nonce -out case-0042/re-
        quest.tsq
curl -s -H "Content-Type: application/timestamp-query" \
  --data-binary @case-0042/request.tsq \
  https://freetsa.org/tsr > case-0042/response.tsr
openssl ts -reply -in case-0042/response.tsr -text
```

Timestamp the manifest itself, not each individual file: because the manifest contains the hash of every artifact, a single timestamp token on the manifest transitively proves the existence-at-time of everything it references.

## 5.3 Write-Once and Tamper-Evident Storage

A hashed, timestamped file is still just a file until it sits somewhere that resists modification. Write-once-read-many (WORM) storage enforces at the infrastructure layer what a hash manifest only detects after the fact: the object cannot be overwritten or deleted for a configured retention period, full stop, regardless of who holds credentials to the storage account. AWS S3 Object Lock is the common cloud implementation, offering two retention modes: *governance* mode, where users with a specific elevated permission can still override the lock, and *compliance* mode, where "the only way to delete an object under the compliance mode before its retention date expires is to delete the associated AWS account" (AWS S3 Object Lock documentation). Compliance mode has been independently assessed against the recordkeeping requirements of SEC Rule 17a-4(f), CFTC, and FINRA regulations, which makes it a reasonable default for evidence a protocol team anticipates needing to defend in a regulatory or legal proceeding.

Pair object-lock retention with a separate, immutable legal hold on any object tied to an active investigation: a legal hold has no expiration and stays in force until explicitly and auditably removed, independent of whatever retention period was originally set. For evidence that must be provably tamper-evident without depending on a single cloud provider's configuration, layer a content-addressed store on top: publish the case manifest's root hash to IPFS (where the content identifier is itself derived from the content, so retrieval under the wrong hash simply fails) or to Arweave for permanent storage, in addition to, not instead of, your primary WORM bucket. The goal is defense in depth: no single storage provider's policy, outage, or compromise should be able to make evidence disappear or go undetected if altered.

## 5.4 Retention and Access Control

Preservation without access discipline creates a different failure mode: evidence that survives intact but whose custody chain is unclear, which weakens its value exactly when it is needed most. Maintain a chain-of-custody log for every artifact: who collected it, when, from which source (node provider, explorer, archive), who has accessed it since, and any transformation applied (decoding, format conversion, redaction). NIST SP 800-86, the *Guide to Integrating Forensic Techniques into Incident Response*, frames this discipline around the operational IT-response context rather than a pure law-enforcement one, which fits a protocol team's typical position: you are building a record good enough to support internal decisions, external disclosure, insurer claims, and potential litigation, without necessarily being a law enforcement agency yourself (NIST SP 800-86). Chapter 2 of Part I covers the legal and admissibility framing this chain-of-custody discipline exists to satisfy; this section covers its mechanics.

Apply least-privilege access to the case store itself: a small, named set of investigators with read access, an even smaller set with write access to add new artifacts (never to modify existing ones, which the WORM configuration in 5.3 should enforce structurally), and a logged approval step before any artifact is shared outside the immediate response team. Set retention periods deliberately rather than by default: evidence supporting an active or reasonably anticipated legal claim should outlive your organization's general log-retention policy, and a hold placed too early (before you know litigation is likely) is far cheaper to justify than evidence destroyed too early after a claim has already been filed.

| Access role | Read case evidence | Add new evidence | Modify or delete evidence | Approve external sharing |
|---|---|---|---|---|
| Investigator | Yes | Yes | No | No |
| Response lead | Yes | Yes | No (bypass requires compliance-mode account deletion) | Yes |
| Legal counsel | Yes | No | No | Yes |
| External auditor (scoped) | Read-only, specific artifacts only | No | No | No |

# 6. Off-Chain and Supporting Evidence

On-chain data proves what the ledger recorded; it says nothing about how a user's browser was tricked into signing a malicious transaction, what an operator saw on a dashboard three minutes before the exploit, or what the team said to each other in the first hour of response. Off-chain evidence closes that gap, and it decays faster than on-chain data: logs rotate, dashboards reset, and memories of who said what in a crisis blur within days.

## 6.1 Logs (RPC, Indexers, Front-End)

Three off-chain log sources routinely carry evidence an on-chain trace cannot reconstruct on its own. RPC provider logs (from your own node infrastructure or a provider such as Infura, Alchemy, or QuickNode) capture *which client called which method, when, from which IP or API key*, which matters when the question is not "what did the chain record" but "who queried for this data, and did an attacker's tooling probe the RPC endpoint before the exploit transaction landed," a pattern consistent with an attacker testing a malicious payload against a fork before firing it at mainnet.

Indexer logs, whether from a self-hosted subgraph, The Graph, Dune, or a custom ETL pipeline, capture the *derived* view of chain data your own systems and dashboards were actually using at incident time. This matters because an indexer can lag the chain, drop events under load, or apply decoding logic that was subtly wrong, meaning your monitoring may have shown a different picture than raw chain state at the same moment; preserving the indexer's own logs and its output as of the incident window lets you reconstruct what your team actually saw, not just what was objectively true on-chain.

Front-end logs (application logs, browser console errors reported via a service like Sentry, and CDN access logs) are the closest evidence to what a *user* experienced, and are essential when the incident vector runs through the UI rather than purely through contract logic, such as a malicious transaction constructed by compromised front-end JavaScript (the failure mode the CDN and Front-End Supply Chain Handbook, 01, covers in depth). Where the compromised front end is no longer serving the malicious version by the time you investigate, the Internet Archive's Wayback Machine or your own CDN's historical access logs may hold the only surviving copy of what was actually served to users during the incident window; capture both immediately rather than assuming the live site still reflects what happened.

## 6.2 Monitoring and Alert Outputs

Whatever monitoring stack was in place before the incident produced a record independent of the incident's own artifacts, and that independence is exactly what makes it valuable: an alert fired (or conspicuously failed to fire) based on rules written before anyone knew what the attack would look like. Preserve, verbatim and with original timestamps, every alert notification

(PagerDuty, Opsgenie, a Discord or Slack webhook, email), the underlying metric or rule definition that triggered it, and the raw time-series data the alert was evaluated against, not just a screenshot of a dashboard panel that will reset its time window on next page load.

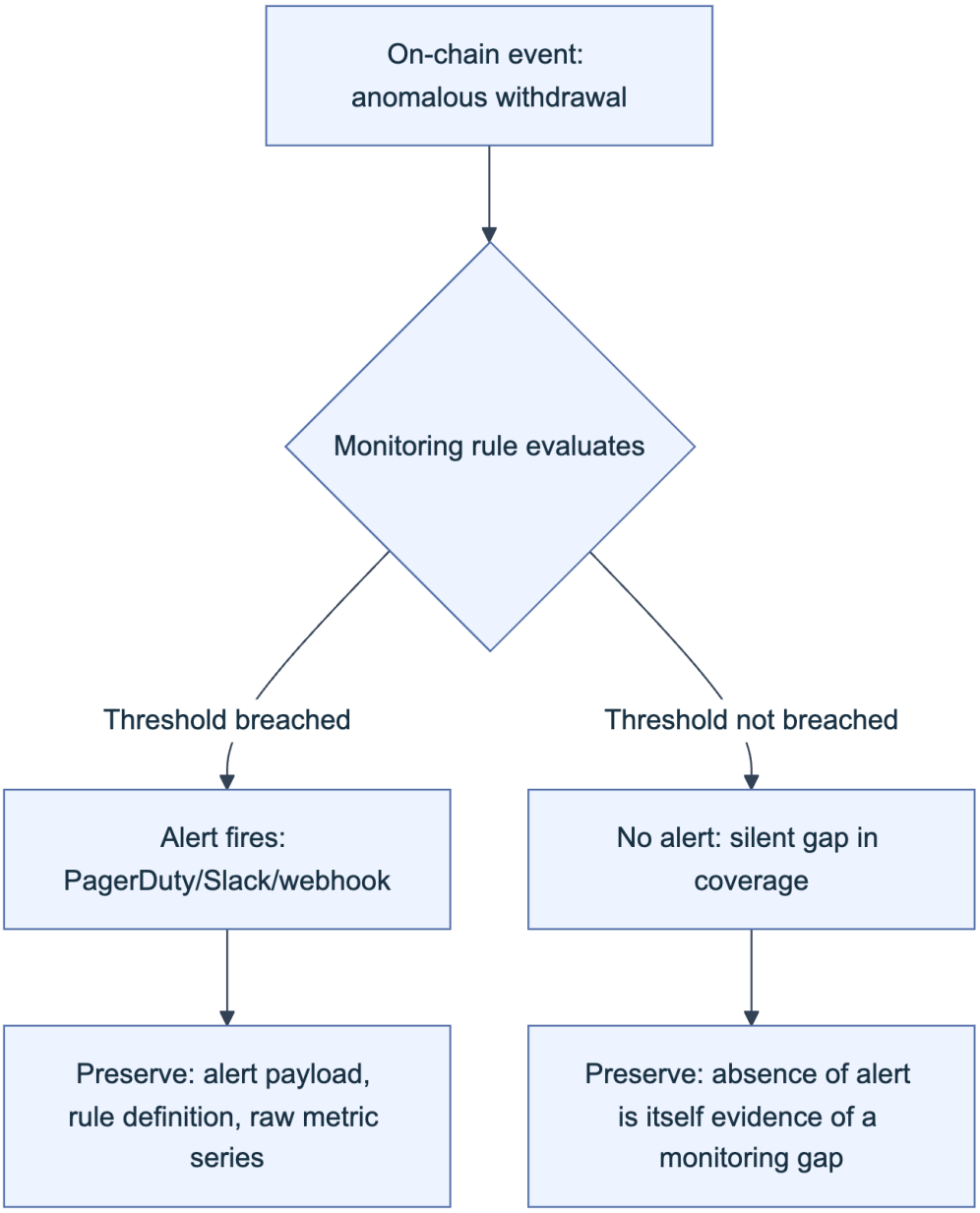


*Figure 3. Both outcomes are evidence. A fired alert shows what the team knew and when; a missing alert for an anomaly that, in hindsight, should have triggered one is a finding about monitoring coverage that belongs in the post-mortem.*

Where monitoring did not fire for an anomaly that in hindsight should have triggered a rule, document that gap explicitly rather than letting it disappear; it is a legitimate root-cause contributor (a Tier 2 lifecycle and governance gap in the four-tier framework from Part I) and belongs in both the technical post-mortem and any coverage improvements that follow. Capture

the on-call schedule and escalation timeline alongside the alert data, since "who was paged, when, and how quickly did they acknowledge" is frequently a direct input to the recovery-window analysis in Part IV.

## 6.3 Communications and Timeline Notes

The human decisions made in the first hours of an incident are themselves evidence, and they are the fastest category to lose, because Slack retention policies expire, memories compress, and a verbal decision made on an ad hoc call leaves no artifact at all unless someone deliberately creates one. Start a timeline document the moment the incident is suspected, not once it is confirmed, and log every entry with a real timestamp (including timezone) rather than a relative one: "detected anomalous withdrawal," "paused contract via multisig tx 0x…," "posted public acknowledgment," "engaged [named exchange] to flag destination address." Export the actual chat transcripts (Slack, Discord, Telegram) covering the response window rather than paraphrasing them after the fact; a paraphrase loses exactly the nuance (who proposed the pause, who objected, what alternative was considered and rejected) that later scrutiny, whether internal, journalistic, or legal, tends to focus on.

Maintain a single canonical, append-only timeline that consolidates the technical (on-chain), monitoring, and communications timelines into one sequence, since these frequently get created by different people in different tools and reconciling them after the fact, once memories have faded, is far harder than merging timestamps as the incident unfolds. Chapter 2's evidence-admissibility guidance applies here as much as to on-chain data: a communications record with clear provenance and no gaps holds up to scrutiny in a way a reconstructed narrative does not.

## 6.4 Code and Deployment Artifacts

The last off-chain category is the software development record behind the exploited contract: the exact source commit that was deployed, the compiler version and settings used to build it, the deployment script and its parameters, and any audit or review artifacts that touched the vulnerable code path. Even when a contract is verified on-chain (Sourcify or an explorer), archive the underlying git repository state independently, because a verified-source record proves the deployed bytecode matches *some* source, not that the source repository itself remains available, unaltered, or that its commit history (which may show when a vulnerable change was introduced, by whom, and whether it was reviewed) survives.

```
# Capture the exact commit, compiler, and settings behind a deployed contract
git -C protocol-repo log -1 --format="%H %ad %an %s" -- src/Vault.sol
git -C protocol-repo show HEAD:foundry.toml | grep -E "solc|optimizer|evm_version"
cp -r protocol-repo/out/Vault.sol/Vault.json case-0042/source/Vault.metadata.json
```

Capture the continuous integration and deployment (CI/CD) pipeline configuration and logs for the deployment run itself, the deployer's transaction and its `input` calldata (which for a factory-based deployment reveals the constructor arguments actually used, as distinct from what a deployment script's source claims it passed), and, where available, any prior audit reports covering the exploited function, noting explicitly whether the vulnerable code path was in scope for that audit or was added afterward. This artifact set is what Chapter 8's root-cause and SCWE-mapping analysis draws on most directly, and its absence is itself diagnostic: a protocol with no available audit history or an audit that predates the vulnerable change by several deployments tells its own story about Tier 2 lifecycle exposure.

---

**Key controls for Part 2**

- Collect the transaction envelope, receipt, full call trace, and decoded logs together for every transaction of interest; a receipt alone misses internal transfers and a trace alone misses which events downstream consumers saw.
- Pair every storage read and cross-chain artifact with the exact block number and block hash it was taken at, and cross-verify against a second independent node.
- Hash every exported artifact at collection time, timestamp the manifest with an RFC 3161 authority or OpenTimestamps, and store it under WORM (compliance-mode) retention with a documented chain of custody.
- Preserve off-chain RPC, indexer, front-end, monitoring, and communications logs immediately; they decay far faster than on-chain state and cannot be re-derived later.
- Archive the exact deployed source, compiler settings, and deployment calldata independently of any third-party verification service.

# Part III: Analysis Techniques

Evidence preserved under Part II is raw material, not a finding. This Part turns a transaction hash into a reconstructed mechanism: tracing execution and value flow, classifying the exploited weakness against OWASP's own control catalog, attributing the actor with an honest confidence level, and handling the cross-chain complications that produce the largest losses in the industry. Each chapter moves the investigation one level closer to a defensible, citable root cause.

## 7. Transaction and Flow Analysis

Evidence preservation freezes the scene. Flow analysis is where the investigation actually starts, turning a bare transaction hash into a sequence of calls, state changes, and asset movements a human can reason about. Every technique below answers the same question at a different resolution: what moved, along what path, and who authorized it.

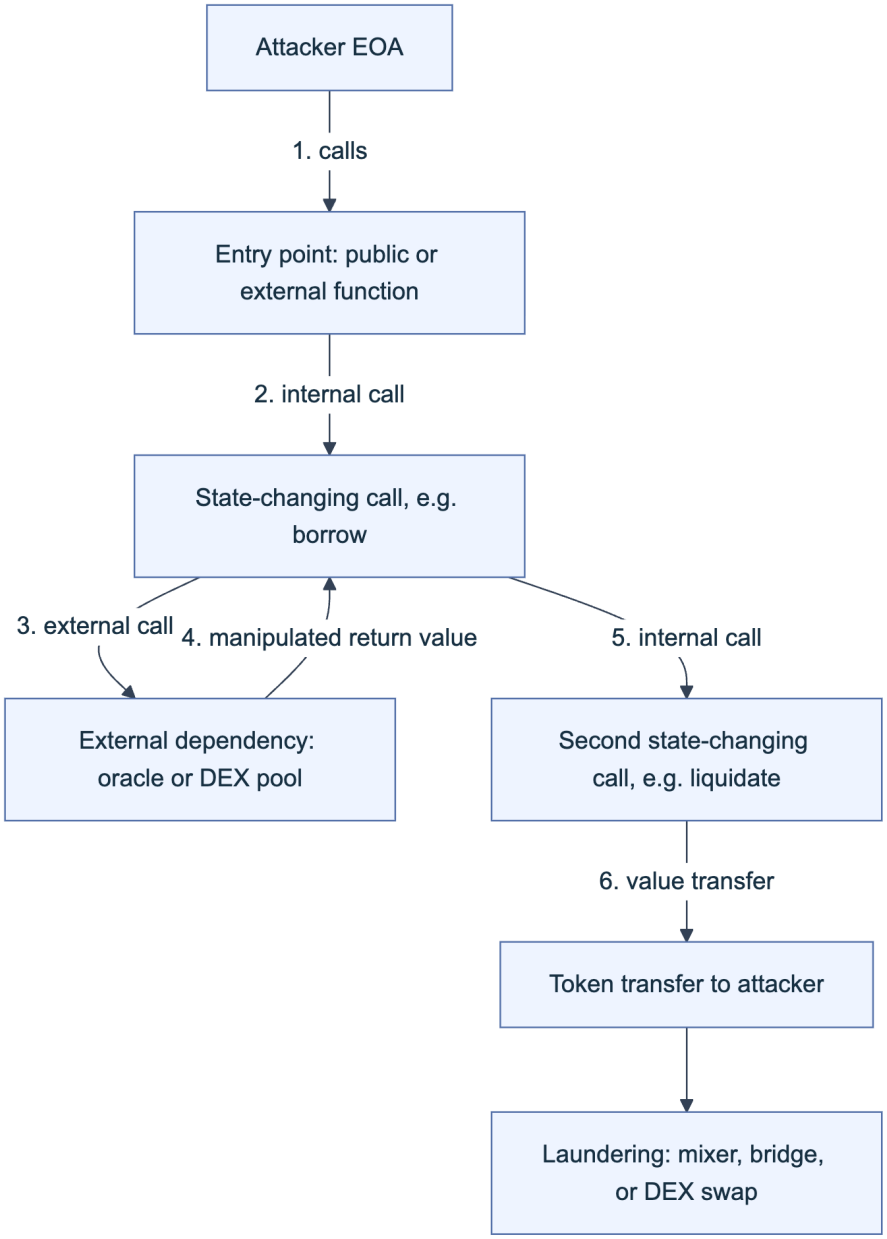


*Figure 6. A generic exploit call chain. Chapter 7 reconstructs steps 1 through 6 from raw trace and log data before Chapter 8 asks why step 4 was possible.*

## 7.1 Tracing Execution Paths

A transaction receipt gives a pass or fail result and a list of emitted logs. It does not give the sequence of internal calls, the opcode-level state transitions, or the intermediate values that produced the final outcome. Reconstructing that requires a debug-level trace. Two RPC (Remote Procedure Call) trace families dominate: Geth's `debug_traceTransaction` with a `callTracer` for a call graph or a `prestateTracer` for storage-state context, and Erigon/OpenEthereum-style `trace_transaction` producing structured create, call, and self-destruct frames. Most public endpoints disable these namespaces because tracing is computationally expensive and exposes implementation-specific behavior. Whether a historical transaction can be replayed depends on the client, trace mode, synchronization mode, and retained block/state history; do not equate

"debug tracing available" with "archive node" without testing the exact client and provider. Record client name/version and trace configuration with the evidence because traces from different clients and tracers are not byte-for-byte interchangeable.

```
curl -s -X POST -H "Content-Type: application/json" \
  --data '{"jsonrpc":"2.0","id":1,"method":"debug_traceTransaction",
           "params":["0xEXPLOIT_TX_HASH", {"tracer":"callTracer"}]}' \
  "$ARCHIVE_RPC_URL"
```

Read the result outside-in: the top frame is the entry point, each nested frame is a call the parent frame made, and an `error` field on any frame marks where a revert originated even when the outer transaction succeeded overall, a common pattern in exploits that probe several paths and keep only the one that works. Cross-reference opcode-level output against the EVM opcode reference and geth's debug namespace docs whenever exact gas forwarding or a low-level `staticcall` matters to the finding.

## 7.2 Identifying Entry Points and Call Chains

The entry point is the first externally callable function the attacker invoked, and it is rarely where the actual bug lives. Decode the leading four bytes of the calldata (the function selector) against a signature database; when source is unverified, Openchain's signature lookup, successor to the community-run 4byte.directory registry, resolves a selector to a human-readable signature without needing source at all. From there, walk the call tree from 7.1 and classify each hop by opcode. `CALL` changes both storage context and `msg.sender` to the callee. `DELEGATECALL` keeps the caller's storage and identity while borrowing the callee's code, the mechanism behind every proxy pattern and a repeat factor in upgrade-related incidents. `STATICCALL` forbids state changes, marking a branch where value could not have moved. Flag every point where the chain crosses a trust boundary, a call into a contract the target protocol does not control, because that is where an attacker-supplied return value or a reentrant callback gets smuggled in. A clean, annotated call chain (function, contract, call type, one-line note on what changed) is the artifact every later section of this chapter builds on.

## 7.3 Token and Value Flow Mapping

Native ETH transfers happen through the value field inside `CALL` frames and never emit a log, so they are invisible to log-based tooling and visible only in the internal-transaction trace from 7.1. ERC-20 and ERC-721 transfers, in contrast, emit a `Transfer` event whose signature hash (`keccak256("Transfer(address,address,uint256)")`, computable directly and commonly seen written as `0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef`) appears as topic zero on every relevant log entry, letting you filter a block range for asset movement without replaying execution. Build the flow map from both sources together: internal-trace value transfers for ETH and wrapped-native movement, decoded `Transfer`, `TransferSingle`, and `TransferBatch` logs for tokens, ordered by log index within the transaction to get a true sequence rather than an

unordered set. Multi-hop paths, deposit into a lending pool, borrow against it, swap the borrowed asset, bridge it out, are the norm in real exploits, and skipping a hop is the most common way a flow map understates the loss or misattributes the final destination. Cross-reference the resulting graph against known exchange deposit addresses and mixer contracts (Chapter 9) before publishing a "final destination," since an intermediate hop is easy to mistake for an endpoint.

### 7.4 Tools and Methodologies (Tenderly, Etherscan, Custom Scripts)

Tenderly's transaction debugger turns a raw trace into a browsable, source-mapped view: it steps through execution one call and source line at a time, decodes function names, arguments, and gas costs, exposes contract state at any point, and pinpoints the exact instruction where a revert fired. Its simulation mode re-executes a transaction against modified state (a different block, a patched contract, a different input) without broadcasting anything, which is how most public post-mortems produce their "here is what would have happened" counterfactuals. Etherscan's transaction page complements this for verified contracts: the **Internal Txns** tab (an explorer label, not a consensus transaction type) surfaces trace-derived internal calls and native-value transfers missing from the receipt, **Decode Input Data** resolves calldata against a verified ABI (Application Binary Interface), and the Etherscan API lets you script a full address history rather than click through it. For anything repeatable, write it down as code: a Foundry command such as `cast run --trace <tx_hash> --rpc-url $TRACE_RPC` reproduces a trace against a trace-capable endpoint, and an `ethers.js` or `web3.py` script that loops `getLogs` over a block range and reconstructs a flow graph turns a one-off investigation into a reusable tool for the next incident. Favor scripts over manual clicking wherever the same few steps recur; custom tooling is what separates a reproducible finding from an anecdote.

---

## 8. Contract and Vulnerability Analysis

Flow analysis explains what happened. This chapter explains why it was possible, grounding the answer in a named weakness class, an OWASP control reference, and, wherever feasible, a working reproduction.

### 8.1 Identifying Exploited Weaknesses

Start from the state transition that should not have been possible, given the trace and flow map from Chapter 7, and locate the exact statement in source (or, absent verified source, in decompiled bytecode) that permitted it. Diff the pre-exploit deployed bytecode against any patched version released afterward: protocols that ship a fix typically do so within hours, and the diff isolates vulnerable logic far faster than re-reading the entire contract. If no patch exists yet, diff instead against verified source on Etherscan or Sourcify to confirm you are reading the actual deployed bytecode's logic rather than a stale repository branch; deployed bytecode and a project's public HEAD diverge more often than teams admit. Confirm the weakness class with a minimal

counterexample: the smallest call sequence, ideally shorter than the attacker's actual transaction, that reproduces the broken invariant on a forked chain (Section 8.6). A finding that cannot be reduced to a minimal counterexample remains a hypothesis, not a confirmed root cause.

## 8.2 Mapping to SCWE and SCSVS Controls

The Smart Contract Weakness Enumeration (SCWE) catalogs concrete weakness patterns and organizes them under the eleven top-level domains of the Smart Contract Security Verification Standard (SCSVS), so every SCWE entry inherits an SCSVS domain as its category (OWASP SCS project). Mapping an exploited weakness to its SCWE identifier before writing the report gives the finding a stable identity independent of your own phrasing, and mapping onward to the SCSVS domain tells a reader which pre-deployment verification control, had it actually been enforced, would have caught it.

| SCSVS domain | Focus | Typical exploited weakness (this chapter's root causes) |
|---|---|---|
| SCSVS-AUTH | Access control and authentication | Missing or misordered role check, `tx.origin` misuse |
| SCSVS-ORACLE | Arithmetic and logic security | Price manipulation, rounding-direction bugs (8.4, 8.5) |
| SCSVS-COMM | Secure interactions and communications | Reentrancy, unchecked external call return values |
| SCSVS-GOV | Business logic and economic security | Invariant violations, incentive-design flaws |
| SCSVS-BRIDGE | Blockchain data and cross-chain messaging | Bridge verification-logic failures (10.1) |
| SCSVS-ARCH | Architecture, design, and upgradability | Unprotected initializer, storage-layout collision on upgrade |

Treat this table as a starting index, not a substitute for the live catalog. SCWE assigns specific numeric identifiers under each domain, the SCSVS-AUTH domain alone spans roughly SCWE-016 through SCWE-020 plus several non-contiguous IDs elsewhere in the catalog, and those assignments are revised as the project grows. Confirm the current identifier against scs.owasp.org's SCWE catalog rather than citing a number from memory in a report meant to age well.

## 8.3 Root Cause: Logic, Oracle, Access Control, Reentrancy, etc.

| Root cause | Diagnostic question | What to look for in the trace |
|---|---|---|
| Business logic | Does an invariant hold before and after every state-changing function, not only the ones the developer tested? | An operation that changes one side of a paired value (shares vs. assets, debt vs. collateral) without updating the other |
| Oracle dependency | Is price sourced from a single spot reading the attacker can move within one transaction? | A reserve-ratio or `balanceOf` read immediately followed by a large swap in the same call chain |

| Root cause | Diagnostic question | What to look for in the trace |
|---|---|---|
| Access control | Does every privileged function check the caller, and check the right thing (`msg.sender` versus `tx.origin`, role versus ownership)? | A state-changing call with no preceding `require`, modifier, or role check in its own frame |
| Reentrancy | Can an external call in the middle of a function re-enter before that function's state updates finish? | A `CALL` or token-hook frame nested inside a function that writes state after the call (a checks-effects-interactions violation), including read-only reentrancy through a view function during that window |
| Arithmetic and precision | Does rounding or truncation direction ever favor the caller across repeated small operations? | Integer division before multiplication, or a rounding mode that is not uniformly conservative (8.4) |

These categories overlap more than the table suggests. A reentrancy bug is often a business-logic bug wearing a specific costume: state updated too late. An oracle manipulation is an access-control bug in the sense that the protocol implicitly trusted an untrusted price source. Use the table to structure the writeup, not to force a single label onto a failure with more than one contributing factor.

## 8.4 Precision, Rounding, and Invariant Violations (e.g., Balancer-Type)

Integer arithmetic in Solidity truncates toward zero on division, and every automated market maker (AMM) built on an invariant, a constant-product curve, a StableSwap curve, or a weighted-pool formula, must round somewhere when converting between pool tokens, underlying assets, and share amounts. The security-critical question is which direction each rounding operation favors: a protocol stays safe only if every rounding step is conservative in the protocol's own favor, consistently, on every code path touching the same invariant. Balancer's August 2023 disclosure of a critical vulnerability in certain composable stable pool contracts is the reference case for this class (Balancer governance forum): a rounding inconsistency in the pool-rate calculation let an attacker execute a sequence of carefully sized, near-zero-impact trades that accumulated rounding error in the attacker's favor with each iteration, gradually skewing the invariant until it could be drained. Balancer paused affected pools within hours of disclosure, but publicly reported losses still ran into the hundreds of thousands of dollars across pools that had not yet migrated, a reminder that a rounding-direction bug (the SCSVS-ORACLE arithmetic and logic domain) can survive several audits because each individual rounding operation looks locally correct.

The forensic tell in a trace is repetition: dozens or hundreds of structurally identical calls with slightly varying amounts, individually unremarkable, executed inside one transaction or one tight block range. The anti-pattern in miniature:

```
// Unsafe: rounds toward the caller on withdrawal, away from the caller on deposit
function bptToTokens(uint256 bptAmount) public view returns (uint256) {
    return bptAmount * rate / 1e18; // rounds down: fine for deposit accounting
}
function tokensToBpt(uint256 tokenAmount) public view returns (uint256) {
    return tokenAmount * 1e18 / rate; // rounds down here too: favors the CALLER, not
the pool
}
```

When auditing a suspected rounding exploit, isolate every function that both reads and writes the same invariant variable, and verify the rounding direction is uniformly conservative across all of them, not merely sane in isolation.

## 8.5 Price Oracle and Flash-Loan Attack Forensics

A flash loan, an uncollateralized loan that must be borrowed and repaid within a single transaction or the entire transaction reverts, removes the capital constraint that once made price manipulation expensive. Paired with a protocol that reads price from a manipulable on-chain source rather than a time-weighted or externally attested feed, it produces the most common single exploit pattern in DeFi history: from the first flash-loan attacks against bZx in February 2020, through Cream Finance's roughly $130 million loss in October 2021, to Mango Markets in October 2022, where an attacker used borrowed capital to move the thinly traded MNGO perpetual market from roughly $0.03 to $0.91, then borrowed against the inflated collateral value, draining about $115 million before a negotiated partial return.

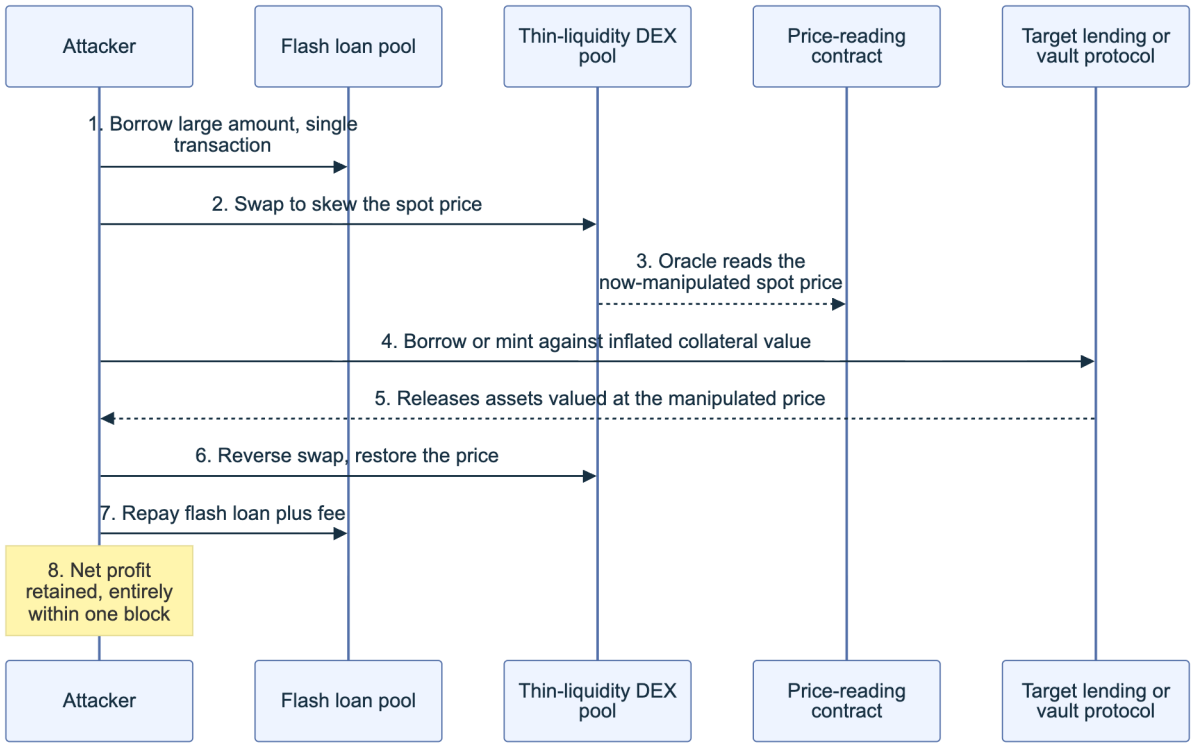


*Figure 7. Canonical flash-loan oracle manipulation sequence. Steps 1 through 8 complete atomically; nothing outside this window is observable until it is already over.*

Reconstruct this class methodically. Identify the exact oracle call, a spot AMM reserve read, a `balanceOf` call, or an external feed such as Chainlink, that the exploited function relied on. Pull the flash loan's borrowed amount from the trace and compute its price impact against the manipulated pool's actual depth at that block; impact scales with loan size relative to liquidity, so an implausibly large flash loan against a shallow pool is itself diagnostic. Confirm the manipulated price was read and consumed inside the same transaction, or the same atomic block, as the swap that produced it. Chainlink's own guidance on the risk of relying on spot reads rather than time-weighted price feeds is the standard citation for why single-block spot pricing is treated as an anti-pattern rather than a mere edge case.

## 8.6 Reproducing the Attack (Testnets, Forks)

A finding is not confirmed until it executes against real, forked chain state; a manual trace read is a hypothesis, a passing forked test is evidence. Foundry's forking cheatcode pins execution to the exact block before the exploit and replays it:

```
forge test --match-test test_reproduceExploit \
  --fork-url "$ARCHIVE_RPC_URL" \
  --fork-block-number <EXPLOIT_BLOCK_MINUS_ONE> -vvvv
```

```
function test_reproduceExploit() public {
    // 1. Fork state is already pinned to the block before the exploit tx
    // 2. Re-execute the attacker's exact calldata via vm.prank plus a low-level call,
    //    or reconstruct the call sequence step by step from the Chapter 7 trace
    // 3. Assert the same invariant violation or balance delta the on-chain tx produced
    // 4. Patch the isolated line and re-run to confirm the fix holds
}
```

Never reproduce against a public testnet with the vulnerable contract redeployed and real, watched infrastructure nearby. A mainnet fork is isolated and disposable and cannot itself cause harm, while a live testnet reproduction can leak a working exploit to anyone monitoring testnet mempools before the mainnet incident is contained. Once the forked reproduction passes, it becomes the load-bearing artifact for the report's proof-of-concept section, the regression test the patched code must pass, and, if recovery negotiation is underway (Part 4), the technical proof grounding whatever a responder claims happened.

---

# 9. Attribution and Threat Actor Context

A confirmed root cause answers what broke. Attribution asks who broke it, and it is the chapter where evidentiary rigor matters most, because a wrong attribution can misdirect an entire response effort or, worse, name an innocent party.

## 9.1 Address Clustering and Labeling

EVM address clustering has no direct analog to Bitcoin's common-input-ownership heuristic, since no single EVM transaction spends from multiple unrelated externally owned accounts the way a UTXO transaction can. EVM-specific heuristics instead center on funding relationships and behavioral fingerprints. Funding-source clustering follows the address that supplied a "fresh" wallet's initial gas: addresses that all receive their first ETH from the same centralized-exchange withdrawal or the same funding wallet are strong candidates for common control. Gas-payer clustering flags a single address that repeatedly pays gas for, or deploys, many otherwise-unrelated contracts, common in exploit infrastructure built from a `CREATE2` factory. Behavioral clustering matches interaction patterns: identical function-call sequencing, identical gas-price bidding strategy, or reused calldata templates across addresses that never transact with each other directly. Commercial platforms operationalize these heuristics at scale and layer proprietary off-chain data on top, including historical case linkage and exchange cooperation: Chainalysis, TRM Labs, and Elliptic are the three most cited in public post-mortems, alongside open, community-driven labeling through Etherscan's public name tags and Arkham Intelligence's open intelligence exchange. Treat every cluster as a hypothesis with a stated basis. "Same funding source" is materially weaker evidence than "identical deployment bytecode and identical private-key-derived nonce sequence," and the strength of the underlying heuristic should travel with the label whenever you cite it.

## 9.2 Behavioral Patterns and TTPs

Tactics, techniques, and procedures (TTPs), a framework popularized by MITRE ATT&CK for describing adversary behavior at three levels of abstraction (the strategic goal, the general method, and the specific implementation, each technique carrying a stable identifier such as T1059), transfer naturally to on-chain threat actors. MITRE's newer AADAPT framework, Adversarial Actions in Digital Asset Payment Technologies, applies the same tactics-techniques-procedures structure specifically to blockchain and digital-asset threats, giving forensic teams shared vocabulary for behaviors ATT&CK's enterprise-IT focus does not cover: on-chain laundering paths, cross-chain hopping, and DeFi-specific exploitation sequences among them.

Documented, reusable TTPs for state-linked actors are among the best characterized in the space. The FBI publicly attributed the March 2022 Ronin bridge theft, roughly $624 million, to North Korea's Lazarus Group, and the US Treasury's Office of Foreign Assets Control sanctioned the Tornado Cash mixer in August 2022 partly for its role laundering Lazarus proceeds. Recurring Lazarus TTPs include small test transactions ahead of a large transfer, funds routed through several sequential mixers or cross-chain bridges rather than a single hop, and deliberate splitting of proceeds across many freshly funded addresses before eventual consolidation. Treat a TTP match as corroborating evidence, not proof of identity on its own. Document the specific technique observed, which mixer, which bridge sequence, which timing pattern, rather than asserting attribution as a conclusion the trace alone cannot support.

## 9.3 Use of Public and Private Intelligence

Open-source intelligence for on-chain attribution includes block-explorer public labels, DeFiLlama's hack tracker, rekt.news's incident archive, and independent researcher threads (ZachXBT's public investigations are the most frequently cited example of crowd-sourced attribution reaching conclusions ahead of, or feeding into, official channels). These sources are freely reusable and independently verifiable, since anyone can re-derive the same on-chain facts, but they lag structured commercial data on off-chain corroboration. Paid intelligence platforms, Chainalysis KYT (Know Your Transaction) real-time alerting, TRM Labs, and Elliptic among them, add proprietary exchange cooperation, historical case linkage across investigations, and cluster databases built from years of subpoena-backed data no single researcher can replicate from public sources alone. For an active incident, SEAL 911 coordinates real-time responder access across both tiers: a 24/7 crisis line connecting an affected protocol to whitehat responders, exchange security teams empowered to freeze assets in transit, and forensic analysts carrying the paid tooling to move faster than a single team could alone. Blend both tiers deliberately: cite open evidence for anything a report needs to be independently reproducible, and reserve paid-platform conclusions for internal triage or for corroborating, never solely establishing, a public claim.

### 9.4 Limitations and Confidence Levels

Attribution claims should carry an explicit confidence tier, not an unqualified assertion, because the underlying evidence types vary enormously in strength.

| Confidence tier | Evidentiary basis | Example |
|---|---|---|
| High | Off-chain corroboration ties an on-chain cluster to a real-world identity (subpoenaed exchange KYC, law-enforcement forfeiture filing, self-doxxing) | A DOJ seizure filing naming a specific wallet cluster |
| Medium | Strong, consistent on-chain heuristics (funding source plus a behavioral TTP match) without off-chain confirmation | Multiple addresses funded from one exchange withdrawal, sharing a reused deployment pattern |
| Low | A single weak signal, or a pattern shared across many unrelated actors | Use of a common mixer alone, or a gas-price fingerprint shared across users of a popular wallet app |

The most common false-positive source is shared infrastructure: a popular mixer, a bridge, or even a wallet provider's relayer can create the appearance of common control between addresses that share nothing but a service provider. Cluster confidence also decays over time as published heuristics get gamed: an attacker who reads a forensic report learns exactly which pattern to avoid next time, one reason serious platforms keep some clustering logic proprietary. State the confidence tier next to every attribution claim in a report, and never let a Medium- or Low-confidence cluster read as settled fact in a document that will outlive the investigation.

## 10. Multi-Chain and Cross-Chain Considerations

Value moves across dozens of EVM-compatible chains and the bridges connecting them, and that surface has produced the single largest category of losses industry-wide. This chapter treats bridges as their own forensic discipline and covers the rescue window, the narrow gap between a non-atomic exploit and irreversible laundering during which a response can still matter.

### 10.1 Bridge and Messaging Layer Analysis

A bridge is not one contract; it is a trust system with three legs: a source-chain lock or burn, an off-chain or on-chain verification layer that attests the event happened, and a destination-chain mint or release that trusts that attestation. Forensic analysis of a bridge incident means identifying which leg failed, because both the technical fix and the attribution depend on the answer.

### 10.1.1 Bridge Attack Vectors (Validator Trust, Verification Logic, Access Control)

| Vector | Mechanism | Named incident |
|---|---|---|
| Validator or guardian trust compromise | Attacker obtains enough private keys or signing power to forge a valid-looking attestation | Ronin, March 2022: 5 of 9 validator keys compromised via social engineering and a stale RPC-node permission, roughly $624 million |
| Verification-logic flaw | The verification contract accepts a proof it should reject | Wormhole, February 2022: Solana-side guardian-signature verification failed to check a required system account, letting the attacker mint 120,000 wETH without matching collateral, roughly $326 million |
| Access control failure | A privileged function meant to be locked to the verification layer is reachable by anyone or by a spoofable caller | Nomad, August 2022: a faulty initialization set the trusted message root to zero, so any message with a "proof" matching that default was accepted, producing a chaotic free-for-all as hundreds of copy-paste transactions drained the bridge in parallel, roughly $190 million |

These vectors are not mutually exclusive. The BNB Bridge (Token Hub) incident of October 2022 combined a verification-logic gap in its IAVL Merkle-proof checking with the practical reality that only a handful of validators needed to be fooled before a forged mint could occur, though the chain's operators halted the network within hours and the liquid value an attacker actually extracted came in well under the roughly $586 million face value of the forged mint. When triaging a new bridge incident, isolate which leg (lock or burn, verify, mint or release) accepted bad input first, then ask whether the failure required compromising a keyholder (validator trust) or merely finding a logic gap any observer could exploit once known (verification or access control). The two categories imply very different remediation and very different attacker sophistication.

### 10.1.2 Bridge as Largest Source of Web3 Losses: Taxonomy

| Incident | Date | Reported loss | Primary vector |
|---|---|---|---|
| Ronin Network | Mar 23, 2022 | ~$624M | Validator key compromise |
| Poly Network | Aug 10, 2021 | ~$611M | Access control (cross-chain manager) |
| BNB Bridge (Token Hub) | Oct 6, 2022 | ~$586M face value | Verification logic (forged proof) |
| Wormhole | Feb 2, 2022 | ~$326M | Verification logic (signature check) |
| Nomad | Aug 1, 2022 | ~$190M | Access control (trusted root default) |

| Incident | Date | Reported loss | Primary vector |
|---|---|---|---|
| Harmony Horizon Bridge | Jun 23, 2022 | ~$100M | Validator key compromise (multisig) |

*Source: rekt.news leaderboard.*

These six incidents alone total more than $2.4 billion, and chain-analytics firms' annual crime research has repeatedly flagged cross-chain bridges as a disproportionate share of total industry losses in the years those incidents concentrated (Chainalysis crypto crime research). The driver is structural: a bridge often custodies the full collateral backing every wrapped asset in circulation, while its actual security model tends to be thin relative to that value, a 5-of-9 or 2-of-5 multisig securing nine or ten figures is not unusual. Both the MITRE AADAPT framework and OWASP's own SCSVS-BRIDGE domain treat cross-chain messaging as a distinct control category for exactly this reason: the failure modes cluster differently than single-chain contract bugs and warrant a dedicated forensic checklist rather than a generic one.

## 10.2 Atomic vs. Non-Atomic Attacks: Rescue Window

An atomic attack, a flash-loan sandwich that borrows, manipulates, profits, and repays within one transaction, completes and often begins laundering before any human observer notices anything abnormal. There is nothing to intercept, because the attacker never holds the stolen funds in a vulnerable, static location. A non-atomic attack instead leaves funds sitting in an attacker-controlled address, sometimes for minutes, sometimes for days, before they move toward a mixer or a cross-chain exit. That gap is the rescue window, and it is the single variable that most determines whether a loss becomes a recoverable incident (Part 4) or a permanent one.

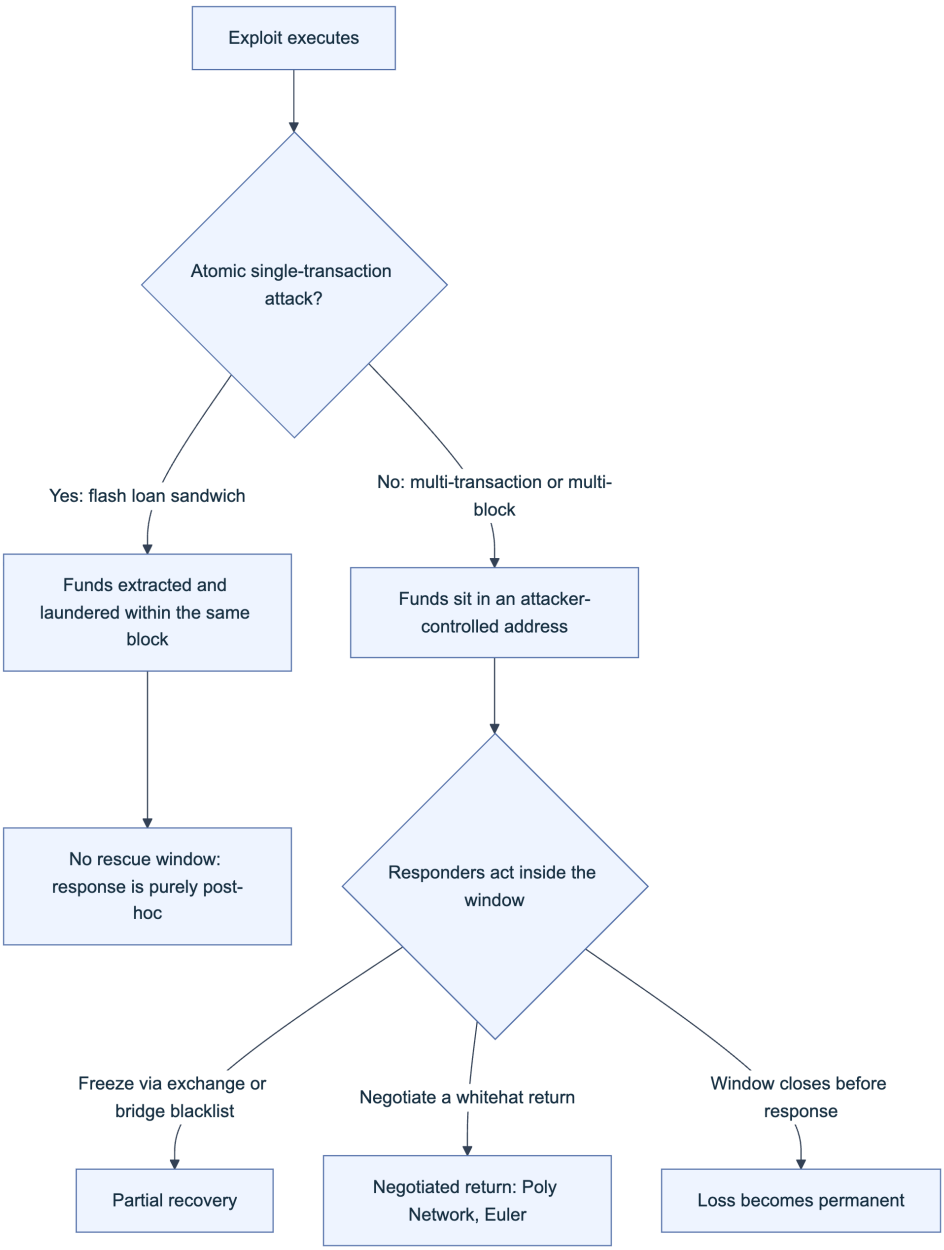


*Figure 8. The rescue window decision path. Detection speed is the one variable a responder fully controls once the atomic-versus-non-atomic branch has already been decided by the exploit's design.*

### 10.2.1 When Execution Is Not Atomic (Rescue Time Frame)

Execution stops being atomic the moment an exploit requires more than one transaction, more than one block, or deliberate attacker patience. Common triggers include a timelock the attacker must wait out to claim governance-controlled funds, a multi-step unstaking or vesting mechanism, or simply an attacker choosing to sit on stolen assets rather than immediately cashing out, to avoid triggering automated freeze alerts. The rescue window's real length is set by three factors: how quickly the affected team detects the incident and engages exchanges or bridge operators capable of freezing the specific asset in transit, how quickly the attacker actually attempts to move or launder the funds, and whether the stolen asset is itself freezable at all (a centralized stablecoin issuer can blacklist an address; a native asset or most DeFi tokens cannot).

Poly Network's August 2021 incident is the clearest illustration of a long window used well: roughly $611 million was exploited through a single access-control failure, but the attacker did not immediately launder the proceeds, and over the following two weeks public pressure, on-chain tracking, and direct negotiation (the attacker, nicknamed "Mr. White Hat" by the project, ultimately characterized the action as a demonstration rather than theft) produced a near-complete return of assets. Euler Finance's March 2023 incident shows a shorter but still workable window: after roughly $197 million was drained through a flawed donation-and-liquidation sequence, the Euler team opened direct on-chain negotiation, and the attacker returned the substantial majority of funds within about two weeks. KyberSwap's Elastic exploit in November 2023 illustrates the opposite outcome: funds began moving toward mixers within hours, the window closed fast, and the attacker's subsequent demands, publicly reported as an ultimatum for governance control of the protocol rather than a conventional negotiated return, left recovery contested rather than resolved. Everything in Part 4's recovery playbooks assumes the window identified here is still open when that work begins.

## 10.3 Consistency of Evidence Across Chains

Evidence gathered on one chain is only as solid as that chain's finality guarantee at the moment you captured it. Ethereum mainnet reaches probabilistic finality quickly but cryptographic finality under Casper FFG only after roughly two epochs, on the order of 13 to 15 minutes; many EVM-compatible chains, several Layer 2 (L2) networks before finalizing to their settlement layer and a number of sidechains among them, offer weaker or purely probabilistic finality and stay reorg-prone for meaningfully longer, so a transaction, or even a full block, pulled moments after inclusion can still be reorganized out before the report is written. Cross-check evidence against at least two independently operated RPC endpoints, a self-hosted node plus a commercial provider such as Alchemy, Infura, or QuickNode, rather than trusting a single source, and record the block hash alongside the transaction hash in every artifact so a later reorg becomes immediately detectable by comparing the recorded hash to the canonical chain.

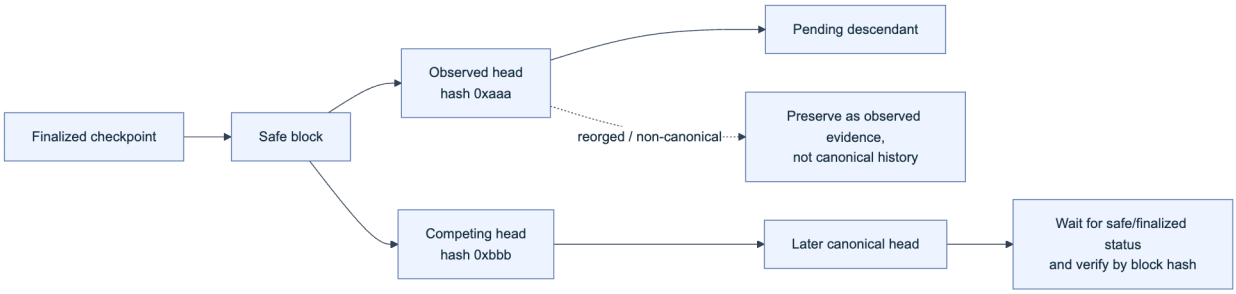


*Figure. Reorganization-aware evidence handling. A block observed at the head can later become non-canonical when fork choice selects a competing branch. Record its block hash and status at collection time, preserve the raw observation, and do not describe it as canonical or finalized until independent consensus-aware sources agree.*

Block explorers built on different indexers, Etherscan's proprietary stack versus Blockscout's open-source indexer, sometimes disagree at the margins (decoded event names, token metadata, pending-state display) even when underlying chain data matches, so treat explorer output as a convenience layer and the raw RPC response as ground truth whenever a finding's precision matters. For multi-chain incidents, timestamp every artifact in UTC and note each source chain's finality status explicitly: "confirmed, N confirmations" is not the same claim as "finalized," and a report that conflates them can later be shown to rely on evidence that no longer exists.

## 10.4 Tooling for L2s and Alternative EVM Chains

The Etherscan family operates parallel, separately branded explorers for most major EVM chains: Arbiscan for Arbitrum, Optimistic Etherscan for OP Mainnet, BscScan for BNB Chain, PolygonScan for Polygon, and Basescan for Base, each with its own API key and rate limits despite a broadly shared interface, so budget for per-chain account setup rather than assuming one key works everywhere. Blockscout is the dominant open-source alternative and the default explorer for many smaller rollups and appchains that never receive an Etherscan-family deployment; treat its self-hosted, per-chain instances as authoritative on chains where no commercial explorer exists. Trace-API availability is the biggest practical gap: `debug_traceTransaction` and its equivalents are disabled on most free public L2 RPC endpoints just as they are on mainnet, and call-level tracing on a rollup typically requires a paid archive-tier plan from a provider like Alchemy or QuickNode, or a self-run node. Use chainlist.org or the underlying ethereum-lists/chains registry to confirm the correct chain ID and canonical RPC endpoint before pulling data; a surprising number of forensic errors trace back to querying a similarly named but unrelated network. For cross-chain aggregate views, Dune Analytics dashboards and L2Beat's risk and activity data give a faster first read on a rollup's bridge design and total value secured than reading its contracts cold.

## 10.5 Real-Time Forensics and Mempool Monitoring

An incident caught while the attacker's follow-on transactions are still pending gives a responder options that vanish once everything is confirmed and mixed. Blocknative's Mempool Explorer and a self-run `eth_subscribe("newPendingTransactions")` listener both expose the public mempool in real time, letting a responder catch a second-stage transaction, draining a related contract or bridging stolen funds out, before it lands. A meaningful share of Ethereum's transaction flow now bypasses the public mempool entirely through private order-flow channels; Flashbots Protect and MEV-Share (maximal extractable value, MEV) route transactions directly to block builders to avoid front-running, which also means a sophisticated attacker's follow-on moves may never appear in a public mempool watcher at all. Plan monitoring coverage for both public and private order flow rather than assuming one channel is sufficient. EigenPhi and similar MEV-analytics platforms index extractable-value activity, sandwich attacks, arbitrage, and liquidations, in near real time and help distinguish an ordinary MEV bot's opportunistic transaction from a targeted follow-on exploit transaction that merely looks similar at a glance. The Forta Network runs a de-

centralized set of real-time detection bots against live transaction streams and can alert a protocol team to an anomalous call pattern against their own contracts before a human notices the drop in total value locked (TVL). None of these tools substitute for the forked, offline reproduction in Section 8.6; they cover the narrow, time-critical window where a live response is still possible, and they should feed directly into the rescue-window decisions from Section 10.2.

---

**Key controls for Part 3**

- Reconstruct execution with a call-level trace (`debug_traceTransaction` / `callTracer` or equivalent) before drawing conclusions from the top-level receipt alone.
- Map every finding to its OWASP SCWE identifier and SCSVS domain so root-cause claims stay independently verifiable and comparable across incidents.
- Treat a manual trace read as a hypothesis; only a forked, executed reproduction (Section 8.6) counts as confirmed evidence.
- Attach an explicit confidence tier (High/Medium/Low) to every attribution claim, and never let heuristic clustering read as settled identity.
- For bridge and non-atomic incidents, clock the rescue window immediately; detection speed is the one variable a responder fully controls.

# Part IV: DeFi Recovery Patterns

Parts II and III built a root-cause finding from raw evidence. This part answers the question a responder asks the moment that finding lands on the table: what, if anything, can still be done. Decentralized Finance (DeFi) recovery is not a single technique but a small set of recurring patterns, pause and upgrade, governance-driven parameter fixes, treasury-funded compensation, fork and migration, direct negotiation, and pre-authorized whitehat rescue, each with its own preconditions, timeline, and failure mode. The chapters below catalog each pattern against the incidents that defined it, so a responder facing a live loss can match the situation to a precedent instead of improvising one.

## 11. DeFi Recovery Patterns Database (Concept and Use)

A recovery patterns database turns scattered post-mortems into a repeatable decision aid. This chapter defines what belongs in it, the hard limits on what recovery can achieve, and the legal ground a responder must check before acting on any pattern in Chapters 12 through 17.

### 11.1 Purpose of a Recovery Patterns DB

Every DeFi exploit produces the same panic: a team with minutes to decide, incomplete information, and a community watching the treasury drain in real time. A patterns database exists to remove the guesswork from that moment by pre-recording what worked, what did not, and under what conditions each pattern applied. This mirrors how incident response matured in traditional security: the NIST Cybersecurity Framework 2.0 formalizes a “Recover” function (categories RC.RP for recovery planning and RC.CO for recovery communications) precisely because ad hoc recovery under pressure produces worse outcomes than a rehearsed plan, and DeFi’s public, adversarial, irreversible-by-default environment raises the stakes on that lesson rather than lowering them.

Each entry in a working database should record four things: the applicability trigger (what conditions must hold for this pattern to be usable at all), the prerequisite infrastructure (a pause switch, a timelock, a treasury, a pre-published recovery address), the realistic timeline from decision to execution, and at least one documented precedent with its outcome. The table below is the roadmap for the rest of this part; each row becomes its own chapter.

| Pattern | Core precondition | Typical timeline | Representative precedent |
|---|---|---|---|
| Pause and upgrade | Contract has a pause switch or is behind an upgradeable proxy | Minutes to pause, days to patch and upgrade | Widely used emergency-admin pattern across lending and DEX protocols |
| Governance and parameter changes | A timelock or multisig with authority over the broken parameter | Hours to days, gated by timelock delay | Compound's 2021 comptroller over-distribution required a governance-path fix |
| Compensation | Sufficient treasury, insurance fund, or external backstop | Weeks to design and distribute | Fei Protocol's Tribe DAO reimbursement after the April 2022 Rari Fuse exploit |
| Fork and migration | Broad community consensus that the alternative is unacceptable | Weeks to months | The DAO hard fork, July 2016 |
| Negotiation | Attacker reachable, funds not fully laundered | Days to weeks, no guaranteed outcome | Poly Network, August 2021; Euler Finance, March 2023 |
| Whitehat safe harbor | Pre-authorization exists before the incident occurs | Minutes during the live exploit | Security Alliance Whitehat Safe Harbor Agreement adopters |

## 11.2 When Recovery Is Feasible vs. When It Is Not

The single largest determinant of feasibility is whether the attack was atomic. A flash-loan-funded exploit that borrows, manipulates, drains, and repays within one transaction leaves no window: by the time the transaction is mined, the funds may already be mid-route through a decentralized exchange, a cross-chain bridge, or a mixing service, and nothing short of downstream freezing by a centralized counterparty can touch them. A non-atomic attack, by contrast, one that requires the attacker to wait for a timelock, move funds across multiple blocks, or hold assets in an intermediate address before laundering, leaves a real window during which a pause, a race transaction, or a negotiation can still change the outcome. Poly Network's August 2021 attacker left roughly six hundred million dollars sitting in identifiable addresses for days rather than exiting immediately (rekt.news, Poly Network), which is precisely why negotiation worked there and would not have worked against a single-block sandwich attack.

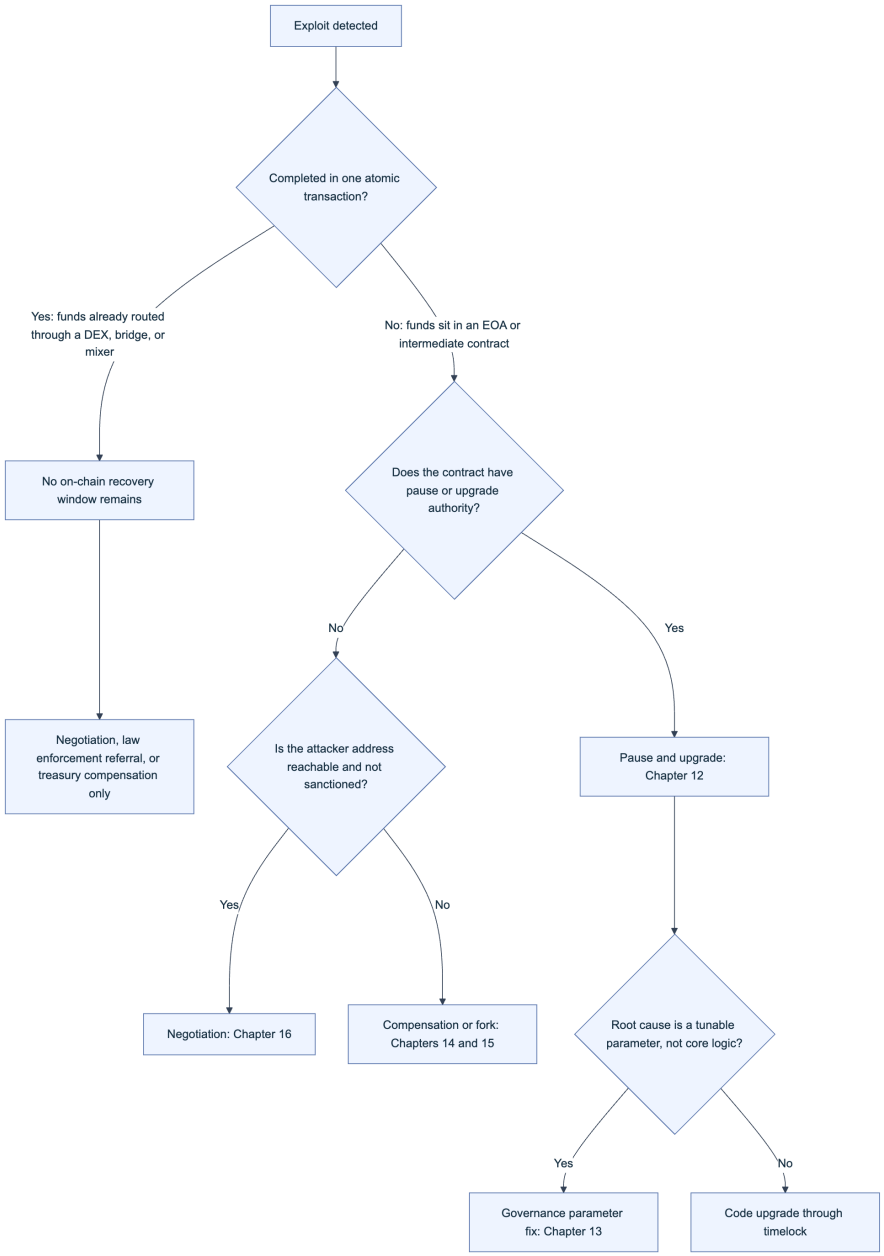


*Figure 1. Feasibility decision path from exploit detection to the applicable recovery pattern. Atomicity is evaluated first because it determines whether any on-chain window exists at all.*

Beyond atomicity, feasibility depends on four further factors: whether unaffected user funds remain exposed to a repeat of the same bug, whether the protocol's remaining treasury can absorb a compensation shortfall if other patterns fail, whether the attacker's address has surfaced on a sanctions list (Section 11.3), and whether the team retains the legal and technical authority (a live multisig, an unexpired timelock role, an EOA (Externally Owned Account) that still controls upgrade rights) to act at all. A protocol that renounced its admin keys for decentralization credibility has, by that same choice, eliminated pause and upgrade as options before any incident occurs.

## 11.3 Legal and Governance Constraints

Recovery is a legal act before it is a technical one, and skipping this check has cost responders dearly. Three constraints recur across every incident in this handbook. First, sanctions screening: paying, negotiating with, or routing recovered funds through an address on the U.S. Treasury's Specially Designated Nationals list is itself a violation independent of intent, a risk made concrete when the Treasury sanctioned Tornado Cash in August 2022 (Treasury press release) and when the FBI publicly attributed the March 2022 Ronin bridge theft to the Lazarus Group (rekt.news, Ronin), a Democratic People's Republic of Korea (DPRK) state-linked actor already under sanction. Screen every counterparty address against the OFAC sanctions list before any negotiation opens.

Second, criminal exposure survives restitution. Returning stolen funds, even under a negotiated governance vote, does not retroactively authorize the original unauthorized access under the U.S. Computer Fraud and Abuse Act (CFAA) or equivalent statutes elsewhere; prosecutors pursued the Mango Markets exploiter on fraud and market manipulation charges after a Mango DAO governance vote had already let him keep a negotiated share of the funds as a "bounty" (rekt.news, Mango Markets). Third, governance capacity: verify that the entity acting, whether a foundation, a Decentralized Autonomous Organization (DAO) with a legal wrapper, or a multisig, actually has standing to negotiate, sign a release, receive recovered assets, and distribute compensation without triggering securities-law exposure on the distribution itself.

**Legal and governance pre-flight checklist**

- ☐ Attacker address(es) screened against OFAC and equivalent sanctions lists before any contact
- ☐ Legal entity with standing identified (foundation, wrapped DAO, or named multisig signers)
- ☐ Outside counsel briefed before any public offer of a bounty, amnesty, or "no prosecution" statement
- ☐ Evidence chain from Parts II and III preserved unmodified regardless of negotiation outcome
- ☐ Jurisdictional exposure assessed for every signer authorizing a compensation payout

# 12. Recovery Pattern: Pause and Upgrade

Pause and upgrade is the fastest pattern available and the only one most teams can execute unilaterally, but it only ever stops a bleed, it never reverses one.

## 12.1 Applicability (Upgradeable Proxies, Pausable)

This pattern requires two pieces of infrastructure to already be in place before the incident, not built during it. First, an emergency stop, typically OpenZeppelin's `Pausable` modifier, which gates state-changing functions behind a `whenNotPaused` check and gives a designated guardian role the ability to flip it (OpenZeppelin Pausable docs). Second, an upgrade path, most commonly a Transparent or UUPS (Universal Upgradeable Proxy Standard) proxy that separates the contract's storage from its logic, letting a new implementation replace the buggy one without redeploying the address every integration and user already trusts (OpenZeppelin UUPS docs).

```
// Minimal emergency-stop pattern, OpenZeppelin Contracts v5
contract VaultCore is Pausable, AccessControl {
    bytes32 public constant GUARDIAN_ROLE = keccak256("GUARDIAN_ROLE");

    function withdraw(uint256 amount) external whenNotPaused {
        // normal withdrawal logic
    }

    function pause() external onlyRole(GUARDIAN_ROLE) {
        _pause(); // callable by a low-threshold emergency multisig
    }
}
```

A protocol with neither control has already forfeited this pattern regardless of how fast the team reacts; that gap should be flagged as a finding in its own right, not discovered mid-incident.

## 12.2 Steps and Considerations

Sequence matters more than speed alone. Pause first, before root cause is fully confirmed: a pause is cheap, reversible, and stops further loss immediately, while waiting for certainty before pausing lets the exploit continue. Once frozen, confirm scope using the transaction and flow analysis techniques from Part III, then develop a patch and validate it against a mainnet fork (Anvil or Hardhat's forking mode) that replays the actual exploit transaction to prove the fix closes it. Only after fork-testing should the fix reach a timelock or multisig for execution, and unpausing should happen deliberately, with monitoring active, rather than automatically once the upgrade lands.

1. Pause immediately on confirmed anomaly, before full root cause is known
2. Confirm exploit scope and remaining exposure using Part III's transaction-tracing methodology

3. Develop and fork-test the patch against the actual exploit transaction
4. Route the upgrade through the normal timelock or emergency multisig process (Chapter 13)
5. Unpause deliberately, with heightened monitoring for a defined observation window

### 12.3 Risks and Mitigations

The same authority that lets a team stop an exploit lets it stop legitimate withdrawals, and that dual-use nature is the pattern's central risk. Bridges and lending protocols have faced sustained community criticism for pause powers that read, in the fine print, as broad enough to freeze user funds indefinitely with no on-chain guarantee of a timely unpause. A single admin key controlling pause is also a single point of failure: if that key is compromised, an attacker can pause the protocol as a denial-of-service action or, worse, unpause a still-vulnerable contract to continue draining it.

| Risk | Mitigation |
|---|---|
| Pause used to grief users or freeze funds indefinitely | Bind pause to a published maximum duration or require a governance vote to extend it |
| Single admin key compromise | Require an N-of-M multisig for pause; require full timelock governance for upgrade |
| Partial pause coverage leaves sibling contracts exposed | Enumerate all contracts sharing the vulnerable logic or shared state before declaring the pause complete |
| Upgrade deployed without fork-testing against the real exploit tx | Make fork-replay a mandatory, checked step before any upgrade transaction is signed |

## 13. Recovery Pattern: Governance and Parameter Changes

Where pause and upgrade fixes code, this pattern fixes the numbers around it, and it depends on the same timelock infrastructure that keeps governance itself from becoming an attack surface.

### 13.1 Using Timelock and Multisig

A timelock contract enforces a mandatory delay between a governance vote passing and its execution, giving the community a window to review, and if necessary exit, before a change takes effect. OpenZeppelin's `TimelockController`, derived from the pattern Compound popularized in its original `Timelock.sol`, is the de facto standard (OpenZeppelin TimelockController docs). Multisig wallets, most commonly Safe, hold the proposer and executor roles that interact with the timelock, adding an N-of-M human approval layer on top of the delay (Safe documentation).

```
# Schedule a timelocked parameter change, then execute after the delay
cast send $TIMELOCK "schedule(address,uint256,bytes,bytes32,bytes32,uint256)" \
  $TARGET 0 $CALLDATA $PREDECESSOR $SALT $DELAY --account guardian

# After the delay elapses, any account holding EXECUTOR_ROLE can execute
cast send $TIMELOCK "execute(address,uint256,bytes,bytes32,bytes32)" \
  $TARGET 0 $CALLDATA $PREDECESSOR $SALT --account executor
```

## 13.2 Parameter Fixes (Oracle, Slippage, Caps)

Not every root cause requires new code. Oracle staleness thresholds, price-deviation circuit breakers, slippage tolerance bounds, deposit and borrow caps, and loan-to-value (LTV) ratios are all values a governance proposal can change without touching contract logic at all, provided the contract was built with those values as adjustable parameters rather than hardcoded constants. Compound's September 2021 comptroller bug, which over-distributed roughly eighty million dollars of COMP tokens to users through a flawed reward-index change, is the reference case for why parameter and reward-logic fixes belong in this pattern: the correction itself required a governance proposal, and because the excess tokens had already left the protocol, the team's only further lever was a public appeal asking recipients to voluntarily return the overpayment, an outcome that blends this pattern with the negotiation pattern in Chapter 16.

| Parameter class | Failure mode it addresses | Example fix |
|---|---|---|
| Oracle heartbeat / staleness bound | Stale price feed accepted as current | Tighten maximum acceptable feed age |
| Price deviation circuit breaker | Single-block price manipulation accepted | Reject updates beyond a bounded percentage move |
| Slippage tolerance | Sandwich or manipulation profit from wide tolerance | Lower default or enforce a hard maximum |
| Deposit / borrow caps | Uncapped exposure to a single asset or market | Cap new deposits pending a full fix |
| LTV ratio | Under-collateralized positions survive a price shock | Reduce LTV for the affected collateral type |

## 13.3 Communication and Execution

A public timelock is, by design, a public countdown, and that transparency cuts both ways. Publicizing a patch before the timelock executes tells an attacker exactly how long they have left to exploit the still-live bug, so disclosure timing must weigh the community's right to know against the risk of accelerating the very attack the fix addresses. Where the vulnerability remains actively exploitable, prefer executing through an emergency-authorized path first (Chapter 12) and reserve the full governance narrative, forum post, and security advisory for after the fix is live. Publish a post-mortem once the immediate risk has passed, following the same evidence-preservation discipline as Part II, since that document becomes the primary public record for the case-study catalog in Chapter 18.

**Communication sequencing checklist**

- ☐ Confirm the fix is deployed or the emergency path is active before any public disclosure
- ☐ Draft the security advisory before the vote, not after, so execution is not delayed by word-smithing
- ☐ Post to governance forum and official channels simultaneously to avoid information asymmetry
- ☐ Publish the full post-mortem only after the immediate exploit window has closed

# 14. Recovery Pattern: Compensation and User Funds

When neither pausing the code nor renegotiating the outcome restores what was lost, compensation shifts the loss from users onto the protocol's own balance sheet or its backers.

## 14.1 Identifying Affected Users and Amounts

The first technical task is reconstructing exactly who lost what, at the block immediately before the exploit, using an archive node query, a subgraph snapshot, or a Dune Analytics query against indexed event logs. Two edge cases routinely complicate this. The attacker's own address, and any address that benefited indirectly from the exploit's side effects (an arbitrageur who happened to profit from the resulting price dislocation, for instance), must be excluded from any compensation set even though they technically held a balance at the snapshot block. And users who sold into a depegged or crashed asset price before the snapshot, realizing a loss the exploit caused without ever holding the affected token at the reference block, raise a fairness question about whether "affected" should be defined by balance-at-snapshot or by demonstrated loss, a distinction every compensation plan must state explicitly before publishing a snapshot.

## 14.2 Treasury and Insurance Considerations

Funding a compensation plan draws on one of several sources, each with a different speed and a different cost to someone who was not at fault for the loss. Fei Protocol's Tribe DAO voted to use protocol-owned treasury funds to reimburse users after the April 2022 Rari Fuse exploit drained roughly eighty million dollars, eventually winding the protocol down entirely and using remaining reserves to make holders whole through a redemption mechanism (rekt.news, Rari Capital). Jump Crypto took a different path after the February 2022 Wormhole bridge exploit, replenishing roughly 120,000 wrapped ETH from its own balance sheet within days to keep the bridge solvent, a private backstop rather than a protocol-native fund (rekt.news, Wormhole).

| Funding source | Speed | Primary tradeoff |
|---|---|---|
| Protocol treasury / protocol-controlled value | Fast if liquid | Depletes reserves needed for future operations |
| On-chain insurance fund (e.g., mutual cover products) | Depends on claims process | Coverage limits and claim disputes can exceed the loss window |
| Token mint or dilution | Fast to authorize | Dilutes every non-affected holder, moral hazard for future risk-taking |
| External capital injection (VC, foundation, exchange) | Fast if a backer commits | Creates a dependency and a precedent that may not repeat |

## 14.3 Distribution Mechanisms and Fairness

Once funds are sourced, distribution should be verifiable without exposing every claimant's balance in a single public transaction. A Merkle-tree claim contract, following the same pattern used for large token airdrops, lets each user prove their allocation against a single on-chain root while claiming individually and at their own pace.

```
// Merkle-based compensation claim, structure follows OpenZeppelin's MerkleProof lib-
rary
function claim(uint256 index, address account, uint256 amount, bytes32[] calldata
proof) external {
    require(!claimed[index], "already claimed");
    bytes32 leaf = keccak256(abi.encodePacked(index, account, amount));
    require(MerkleProof.verify(proof, merkleRoot, leaf), "invalid proof");
    claimed[index] = true;
    token.transfer(account, amount);
}
```

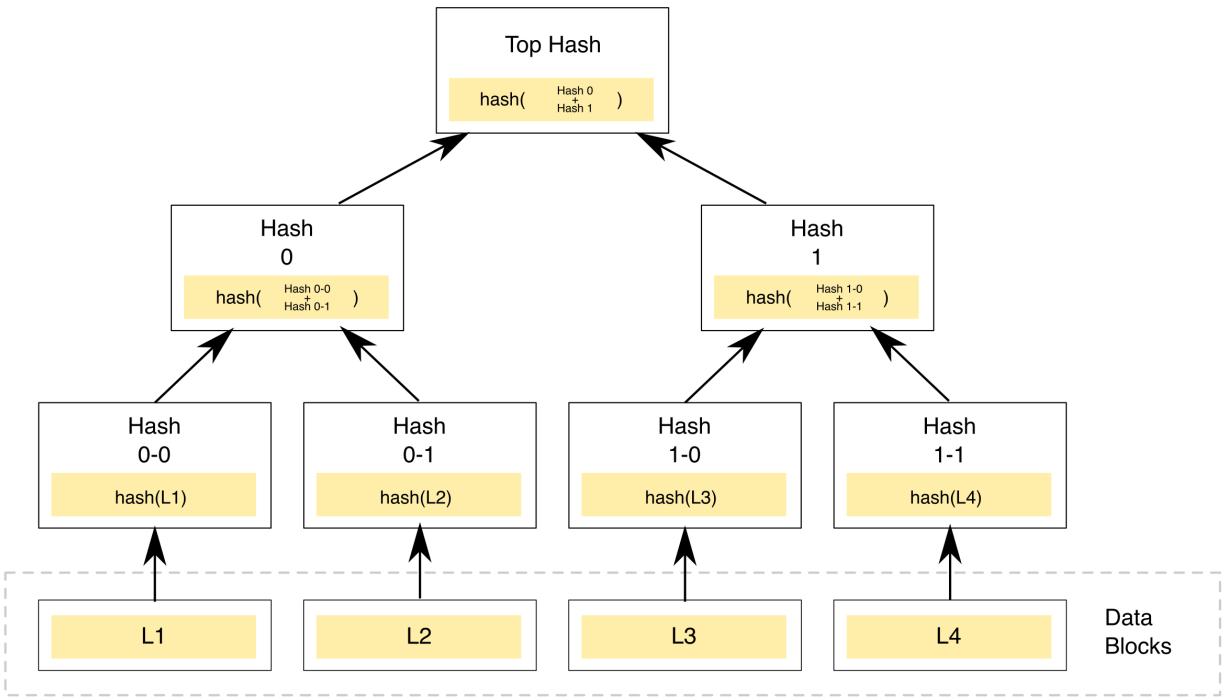


*Figure. The structure the claim contract above verifies against: each leaf commits to one user's allocation, the root is the only value published on-chain, and a proof is just the sibling hashes along the path from a user's leaf to that root. Source: Wikimedia Commons, CC0 (public domain).*

Fairness in the distribution design matters as much as the funding source. Favor pro-rata distribution over flat per-address amounts so a large holder and a small holder are treated by the same rule, consider a vesting or staged schedule if the payout token itself would crash under immediate sell pressure from every claimant exiting at once, and publish the snapshot methodology and exclusion criteria alongside the Merkle root so affected users can independently verify their own inclusion and amount before claiming.

# 15. Recovery Pattern: Fork and Migration

Fork and migration is the pattern of last resort: it works, but it costs the protocol its claim to a single, uncontested canonical state, and at the blockchain level it can split a community permanently.

## 15.1 When to Consider a Fork

Reserve this pattern for cases where every faster option has failed and the community judges the loss severe enough to threaten the protocol's survival or violate a foundational expectation of its users. The canonical case operates at the blockchain level rather than the application level: after The DAO was drained of roughly 3.6 million Ether in June 2016 through a reentrancy bug, the Ethereum community executed an irregular state-change hard fork at block 1,920,000 to return the funds, an intervention completed in July 2016 that produced the permanent Ethereum and Ethereum Classic split and left "code is law" versus pragmatic intervention as a live, unresolved tension in the ecosystem ever since (Ethereum Foundation, hard fork announcement). No Ethereum mainnet fork of this kind has recurred since; the bar for chain-level intervention rose specifically because of how contested that outcome remained.

A far more common and far less controversial variant operates at the application level: a single protocol deploys fresh contracts with the vulnerability fixed and migrates its own users, without asking the underlying blockchain to do anything unusual. Beanstalk relaunched under new governance after its April 2022 flash-loan governance attack drained roughly one hundred eighty million dollars in a single transaction that exploited the absence of any timelock on emergency proposals (rekt.news, Beanstalk), and Cover Protocol migrated its token entirely after a December 2020 infinite-mint bug. Application-level fork and migration requires no chain-wide consensus, only the protocol's own governance and its users' willingness to move.

## 15.2 Snapshot and Migration Design

Execution follows a consistent sequence regardless of scale. Freeze the old contract using the pause pattern from Chapter 12 wherever possible, take a precise state snapshot at a specific block (balances, staked positions, LP tokens, any in-flight or locked state), deploy the new contract with the fix incorporated and independently reviewed, and build a migration path that lets users claim their position in the new system rather than pushing an unrequested transfer to every address. Self-service claiming avoids sending value to addresses that may no longer be controlled by their original owner, and it gives each user a chance to review the new contract before committing further funds to it.

**Migration design checklist**

- ☐ Old contract paused or otherwise frozen before the snapshot block is finalized
- ☐ Snapshot captures every position type: direct balances, LP tokens, staked and locked positions
- ☐ New contract independently reviewed and fork-tested before migration opens
- ☐ Claim mechanism is user-initiated, not a push transfer, to avoid sending funds to compromised or abandoned addresses
- ☐ Legacy contract's read functions remain available post-migration for historical verification

### 15.3 Communication and Community Coordination

A fork's legitimacy rests entirely on the community accepting it as legitimate, which makes communication inseparable from execution. Put the decision to a governance vote before executing, even when the team has the raw technical authority to migrate unilaterally, since a fork imposed without consensus reproduces the exact controversy that made Ethereum Classic a permanent, competing chain rather than a footnote. Run parallel communication across every channel the community actually uses (governance forum, X, Discord, and an in-app banner on the live front end), publish a firm timeline for when the old contract stops being supported, and plan explicitly for a dissenting minority that may choose to keep using the old, unpatched contract rather than migrate, a possibility the Ethereum and Ethereum Classic split shows is not merely theoretical.

---

## 16. Recovery Pattern: Negotiation and Third Parties

Negotiation treats the attacker as a party to bargain with rather than purely a target to prosecute, an uncomfortable posture that has nonetheless recovered more stolen value than any other single pattern in this handbook's case history.

### 16.1 When and How to Engage (Negotiation, Bounty, Legal)

Engagement channels range from the informal to the fully legal. The most common opening move is an on-chain message: a zero-value transaction to the attacker's address carrying a hex-encoded or ASCII memo in the calldata, or a comment left through a block explorer's messaging feature, both of which the attacker can read without needing any off-chain contact information. If a direct channel opens, teams typically offer a formal bounty, often referencing Immunefi's standard whitehat framing even when the actor is not acting in good faith by the bounty program's own definition (Immunefi). Where negotiation stalls or the amount is large enough to warrant it, referral to law enforcement (the FBI's IC3 for U.S. incidents) and engagement with blockchain analytics firms that assist with exchange-level fund freezing run in parallel, not as a replacement for direct contact.

| Channel | Typical speed | Leverage | Precedent |
|---|---|---|---|
| On-chain memo / calldata message | Immediate | Low alone, opens dialogue | Poly Network, August 2021 |
| Formal bounty offer | Days | Moderate, frames a face-saving exit for the attacker | Euler Finance, March 2023 |
| Law enforcement referral | Weeks to months | High, but slow and jurisdiction-dependent | Ronin bridge, FBI attribution |

| Channel | Typical speed | Leverage | Precedent |
|---|---|---|---|
| Exchange-level freezing via analytics firms | Hours to days if funds transit a centralized rail | High for the frozen portion only | Various post-hack fund tracing efforts |

### 16.2 Documentation and Evidence for Negotiations

Every claim made during a negotiation, and every promise the attacker makes in return, needs the same evidentiary rigor as the forensic record in Parts II and III: transaction hashes cited precisely, wallet-clustering conclusions stated with their confidence level, and a timestamped log of every message exchanged, whether on-chain or through an off-chain channel the attacker opens. This record serves two purposes that outlast the negotiation itself. It lets the team hold the attacker to specific, quotable terms if the negotiation later becomes a legal dispute, and it preserves a clean chain of custody regardless of whether the negotiation succeeds, since a failed negotiation still needs to hand off to law enforcement with an intact evidence trail rather than one that negotiation activity has muddied.

### 16.3 Ethical and Legal Boundaries

Negotiation sits close to several lines a responder should not cross, however tempting the pressure to recover funds quickly. Threats to dox, sue, or otherwise punish an attacker as leverage to compel a return can themselves cross into extortion depending on jurisdiction, so keep any pressure framed as a good-faith offer rather than a threat. Sanctions screening (Section 11.3) applies here with full force: paying a bounty to an address later shown to be linked to a sanctioned entity is a violation regardless of when the link was discovered. Critically, agreeing to a bounty after the fact does not retroactively authorize the original unauthorized access under the CFAA or comparable statutes, which is exactly why Chapter 17's safe harbor pattern requires authorization to exist before an incident, not after one. Finally, treat every negotiated promise as non-binding in practice: neither the Nomad bridge's ten percent whitehat bounty program nor Euler Finance's public appeal carried an enforceable contract, and both relied on public pressure and reputational cost rather than legal compulsion to secure the return of funds.

## 17. Safe Harbor and Whitehat Recovery

Safe harbor inverts the timing problem that makes Chapter 16 so uncertain: instead of negotiating authorization after an exploit has already happened, a protocol grants it in advance, so a rescuer's good-faith intervention is lawful and welcomed the moment it happens rather than argued about afterward.

## 17.1 Pre-Authorized Whitehat Rescue During Active Exploits

A Whitehat Safe Harbor Agreement is a standing policy, adopted through a protocol's own governance process before any incident occurs, that authorizes any security researcher who discovers an active, ongoing exploit to intervene and move at-risk funds to safety under defined conditions: act only to prevent further loss, do not exploit beyond the scope needed to secure funds, notify the team promptly, and return the principal within an agreed window in exchange for a pre-set bounty. The Security Alliance (SEAL), a nonprofit security collective, publishes the most widely referenced version of this agreement, developed with input from legal and security practitioners across the industry (Security Alliance; Whitehat Safe Harbor Agreement repository), and Immunefi separately maintains whitehat-friendly bounty terms that several protocols reference alongside or instead of SEAL's agreement (Immunefi).

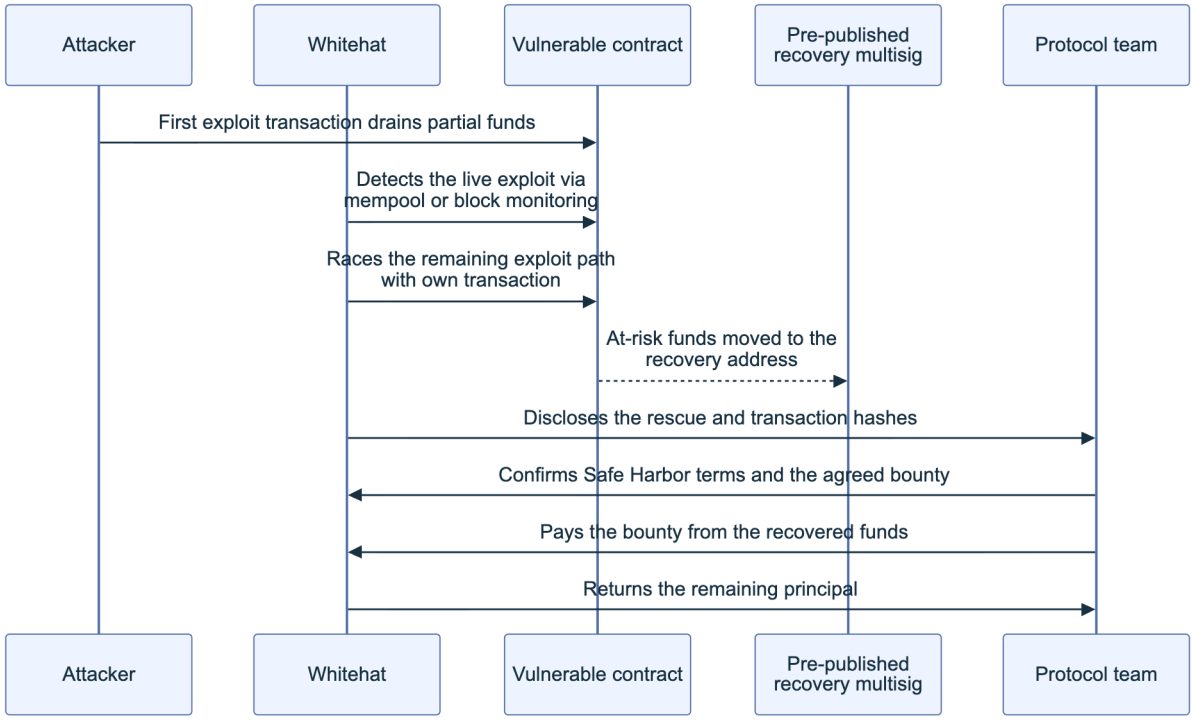


*Figure 2. A pre-authorized whitehat rescue racing an active exploit. Because authorization predates the incident, the rescuer's intervention is a defined, welcomed action rather than a second unauthorized access to litigate later.*

## 17.2 Recovery Addresses and Return of Funds; Bounty Terms

The recovery address a rescuer sends funds to must be published and verifiable before any incident, never improvised mid-crisis: an address announced for the first time during active chaos is itself a well-worn phishing vector, and scammers routinely impersonate legitimate rescue efforts by posting look-alike addresses in the same channels where the real team is coordinating. Ideally the recovery address is itself a multisig the community can verify independently, not a single EOA the rescuer or team controls alone. Bounty terms across published agreements converge on similar defaults: roughly ten percent of recovered value, echoing the figure both Curve

Finance's founder offered during the July 2023 Vyper reentrancy incident and Nomad's post-hack whitehat program used to recover a portion of its losses, with a return window commonly set between twenty-four and seventy-two hours from the rescue.

### 17.3 Relationship to Forensics and Evidence Preservation

Authorization does not exempt a rescue from forensic discipline; if anything it raises the bar. Every rescue transaction is itself now evidence that the rescuer must be able to produce on demand: the exact block, transaction hash, and calldata proving the funds moved to the published recovery address and nowhere else, documented with the same rigor Part II applies to an attacker's transactions. A protocol's insurers, its own governance, or law enforcement reviewing the incident afterward will scrutinize the rescue exactly as closely as the original exploit, and a rescuer who cannot produce a clean, timestamped record of their own actions undermines the good-faith claim the safe harbor agreement depends on.

---

## 18. Other Patterns and Case Studies

No single pattern from Chapters 12 through 17 explains most real incidents alone; nearly every well-documented recovery combines two or more, and this chapter closes the part with the comparative view.

### 18.1 Catalog of Short Case Summaries

| Incident | Date | Approx. loss | Pattern(s) applied | Outcome |
|---|---|---|---|---|
| Poly Network | Aug 2021 | $600M+ | Negotiation, public pressure | Nearly all funds returned within weeks (rekt.news) |
| Wormhole | Feb 2022 | ~$325M (120k wETH) | External capital backstop | Jump Crypto replenished the bridge within days (rekt.news) |
| Ronin bridge | Mar 2022 | ~$625M | Compensation via external funding round | Sky Mavis raised backing and reimbursed users over time (rekt.news) |

| Incident | Date | Approx. loss | Pattern(s) applied | Outcome |
|---|---|---|---|---|
| Beanstalk | Apr 2022 | ~$182M | Fork and relaunch | No recovery of original funds; protocol redeployed with governance timelock (rekt.news) |
| Rari Fuse / Fei Protocol | Apr 2022 | ~$80M | Treasury compensation, later wind-down | Tribe DAO reimbursed users from protocol reserves (rekt.news) |
| Nomad bridge | Aug 2022 | ~$190M | Whitehat bounty program, negotiation | Roughly one-fifth of funds recovered via a 10% bounty offer (rekt.news) |
| Mango Markets (Solana, non-EVM) | Oct 2022 | ~$114M | Governance-vote negotiation | Partial return negotiated via DAO vote; exploiter later prosecuted (rekt.news) |
| Euler Finance | Mar 2023 | ~$197M | Negotiation, public pressure | Nearly full restitution within weeks (rekt.news) |
| Curve Finance (Vyper reentrancy) | Jul 2023 | $70M+ | Negotiation, whitehat front-running rescue | Roughly 70% of at-risk funds recovered within days (rekt.news) |

The spread across this table is the argument for keeping a patterns database at all: the same underlying failure class (a bridge exploit, a flash-loan attack, a reentrancy bug) produced wildly different outcomes depending on whether the protocol had a pause switch, a treasury, a reachable attacker, or a safe harbor agreement already in place before the incident occurred.

## 18.2 References to Public Post-Mortems and RCAs

Beyond the individual incident write-ups already cited, several standing resources aggregate root-cause analyses (RCAs) across the whole DeFi incident history and are worth a responder's bookmark. Rekt.news maintains the most complete running database of DeFi exploit post-mortems with financial figures and timelines (rekt.news). DeFiLlama's hacks dashboard tracks cumulative losses by protocol and category with figures pulled largely from the same public post-mortems. Immunefi periodically publishes aggregate loss reporting drawing on its bug bounty and incident data across the protocols it works with (Immunefi). Where a protocol publishes its own first-party post-mortem, as Euler Labs, Nomad, and Curve Finance each did for the incidents above, that document remains the highest-fidelity single source, since it typically includes internal detail (exact vulnerable commit, exact patch, exact recovery timeline) that third-party aggregators do not reproduce. This handbook's own bibliography (Appendix E) collects the full source list for every incident cited across all six parts.

---

**Key controls for Part IV**

- Maintain a pre-incident recovery patterns database with applicability, prerequisites, timeline, and precedent recorded for each pattern before it is needed.
- Determine atomicity first: a single-transaction exploit has no on-chain recovery window, which redirects effort immediately to negotiation, compensation, or law enforcement.
- Screen every counterparty address against sanctions lists before any negotiation, bounty offer, or payout.
- Treat pause as reversible and cheap; treat every upgrade as requiring fork-tested validation against the real exploit transaction before execution.
- Adopt a Whitehat Safe Harbor Agreement and publish a verified recovery address before an incident, not during one.
- Preserve the forensic evidence chain from Parts II and III unmodified through negotiation, compensation, or fork, regardless of which pattern is used or how the incident resolves.

# Part V: Tooling and Automation

Part V catalogs the concrete tools a responder reaches for once Parts II through IV have defined what to collect and how to reason about it, then shows how to wire those tools into scripts and pipelines so evidence collection survives a second incident without being reinvented from scratch. Nothing here replaces judgment: a decompiler surfaces bytecode and a risk score flags an address, but the analyst still decides what the evidence means. The chapters move from the individual tool (Chapter 19) to the repeatable process that turns a one-off investigation into a standing capability (Chapter 20).

## 19. Public and Open-Source Tools

The EVM forensic toolchain splits into four functions: reading what happened (explorers and indexers), reconstructing exactly how it happened (tracing and debugging), watching for it happening again in real time (risk scoring), and gluing the first three together with code you control (scripting). Part III's transaction and flow analysis chapter puts these tools to work inside a live investigation; this chapter is the reference catalog they draw from, and it feeds directly into Appendix D.

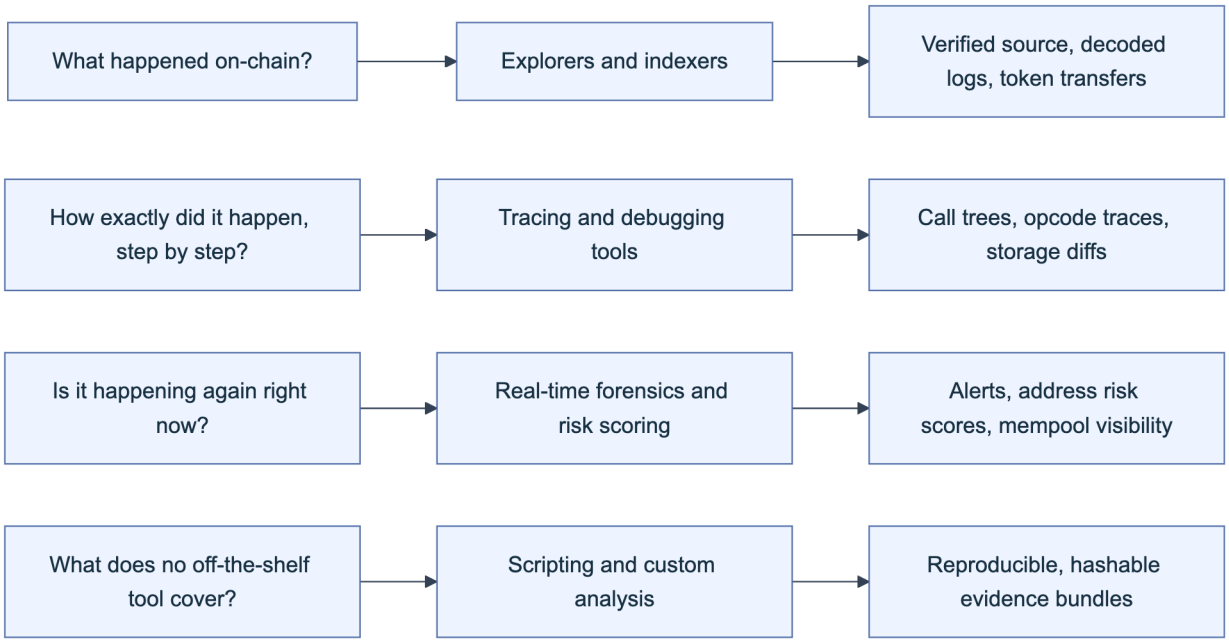


*Figure 1. The forensic tool landscape organized by investigative question rather than by vendor. Each category answers a different question and produces a different class of evidence artifact.*

## 19.1 Explorers and Indexers

An explorer is usually the first tool a responder opens, because it turns raw remote procedure call (RPC) node output into something readable: verified source code, decoded event logs, a token transfer list, and an internal transaction tree assembled from the client's own trace engine. Etherscan remains the default for Ethereum mainnet and maintains matching instances for most major EVM chains (Arbiscan, BscScan, Basescan, Polygonscan among them), plus a unified multichain search that lets a responder pivot across chains from one query. Its API is the backbone of most automation described later in this Part, and a paid key raises the rate limit past what free-tier scripting hits during a fast-moving incident.

Not every chain has an Etherscan instance, and that is where Blockscout matters: it is open source (GitHub), self-hostable, and has become the default explorer for many L2s and app-specific chains that never signed an Etherscan deal. Otterscan solves a different problem. Built to run against an Erigon archive node, it gives you a fully local, unlimited-history explorer with no query going to a third party, which matters when the address you are quietly watching is itself sensitive investigative context you do not want to leak through a public API's access logs. For flow analysis across thousands of transactions rather than one, Dune Analytics exposes indexed chain data through SQL, and The Graph lets you query structured historical events through a subgraph, both stepping past the block-range limits that a plain `eth_getLogs` call runs into on a busy contract.

| Tool | Type | Access | Best for |
|---|---|---|---|
| Etherscan (+ chain equivalents) | Hosted explorer and API | Free tier, paid API key | Fast triage, verified source, decoded events |

| Tool | Type | Access | Best for |
|---|---|---|---|
| Blockscout | Open source, self-hostable | Free (MIT) | Chains without a native Etherscan instance |
| Otterscan | Open source, self-hosted (Erigon) | Free, run locally | Query privacy, unlimited history, no rate limits |
| Dune Analytics | Hosted SQL over indexed chain data | Free tier, paid plans | Cross-transaction flow analysis, reusable dashboards |
| The Graph | Decentralized indexing (subgraphs) | Free, query fees | Structured historical event queries at scale |

## 19.2 Tracing and Debugging Tools

An explorer tells you a transaction reverted or moved value; a tracer tells you exactly which internal call, at which opcode, made it happen. Tenderly offers both: a step debugger that walks a transaction opcode by opcode with the full stack and storage visible, and a fork feature that lets you replay the exact attack against a copy of mainnet state at the block before it hit, safely and repeatably. BlockSec's Phalcon explorer renders the same kind of trace as a human-readable call tree with fund flow highlighted, which is often faster for a first read of an unfamiliar exploit than a raw opcode dump.

For a scriptable, local-first option, Foundry's `cast` and `anvil` (GitHub) fork a chain at a specific block and replay a transaction against it:

```
# Fork mainnet at the block immediately before the exploit transaction
anvil --fork-url $RPC_URL --fork-block-number 18500000

# Replay the transaction and open Foundry's interactive opcode debugger
cast run 0xEXPLOIT_TX_HASH --rpc-url http://127.0.0.1:8545 --debug
```

Underneath most of these tools sits the same primitive: Geth's `debug` JSON-RPC namespace (Geth docs), which every major client (Geth, Erigon, Reth, Nethermind) implements in compatible form.

```
{
  "jsonrpc": "2.0", "id": 1,
  "method": "debug_traceTransaction",
  "params": ["0xEXPLOIT_TX_HASH", {"tracer": "callTracer", "tracerConfig": {"onlyTop-
        Call": false}}]
}
```

Pair `callTracer` (the nested call tree, with gas, input, and output at every frame) with `prestateTracer` (a dump of every account and storage slot the transaction touched *before* execution) to capture the exact pre-attack state referenced in Part II, Section 4.2.

## 19.3 Real-Time Forensics and Risk Scoring (EVM-Specific)

Part III's real-time forensics and mempool monitoring section covers the methodology of watching an incident unfold; this section is the tool catalog behind it. Forta Network, originally built out of the OpenZeppelin security team before spinning out as an independent foundation, runs a decentralized network of detection bots that score transactions against known attack signatures across many EVM chains in real time and can be self-hosted or subscribed to. OpenZeppelin's hosted Defender platform historically paired Sentinel monitoring with Autotasks, but the company retired Defender in favor of open-source tooling. Its successors are OpenZeppelin Monitor, for condition-based on-chain monitoring and notifications, and OpenZeppelin Relayer, for policy-controlled transaction submission and automation that teams operate within their own infrastructure.

On the compliance and attribution side, Chainalysis combines its graph-based Reactor investigation tool with a Know Your Transaction (KYT) API for real-time address risk scoring, and reportedly acquired the Web3 threat-detection firm Hexagate in 2024 to add pre-transaction simulation to that stack. TRM Labs and Elliptic offer comparable wallet-screening and investigation tooling used widely by exchanges and incident responders. For maximal extractable value (MEV) and price-manipulation-specific forensics, EigenPhi labels sandwich attacks, arbitrage, and exploit transactions with dollar values in near real time, and Blocknative and Flashbots provide mempool visibility that lets you see a pending attack before it lands, not just reconstruct it after.

| Tool | Category | Primary signal |
|---|---|---|
| Forta Network | Decentralized bot detection | Real-time transaction anomaly alerts |
| OpenZeppelin Monitor and Relayer | Monitoring and auto-response | Condition-based alerts and policy-controlled transaction automation |
| Chainalysis Reactor / KYT | Investigation and risk scoring | Address clustering, transaction risk score |
| TRM Labs, Elliptic | Blockchain intelligence | Wallet screening, sanctions exposure |
| EigenPhi | MEV and exploit analytics | Dollar-labeled arbitrage, sandwich, exploit txs |
| Blocknative, Flashbots | Mempool visibility | Pending-transaction alerting before inclusion |

## 19.4 Scripting and Custom Analysis

Off-the-shelf explorers and tracers cover a single transaction well; a multi-step, multi-contract exploit that unfolds across dozens of transactions and several blocks usually needs a script that stitches the pieces together, because no dashboard was built for your specific incident. Python with `web3.py` or JavaScript with `ethers.js` or `viem` are the common bases, calling the same RPC methods the explorers use under the hood but composed into a purpose-built pipeline.

Two recurring obstacles justify their own tools. First, attackers rarely verify their exploit contract's source, so you are reading raw bytecode. heimdall-rs, a Rust-based EVM decompiler and calldata decoder, and Dedaub's decompiler and security suite both turn unverified bytecode back into readable pseudocode, and evm.codes is the opcode-and-gas-cost reference you keep open while reading the result by hand. Second, an unknown four-byte function selector in a trace is opaque until decoded; 4byte.directory and OpenChain's signature database crowdsource that mapping, and Foundry's `cast` calls it directly:

```
# Decode an unknown selector from a trace
cast 4byte 0x23b872dd
# -> transferFrom(address,address,uint256)

# Independently verify deployed bytecode against source, rather than trusting
# an explorer's verification checkmark alone
# see https://sourcify.dev
```

Sourcify is worth calling out specifically: it verifies bytecode against source through a decentralized, IPFS-backed process independent of any single explorer, and re-running that verification yourself is a cheap check against a compromised or stale explorer record.

---

# 20. Automation and Repeatability

Tools only produce usable evidence if the process around them survives a 3 a.m. page and a stressed responder. Automation turns an ad hoc investigation into a rehearsed capability: the same evidence bundle, collected the same way and hashed the same way, every time, which is what makes the output usable later in a legal or negotiation context per Part I, Section 2.1.

## 20.1 Scripts for Common Analyses

The highest-value scripts are the ones you run in the first ten minutes of every incident, before analysis diverges by case. A minimal evidence-snapshot script wraps the RPC calls from Chapter 19 into one command, hashes every output file, and writes a manifest that timestamps the collection:

```bash
#!/usr/bin/env bash
set -euo pipefail
TX=$1; RPC=$2; OUT="evidence_${TX:0:10}_$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$OUT"

cast tx "$TX" --rpc-url "$RPC" > "$OUT/tx.json"
cast receipt "$TX" --rpc-url "$RPC" > "$OUT/receipt.json"
cast run "$TX" --rpc-url "$RPC" --quick > "$OUT/trace.txt"

BLOCK=$(cast tx "$TX" --rpc-url "$RPC" --json | jq -r '.blockNumber')
for ADDR in $(jq -r '.to, (.logs[]?.address)' "$OUT/receipt.json" | sort -u); do
  cast code "$ADDR" --rpc-url "$RPC" --block "$BLOCK" > "$OUT/code_${ADDR}.hex"
done

sha256sum "$OUT"/* > "$OUT/manifest.sha256"
echo "Snapshot written to $OUT, manifest hashed."
```

This script is deliberately narrow: it collects, it hashes, and it stops, in keeping with the evidence-first mindset of Part I, Section 3.1. Build a small library of these (a token-approval-flow extractor, a cross-contract call-graph walker, a bridge-message correlator) rather than one large tool, so a responder under pressure can reach for the one script that matches the question in front of them instead of learning a monolith's flags mid-incident.

## 20.2 Checklists for Consistent Evidence Collection

A script guarantees a command ran the same way twice; a checklist guarantees a human did not skip a step under pressure. Keep this short version at hand when kicking off a snapshot script, and treat the full version in Appendix B as the audit-ready reference:

- ☐ Confirm the RPC endpoint is an archive node (state at old blocks is unavailable on a pruned full node).
- ☐ Record the exact block number and timestamp before running any script, not after.
- ☐ Run the snapshot script and verify the manifest's hash count matches the file count.
- ☐ Store the output in write-once or otherwise tamper-evident storage per Part II, Section 5.3.
- ☐ Log who ran the script, from which machine, and why, as a chain-of-custody entry.
- ☐ Re-verify contract bytecode against source independently (Sourcify or a second explorer) rather than trusting one verification badge.
- ☐ Note every tool version used (client version, `cast` version, tracer name) so results are reproducible later.

Consistency matters more than exhaustiveness here: a checklist that is actually followed every time beats a longer one that gets skipped on the incidents where time pressure is highest, which is exactly when the evidence matters most.

## 20.3 Integration with Incident Response Pipelines

Tooling delivers the most value when detection, evidence collection, and human escalation are wired together rather than triggered manually in sequence. A typical pipeline routes a Forta or Defender alert into a ticketing or paging system, fires an automated snapshot script the moment the alert lands (before anyone has read it), and only then hands the packaged evidence to a human, who decides whether to escalate into the fuller process owned by the Incident Response Handbook (06).

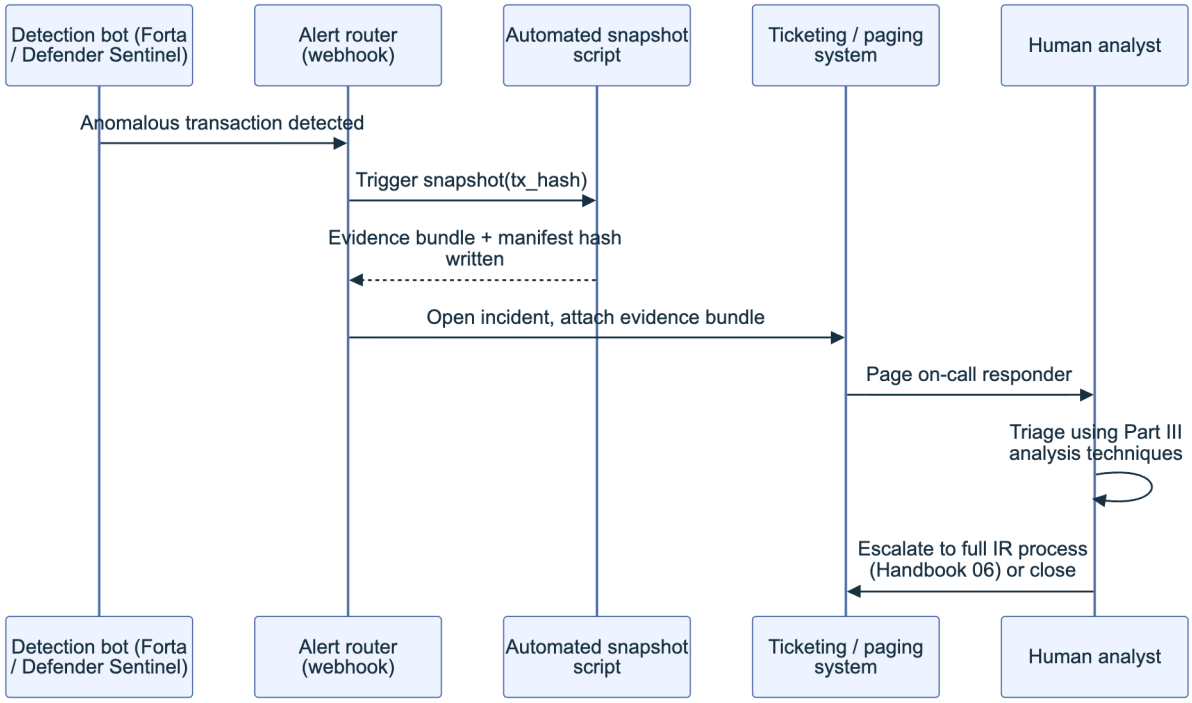


*Figure 2. An automated detection-to-triage pipeline. The snapshot fires before human triage begins, so the evidence exists even if the analyst is slow to respond, and the manifest hash from Section 20.1 travels with the ticket as its integrity proof.*

| Alert source | Automated action | Escalation trigger |
|---|---|---|
| Forta bot finding | Run snapshot script, open ticket | Any Critical or High severity finding |
| Defender Sentinel condition | Run snapshot script, optional Autotask pause | Contract balance or supply moves past threshold |
| Chainalysis/TRM risk alert | Log address, no auto-pause | Sanctioned or high-risk counterparty match |
| Manual analyst report | Run snapshot script on demand | Any suspected active exploit |

Keep the automated leg conservative: auto-pausing a contract is powerful but not free, since a false positive that halts a live protocol is its own incident, so reserve auto-response for high-confidence signals (a known drain signature, a supply invariant broken) and default to alert-plus-snapshot for everything else, leaving the pause decision to a human who can weigh the false-positive cost in seconds rather than in a postmortem.

**Key controls for Part 5**

- Run explorers and tracers you control (Otterscan, a forked Foundry `anvil` instance) alongside hosted ones, so an active investigation never leaks its query pattern to a third party.
- Capture both `callTracer` and `prestateTracer` output for every exploit transaction to fix the exact pre-attack state, not just the call sequence.
- Independently re-verify bytecode against source (Sourcify or a second explorer) rather than trusting a single verification badge.
- Script the first ten minutes of every incident: snapshot, hash, manifest, before analysis diverges by case.
- Wire detection tools (Forta, Defender Sentinel) to fire an automated evidence snapshot before human triage begins, and reserve automated response actions for high-confidence signals only.

# Part VI: Appendices

This part collects the reference material the rest of the handbook assumes you have on hand during an actual incident, when there is no time to hunt through six parts for a definition or a checklist. Appendix A fixes the vocabulary spanning forensics, on-chain analysis, and recovery law. Appendix B turns the evidence discipline from Parts I and II into a checklist you run the moment an exploit is confirmed. Appendix C compares the six recovery patterns from Part IV, Appendix D indexes the tools named throughout Parts III and V, and Appendix E names the research the four-tier framework and the bridge/oracle taxonomies draw from, then points to the full bibliography.

## 21. Appendix A: Glossary

Entries are grouped by where the term does the most work in the incident lifecycle, not strict alphabetical order, since a responder pulling evidence at hour one and a governance lead negotiating recovery in week three reach for different vocabulary. Within each group, entries run alphabetically, acronyms expand on first appearance, and each entry names the fuller chapter treatment.

### 21.1 Forensics, Evidence, and Legal Terms

- **Admissibility**: the standard evidence must meet before a court accepts it. Electronic evidence self-authenticates under FRE 902(13) and 902(14), covering system-generated records and hash-verified copies. Chapter 2.1.
- **Chain of custody**: the unbroken, documented record of who collected an item of evidence, when, and who has held it since, proving the artifact in hand was not altered. Chapter 2.1.
- **Digital forensics**: identifying, preserving, analyzing, and presenting digital evidence, formalized for responders in NIST SP 800-86 (2006). Chapter 1.1.
- **Evidence-first mindset**: capturing and hashing evidence before taking any action that could change the state you are trying to document. Chapter 3.1.

- **Mutual Legal Assistance Treaty (MLAT)**: a state-to-state agreement for exchanging evidence in criminal matters, the usual, slow path for compelling an offshore exchange to freeze an account. Chapter 2.3.
- **Responsible disclosure**: privately reporting a vulnerability or an active exploit's details to the affected project only, timed to avoid tipping off the threat actor. Chapter 2.4.
- **Write-once, tamper-evident storage**: a Write Once Read Many (WORM) configuration or append-only ledger preventing an already-written record from being silently modified. Chapter 5.3.

## 21.2 On-Chain Evidence and Analysis Terms

- **Address clustering**: a heuristic grouping addresses as likely controlled by one entity, from signals such as shared funding, co-spending, or shared infrastructure. Chapter 9.1.
- **Archive node**: a full node retaining complete historical state for every block rather than pruning it, required for a balance, call, or trace query against an arbitrary past block. Chapter 4.2.
- **Call tracer**: a `debug_traceTransaction` mode returning a transaction's internal call tree, every `CALL`, `DELEGATECALL`, `STATICCALL`, and `CREATE`, instead of a raw opcode log. See the Geth debug namespace, Chapter 7.1.
- **Externally Owned Account (EOA)**: an account controlled by a private key, the only account type able to originate a transaction; distinguishing it from a contract account tells you whether you chase a key or a program. Chapter 4.4.
- **Finality**: the point past which a block is guaranteed not to be reverted by a reorganization, past which citing "block N" stops being ambiguous. Chapter 3.2.
- **Internal transaction**: a value transfer or call made by contract code during a top-level transaction, never independently broadcast, visible only by tracing that transaction. Chapter 4.3.
- **Tactics, Techniques, and Procedures (TTP)**: a threat actor's reusable behavioral fingerprint, tool choices, mixer habits, timing, deployment style, used to link unconnected exploits to one operator. Chapter 9.2.

## 21.3 Root Cause and Vulnerability Terms

- **Four-tier root cause framework**: exploit causes classified as protocol logic flaws (Tier 1), lifecycle and governance issues (Tier 2), external dependencies (Tier 3), and classic implementation bugs (Tier 4). Chapter 1.5, Section 25.1.1.
- **Access control flaw**: a function that should be role-restricted but is missing, or misapplies, that check, letting an unauthorized caller execute privileged logic. Chapter 8.3.
- **Flash loan**: an uncollateralized loan borrowed and repaid within one transaction; not itself a vulnerability, but the enabler letting an attacker briefly command capital enough to move a price or a vote. Chapter 8.5.

- **Invariant**: a property a protocol assumes always holds, supply equal to the sum of balances, a pool's constant-product relationship, whose violation often signals an in-progress exploit. Chapter 8.4.
- **Oracle manipulation**: distorting the price or data feed a contract relies on, by moving a thin pool an oracle reads directly or exploiting a feed update's timing. Chapter 8.5.
- **Reentrancy**: a contract making an external call before finalizing its own state, letting the called contract call back in and execute the original function again against stale state. Chapter 8.3.
- **SCWE / SCSVS**: OWASP's Smart Contract Weakness Enumeration (150+ classes) and companion Security Verification Standard (eleven control categories), mapping a root cause to the control that should have prevented it. See scs.owasp.org.

## 21.4 Bridge, Cross-Chain, and Multi-Chain Terms

- **Atomic execution**: a transaction, or bundled set, completing entirely or reverting entirely, leaving no window between an exploit's trigger and completion for a defender to act. Chapter 10.2.
- **Cross-chain bridge**: infrastructure moving an asset or message between incompatible chains, typically locking or burning on the source chain and minting or releasing on the destination. Chapter 10.1.
- **Layer 2 (L2)**: a chain executing transactions off the base layer and periodically committing state or proofs back to it, inheriting some of that layer's security. Chapter 10.4.
- **Lock-and-mint bridge**: a design locking the source asset and minting a wrapped, IOU-style token on the destination, redeemed by burning it to unlock the original. Chapter 10.1.
- **Real-time forensics**: monitoring live chain data, pending mempool transactions and fresh blocks, to detect or interrupt an exploit while still in progress. Chapters 10.5, 19.3.
- **Rescue window**: for a non-atomic attack, the interval between the first observable transaction and final extraction, during which a fast responder can pause or front-run to blunt the loss. Chapter 10.2.1.
- **Validator trust model**: the parties, fixed multisig, rotating validator set, light client, whose attestations a bridge accepts as proof of a source-chain event, its real security boundary. Chapter 10.1.1.

## 21.5 DeFi Recovery and Governance Terms

- **Multisignature (multisig) wallet**: a contract account requiring a threshold of independent signers, the standard custody model for a treasury or admin key and a precondition for most recovery patterns. Chapter 13.1.
- **Pausable contract**: a contract with a circuit-breaker modifier a privileged role can trigger to halt functions, typically deposits and withdrawals, during an incident. Chapter 12.1.

- **Safe Harbor Agreement**: a legal and on-chain framework from the Security Alliance (SEAL) under which a protocol pre-authorizes whitehat rescue within defined terms, recorded in an on-chain registry before any incident. See the SEAL Safe Harbor repository, Chapter 17.1.
- **Social or state fork**: deploying a new version of the protocol, in the extreme a new chain, that excludes exploited state, coordinated as governance and communications, not a code fix alone. Chapter 15.1.
- **Timelock**: a contract enforcing a mandatory delay between a governance action being queued and executed, giving holders a window to react before it takes effect. Chapter 13.1.
- **Treasury**: a protocol's own reserve under governance or multisig control, one possible, not guaranteed, source for compensating users. Chapter 14.2.
- **Upgradeable proxy**: a fixed proxy address delegating logic to a replaceable implementation contract, letting a protocol ship a fix without migrating every user. Chapter 12.1.
- **Whitehat rescue**: an unsolicited but well-intentioned intervention during an active exploit, moving at-risk funds to safety using the same vulnerability an attacker is exploiting. Chapter 17.1.

# 22. Appendix B: Evidence Collection Checklist

Run this checklist the moment an exploit is confirmed, not once it is fully understood; evidence uncaptured in the first hour is often gone by the second. It operationalizes the collection duties in Chapters 4 through 6 and pairs with the general triage process in the Incident Response Handbook (06).


*Figure 1. The evidence collection sequence. Skipping a stage to save time usually costs more time later reconstructing what was lost.*

## 22.1 Immediate Capture Checklist (First Hour)

The first hour sets the ceiling on everything an investigation can later prove. Before touching an admin key, pausing a contract, or posting a public statement, get the exploit transaction hash, the affected addresses, and a screenshot of the public narrative on record. Every minute spent debating the response before that baseline exists is a minute an attacker cleaning a trail can erase first.

**Immediate Capture Checklist**

- ☐ Record the exploit transaction hash(es) and block number(s) as soon as identified
- ☐ Take a timestamped screenshot of the first public report (Twitter/X, Discord, forum)
- ☐ Do NOT interact with attacker-controlled or exploited contracts before evidence is captured
- ☐ Notify the incident response lead and legal counsel per Chapter 2.3
- ☐ Start a running, timestamped incident log (Chapter 6.3) before any other action
- ☐ If a pause or emergency action is warranted, capture pre-action state first (Chapter 3.1)

## 22.2 On-Chain Evidence Checklist

On-chain evidence does not disappear, but the convenience of pulling it does: rate-limited APIs, pruning nodes, and explorer schema changes can make a query harder tomorrow than today. Pull broadly before you know exactly what you need; a full receipt and trace costs little to store and much to reconstruct later from memory.

**On-Chain Evidence Checklist**

- ☐ Transaction data: hash, block, input calldata, value, gas (Chapter 4.1)
- ☐ Full internal call trace via debug_traceTransaction / callTracer (Chapter 4.1, 7.1)
- ☐ Contract state and storage slots at the block before and after the exploit (Chapter 4.2)
- ☐ Complete event log and internal transaction list for every affected contract (Chapter 4.3)
- ☐ Every EOA and contract address involved, labeled by role (Chapter 4.4)
- ☐ Token transfer and approval history for affected tokens (Chapter 4.5)
- ☐ Cross-chain or bridge-relayed messages, if applicable (Chapter 4.6, Chapter 10.1)
- ☐ Verified source code and ABI (or decompiled bytecode if unverified) for every contract

## 22.3 Off-Chain, Preservation, and Chain-of-Custody Checklist

On-chain data proves what happened; off-chain data proves what the team knew and when. Capture both: a later reconstruction, an insurance claim, a law enforcement referral, a post-mortem, depends on showing the response's timeline and diligence, not just the exploit's mechanics.

**Off-Chain and Preservation Checklist**

- ☐ RPC, indexer, and front-end logs covering the incident window (Chapter 6.1)
- ☐ Monitoring and alert system outputs, including any missed or late alerts (Chapter 6.2)
- ☐ Internal and external communications with timestamps (Chapter 6.3)
- ☐ Deployment artifacts: build hashes, verified compiler settings, deployment scripts (Chapter 6.4)
- ☐ SHA-256 hash of every exported artifact, recorded alongside the artifact itself (Chapter 5.2)
- ☐ All artifacts moved to write-once, access-controlled storage with a documented retention policy (Chapter 5.3, 5.4)
- ☐ Chain-of-custody log started: who collected what, when, and where it is stored (Chapter 2.1)

# 23. Appendix C: Recovery Pattern Summary Table

The six recovery patterns in Part IV are not mutually exclusive or equally available: a protocol without a pre-adopted Safe Harbor agreement cannot retroactively grant one, and a fork rarely suits a loss a treasury can absorb outright. Use the decision flow to find a starting pattern, then the table for what it actually requires.

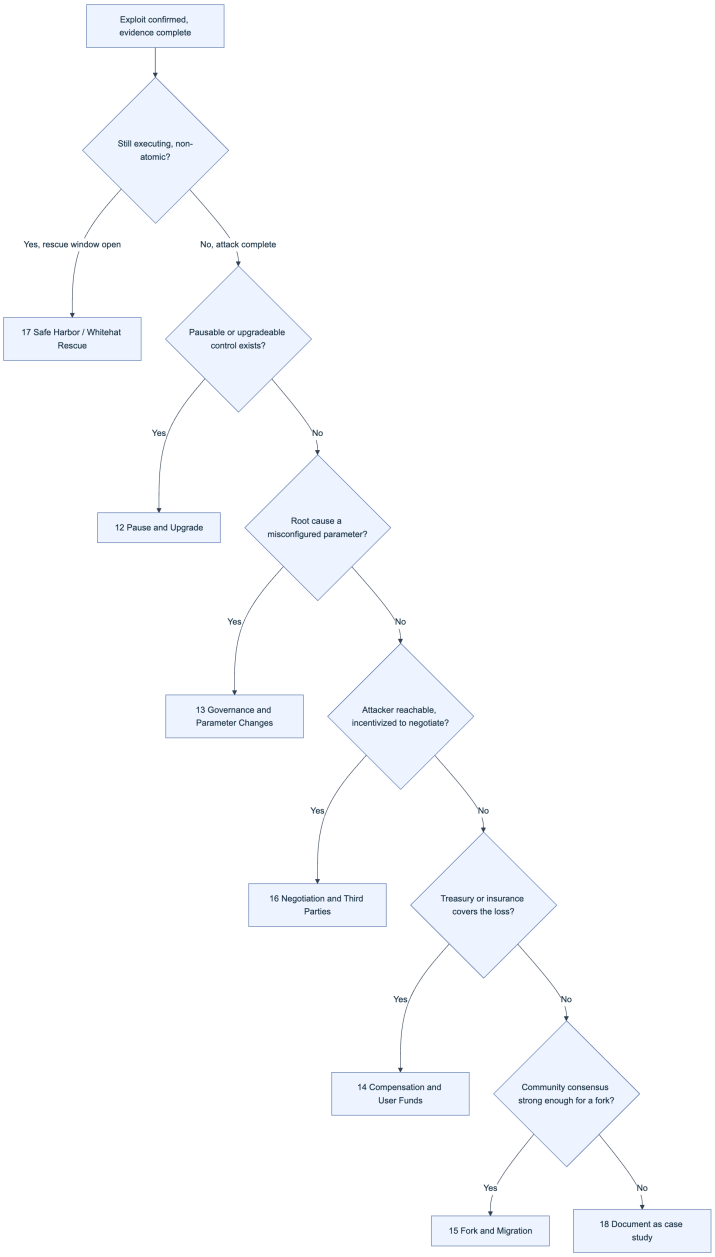


*Figure 2. Recovery pattern selection by incident characteristic. Most incidents apply more than one branch in sequence: a pause (12) buys time to negotiate (16), which buys time to plan compensation (14).*

| # | Pattern | Ch. | Prerequisite | Timeline | Governance | Primary Risk | Precedent |
|---|---|---|---|---|---|---|---|
| 1 | Pause and Upgrade | 12 | Upgradeable proxy or pausable modifier already deployed | Minutes to pause; weeks for a verified upgrade | Admin key or timelock-gated pauser role | Pause key is itself a centralization/ rug-pull surface | |
| 2 | Governance and Parameter Changes | 13 | Root cause is a misconfigured parameter, not a code defect | Hours via emergency multisig; days via timelock vote | Multisig or DAO vote via timelock | Rushed emergency votes bypass normal review | |
| 3 | Compensation and User Funds | 14 | Treasury or insurance capital sufficient to cover the loss | Weeks to identify users and distribute | Treasury release needs governance/ multisig approval | Distribution-fairness disputes; tax/securities exposure varies | Sky Mavis reimbursed users after the March 2022 Ronin hack |
| 4 | Fork and Migration | 15 | Loss and alignment large enough to exclude exploited state | Weeks to months | Broad community and validator/ miner consensus | Splits the community and the asset | |
| 5 | Negotiation and Third Parties | 16 | Attacker reachable, incentivized to negotiate over prosecution risk | Days to weeks | Team or counsel authorizes bounty terms | Normalizes theft-then-bounty as a model | Poly Network (Aug 2021, ~$611M, recovered by Aug 25); Euler (Mar 2023, ~$197M) |
| 6 | Safe Harbor and Whitehat Recovery | 17 | Protocol pre-adopted a Safe Harbor agreement | Real time, during the exploit | On-chain registry adoption, decided in advance | Does not shield a rescuer from every jurisdiction's law | SEAL Safe Harbor registry |
| 7 | Other Patterns and Case Studies | 18 | Hybrid or novel responses outside the six patterns above | Varies | Varies | Varies | See Chapter 18.1 |

# 24. Appendix D: Tool References and Links

The tools below are the working set referenced across Chapters 7, 9, 16, 17, and 19. None substitutes for the methodology in those chapters, but each removes hours of manual work from a step. Prefer tools that expose raw, exportable data (a JSON trace, a CSV of transfers) over ones that only render a dashboard, since raw exports go into your evidence archive (Chapter 5).

| Tool | Category | Primary Use | Chapter | Link |
|---|---|---|---|---|
| Etherscan (V2 API) | Explorer / Indexer | Verified source, ABI decoding, multi-chain queries under one key | 4.1, 19.1 | etherscan.io |
| Blockscout | Explorer / Indexer | Open-source explorer, common default for L2s and alt-EVM chains | 10.4, 19.1 | blockscout.com |
| Dune Analytics | Explorer / Indexer | SQL-queryable indexed data, custom fund-flow dashboards | 7.3, 19.1 | dune.com |
| Tenderly | Tracing / Debugging | Simulation, mainnet forking, call-tree debugger | 7.4, 8.6 | tenderly.co |
| Foundry (cast, anvil) | Tracing / Debugging | CLI forking, tracing, scripted exploit reproduction | 8.6, 19.4 | book.getfoundry.sh |
| BlockSec Phalcon | Tracing / Debugging | Trace explorer with visual fund-flow mapping | 7.1, 19.2 | phalcon.blocksec.com |
| Forta Network | Real-Time Forensics | Decentralized real-time threat-detection bots | 10.5, 19.3 | forta.org |
| OpenZeppelin Monitor and Relayer | Real-Time Forensics | Self-hosted monitoring, notifications, and on-chain response automation | 10.5, 20.3 | OpenZeppelin Monitor |
| Chainalysis Reactor | Attribution / Compliance | Address clustering, attribution investigation | 9.1, 9.3 | chainalysis.com |
| TRM Labs | Attribution / Compliance | Blockchain intelligence, sanctions screening | 9.1, 9.3 | trmlabs.com |
| Arkham Intelligence | Attribution / Compliance | Intelligence platform, crowd-sourced entity labels | 9.1, 9.4 | intel.arkm.com |
| SlowMist MistTrack | Attribution / Compliance | Address tracing and fund-flow tracking | 9.1, 10.3 | misttrack.io |
| 4byte.directory | Signature Recovery | Public function-selector signature database | 4.1, 7.1 | 4byte.directory |
| OpenChain | Signature Recovery | Crowd-sourced selector/event signature lookup | 4.1, 7.1 | openchain.xyz |
| Dedaub | Signature Recovery | Decompiler and security suite for unverified bytecode | 6.4, 8.1 | dedaub.com |
| Sourcify | Signature Recovery | Decentralized source verification, metadata matching | 6.4 | sourcify.dev |
| SEAL 911 | Recovery / Coordination | Emergency responder directory for whitehat rescuers | 16.1, 17.1 | security-alliance.org |
| Immunefi | Recovery / Coordination | Bug bounty platform, also broadcasts bounty/negotiation terms | 16.1 | immunefi.com |

# 25. Appendix E: References and Further Reading

## 25.1 Research and Standards

The four-tier framework in Chapter 1.5 and the bridge/oracle taxonomies in Chapters 8 and 10 are not this handbook's invention; they synthesize systematic literature, post-mortem data, and named incident reporting. This section names the specific research drawn on; the full citation list lives in `references.md`, described below.

### 25.1.1 Four-Tier Root Cause and Exploit-Chain Literature

The clearest academic anchor for a systematic DeFi root cause taxonomy is Zhou et al., "SoK: Decentralized Finance (DeFi) Attacks" (IEEE S&P, 2023), built from 77 papers, 30 audit reports, and 181 real-world incidents. It found price oracle attacks and permissionless-interaction abuse the most frequent categories, ahead of the code-level bugs earlier surveys emphasized, justifying this handbook's treatment of Tier 3 (external dependencies) as first-class rather than a footnote to Tier 4. The exploit-chain framing in Chapter 1.5.6 reflects the same paper's finding that academic and practitioner models diverge most on multi-step compound attacks, the category single-function audits catch least. OWASP's Smart Contract Weakness Enumeration supplies the taxonomy Chapter 8.2 maps Tier 4 findings against.

### 25.1.2 Bridge and Oracle Attack Taxonomies

Bridges get dedicated treatment in Chapter 10 because the record justifies it: Chainalysis's 2022 analysis found thirteen bridge hacks had accounted for roughly $2 billion in losses by August 2022, 69% of that year's stolen funds, making bridges DeFi's largest structural loss category. Lee, Murashkin, Derka, and Gorzny, "SoK: Not Quite Water Under the Bridge" supplies the architectural breakdown behind Chapter 10.1.1's validator-trust-model framing, decomposing a bridge into its lock/mint mechanism, relay layer, and validation logic, and showing most major hacks targeted validation, not token custody. Oracle manipulation (Chapter 8.5) draws on the same Zhou et al. taxonomy above; both attack classes compromise a value a contract trusts without independently re-deriving it.

### 25.1.3 Real-World Incident RCAs and Forensic Reports

Case-level grounding draws on public root-cause analyses (RCAs) and incident journalism, not secondhand summaries. rekt.news functions as a standing incident wire, publishing technical post-mortems within days of major exploits, including the Euler Finance hack of March 2023. Where a project publishes its own post-mortem, this handbook prefers that primary source, supplemented by independently verified reporting such as the Wikipedia record of the FBI's April 2022 attribution of the Ronin Bridge hack to the Lazarus Group. The Poly Network exploit of August 2021 recurs in Chapter 16 as the reference case for attacker negotiation: its full timeline,

from theft to recovery fourteen days later, is unusually well documented on-chain and in contemporaneous reporting. Chapter 18.1's case catalog names further incidents, each linking its own primary RCA.

---

`references.md` collects every citation from all six parts into one deduplicated bibliography: Standards and Frameworks (OWASP SCSVS, SCSTG, SCWE; NIST SP 800-86; the Federal Rules of Evidence; MITRE ATT&CK and AADAPT), Incidents and Case Studies (Poly Network, Ronin Bridge, Euler Finance, Chapter 18.1's catalog), Tools (Appendix D plus Part V), and Further Reading (Section 25.1's literature). If a link goes stale, check there before assuming the claim no longer holds. Add each new source in the same change that cites it.

---

**Key controls for Part 6**

- Treat Appendix A (21) as the canonical vocabulary; link to it from tickets and post-mortems instead of redefining a term each time.
- Run the immediate capture checklist (22.1) within the first hour of any confirmed exploit, before any pause, upgrade, or public statement.
- Keep the evidence checklists (22.2, 22.3) synchronized with the collection guidance in Chapters 4 through 6.
- Walk the recovery pattern decision flow (23) before committing to a fork, negotiation, or compensation plan; most incidents combine patterns in sequence.
- Pre-adopt a Safe Harbor agreement (24, 17) before an incident, not during one; it only protects rescue activity registered in advance.
- Add every new citation to `references.md` in the same change that introduces it (25).

# Appendix C: References and Further Reading

This appendix consolidates every external source cited across the handbook's parts, grouped by category for quick lookup.

## Standards and Frameworks


- MITRE ATT&CK
- MITRE AADAPT framework
- MITRE AADAPT cyber threat framework fact sheet
- NIST Cybersecurity Framework 2.0
- NIST SP 800-61 Revision 3, Incident Response Recommendations and Considerations
- NIST SP 800-86, Guide to Integrating Forensic Techniques into Incident Response
- RFC 3161, Time-Stamp Protocol
- RFC 3227, Guidelines for Evidence Collection and Archiving
- Federal Rule of Evidence 901
- Federal Rule of Evidence 902
- General Data Protection Regulation (GDPR)
- ERC-20 Token Standard
- ERC-721 Non-Fungible Token Standard
- ERC-1155 Multi-Token Standard
- ERC-1967 Proxy Storage Slots
- OWASP Smart Contract Security project
- OWASP Web3 Attack Vectors Top 15
- OFAC Sanctions List Search
- OFAC recent sanctions actions


## Incidents and Case Studies


- rekt.news
- rekt.news leaderboard
- rekt.news, Beanstalk
- rekt.news, Curve
- rekt.news, Euler
- rekt.news, Mango Markets
- rekt.news, Nomad

- rekt.news, Poly Network
- rekt.news, Rari Capital
- rekt.news, Ronin
- rekt.news, Wormhole
- DeFiLlama hack tracker
- Poly Network exploit (Wikipedia)
- Ronin Network (Wikipedia)
- Ethereum Foundation, hard fork announcement (The DAO)
- FBI statement on attribution of malicious cyber activity by the Lazarus Group
- US Treasury press release on Tornado Cash sanctions
- Chainalysis, “2022 Biggest Year Ever for Crypto Hacking”
- Chainalysis, cross-chain bridge hacks 2022 analysis
- Balancer governance forum

## Tools and Documentation

- Etherscan
- Etherscan API docs
- Etherscan API V2 docs
- Blockscout
- Blockscout GitHub repository
- Otterscan
- Sourcify
- Sourcify project introduction
- Dune Analytics
- Dune Analytics docs
- The Graph
- The Graph docs
- EigenPhi
- Tenderly
- Tenderly’s transaction debugger docs
- Foundry
- Foundry book
- Foundry GitHub repository
- Geth debug namespace docs
- Erigon
- heimdall-rs
- EVM opcode reference (evm.codes)
- OpenChain

- OpenChain signature database
- 4byte.directory
- 4byte.directory (.org mirror)
- Chainlist
- ethereum-lists/chains registry
- Chainalysis crypto crime research
- TRM Labs
- Elliptic
- Arkham Intelligence
- Arkham Intel platform
- MistTrack
- BlockSec
- BlockSec's Phalcon
- Dedaub
- Forta Network
- Blocknative's Mempool Explorer
- Flashbots (Protect and MEV-Share)
- L2Beat
- Chainlink data feeds docs
- OpenZeppelin, "Doubling Down on Open Source Tools and Phasing Out Defender"
- OpenZeppelin Pausable docs
- OpenZeppelin TimelockController docs
- OpenZeppelin UUPS docs
- Safe documentation
- OpenTimestamps
- Internet Archive's Wayback Machine
- AWS S3 Object Lock documentation
- Immunefi

## Further Reading


- Zhou et al., "SoK: Decentralized Finance (DeFi) Attacks" (arXiv:2208.13035)
- Lee, Murashkin, Derka, and Gorzny, "SoK: Not Quite Water Under the Bridge" (arXiv:2210.16209)
- Security Alliance (SEAL)
- Security Alliance (SEAL), alternate domain
- SEAL Safe Harbor / Whitehat Safe Harbor Agreement repository